



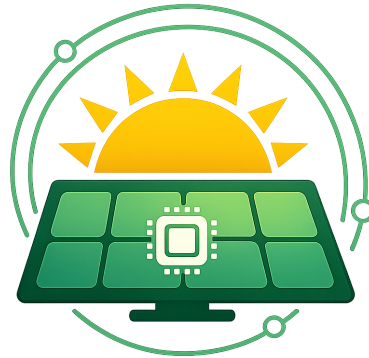
SOLARSENSE: INTELLIGENT SOLAR PANEL MONITORING SYSTEM

WESSAL AHMED ALRABIGHI	2205591
SHAHAD AHMED ALSAMRANI	2205802
JOUD FAHAD ALSULAMI	2206110

**SUPERVISED BY:
DR. SAMAR ALKHURAIJI**

**DEPARTMENT OF COMPUTER SCIENCE
FACULTY OF COMPUTING AND INFORMATION
TECHNOLOGY
KING ABDULAZIZ UNIVERSITY**

May, 2026



SOLARSENSE: INTELLIGENT SOLAR PANEL MONITORING SYSTEM

WESSAL AHMED ALRABIGHI	2205591
SHAHAD AHMED ALSAMRANI	2205802
JOUD FAHAD ALSULAMI	2206110

THIS REPORT IS SUBMITTED IN PARTIAL FULFILMENT
OF THE REQUIREMENTS FOR THE BACHELOR DEGREE
IN COMPUTER SCIENCE

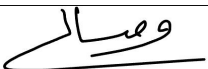
SUPERVISOR NAME: DR. SAMAR ALKHURAIJI

DEPARTMENT OF COMPUTER SCIENCE
FACULTY OF COMPUTING AND INFORMATION
TECHNOLOGY
KING ABDULAZIZ UNIVERSITY

May, 2026

Declaration of Originality

We hereby declare that this project report is based on our original work except for citations and quotations, which had been duly acknowledged. We also declare that it has not been previously and concurrently submitted for any other degree or award at KAU or other institutions.

Student Name	ID	Signature	Date
Wessal Ahmed Alrabighi	2205591		May, 2026
Shahad Ahmed Alsamrani	2205802		May, 2026
Joud Fahad Alsulami	2206110		May, 2026

Acknowledgements

One of the most important stages in our journey toward earning a bachelor's degree is the graduation project. First and foremost, Alhamdulillah, all praise and thanks are due to Allah for granting us the strength, patience, and ability to complete this project successfully. We would like to express our sincere appreciation to the Faculty of Computing and Information Technology and the Computer Science Department for providing us with the academic environment, knowledge, and support that helped us throughout our educational journey and contributed to the successful completion of this project. Our sincere gratitude goes to our supervisor, Dr. Samar Alkhuraiji, for her continuous support, guidance, valuable feedback, and encouragement throughout all stages of the project. Her advice and supervision played a major role in helping us complete this work successfully. We would also like to extend our special thanks to the director of the Center of Research Excellence in Renewable Energy and Power Systems Center, Dr. Sultan Alghamdi, for his support, cooperation, and assistance during the project. His help and encouragement were greatly appreciated. Our appreciation also goes to the project evaluation committee, Dr. Rasha Mirza and Dr. Olaa Alsaedi, for their valuable comments, constructive feedback, and suggested improvements that helped enhance the quality of our project. Special thanks to the graduation project coordinators, Dr. Helen Baksh, Dr. Nouf Abukhodair, and Dr. Rania Molla, for their continuous efforts, organization, clear instructions, and support throughout the graduation project process. We are also thankful to all participants who took part in testing our website and providing feedback, as their participation contributed significantly to improving the system and evaluating its usability. In addition, we would like to appreciate ourselves as team members for the dedication, teamwork, commitment, and countless hours of effort invested throughout this journey. Completing this project required patience, cooperation, problem solving, and continuous learning, and we are proud of what we achieved together as a team. Finally, we would like to express our deepest gratitude to our families and friends for their endless love, patience, motivation, and emotional support throughout this journey. Their encouragement gave us the strength to continue and complete this project successfully.

Abstract

Solar energy has become one of the most important renewable energy sources due to its environmental and economic benefits. However, maintaining solar panel efficiency and detecting faults in real time remain major challenges, especially in large-scale solar installations. Traditional monitoring methods often require manual inspection, which can be time-consuming, costly, and inefficient. Therefore, intelligent monitoring systems are needed to improve fault detection, performance analysis, and maintenance processes.

Based on the increasing adoption of artificial intelligence and smart technologies, this project presents SolarSense, an intelligent solar panel monitoring system that combines Internet of Things, machine learning, computer vision, and web technologies into one integrated platform. The system is designed to monitor solar panel conditions, analyse telemetry data, detect possible anomalies, and assist users in identifying panel faults through a real-time dashboard.

SolarSense consists of multiple integrated components. The hardware side includes an ESP32-based sensing unit connected to voltage, current, temperature, and light sensors for collecting real-time telemetry data from solar panels. The software side includes a front-end dashboard, a back-end API, a database system, and AI-based analysis modules. The machine learning component analyses telemetry data to identify abnormal behaviour, while the computer vision component analyses solar panel images for fault detection and condition monitoring.

The developed platform supports real-time monitoring, telemetry visualisation, AI-based analysis, and image inspection through an interactive dashboard interface. Different testing methods were performed during development, including unit testing, integration testing, hardware testing, AI model testing, and usability testing to validate the functionality and integration of the system components.

SolarSense demonstrates how intelligent technologies can be integrated into renewable energy systems to improve monitoring efficiency, support maintenance operations, and provide a scalable foundation for future smart solar management solutions.

المُستخلص

تُعد الطاقة الشمسية من أهم مصادر الطاقة المتجددة لما توفره من فوائد بيئية واقتصادية. ومع ذلك، لا تزال المحافظة على كفاءة الألواح الشمسية واكتشاف الأعطال بشكل فوري من التحديات الرئيسية، خصوصًا في الأنظمة الشمسية الكبيرة. وتعتمد طرق المراقبة التقليدية غالبًا على الفحص اليدوي، مما يجعلها مستهلكة للوقت والتكلفة وأقل كفاءة. لذلك ظهرت الحاجة إلى أنظمة ذكية تسهم في تحسين اكتشاف الأعطال وتحليل الأداء وتسهيل عمليات الصيانة.

وانطلاقًا من التوجه المتزايد نحو استخدام تقنيات الذكاء الاصطناعي والتقنيات الذكية، يقدم هذا المشروع نظامًا ذكيًا لمراقبة الألواح الشمسية يجمع بين إنترنت الأشياء والتعلم الآلي والرؤية الحاسوبية وتقنيات الويب ضمن منصة متكاملة. ويهدف هذا النظام إلى مراقبة حالة الألواح الشمسية، وتحليل بيانات القراءات، واكتشاف المشكلات المحتملة، ومساعدة المستخدمين في تحديد الأعطال من خلال لوحة تحكم تعمل بشكل لحظي.

يتكون النظام من عدة مكونات مترابطة. يشمل الجانب المادي وحدة استشعار تعتمد على متحكم دقيق متصل بشريحة إلكترونية، إضافة إلى مجموعة من الحساسات لقياس الجهد والتيار ودرجة الحرارة وشدة الإضاءة بهدف جمع البيانات اللحظية من الألواح الشمسية. أما الجانب البرمجي فيتضمن واجهات مستخدم، وقاعدة بيانات، بالإضافة إلى وحدات تحليل تعتمد على الذكاء الاصطناعي. ويقوم نموذج التعلم الآلي بتحليل بيانات القياسات لاكتشاف السلوك غير الطبيعي، بينما يقوم نموذج الرؤية الحاسوبية بتحليل صور الألواح الشمسية لاكتشاف الأعطال ومراقبة حالتها.

يدعم النظام المراقبة اللحظية، وعرض البيانات، والتحليل المعتمد على الذكاء الاصطناعي، وفحص الصور من خلال لوحة تحكم تفاعلية. كما تم تنفيذ عدة أنواع من الاختبارات أثناء التطوير، تشمل اختبار الوحدات، واختبار التكامل، واختبار نماذج الذكاء الاصطناعي، واختبار سهولة الاستخدام، وذلك للتحقق من سلامة مكونات النظام وتكاملها. ويُظهر مشروع سولارسنس كيفية دمج التقنيات الذكية في أنظمة الطاقة المتجددة بهدف تحسين كفاءة المراقبة، ودعم عمليات الصيانة، وتوفير أساس قابل للتطوير للحلول الذكية المستقبلية في إدارة الطاقة الشمسية.

Contents

Declaration of Originality	ii
Acknowledgements	iii
Abstract	iv
Table of Contents	vi
List of Tables	xi
List of Figures	xii
List of Abbreviations	xv
List of Appendices	xvii
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Motivation	2
1.3 Problem Statement	3
1.4 Proposed Solution	3
1.4.1 Panel-Level Monitoring	4
1.4.2 Image-Based Inspection	4
1.4.3 Centralized Analysis and Dashboard	5
1.4.4 System Workflow	5
1.5 Aim	5
1.6 Objectives	6
1.7 Scope	6
1.8 Target Users	6
1.9 Methodology	7
1.9.1 Requirements and System Analysis	7
1.9.2 Design	8
1.9.3 System Development and Implementation	8
1.9.4 Testing and Maintenance	9
1.10 Gantt Chart	11
1.11 Conclusion	12
2 BACKGROUND & LITERATURE REVIEW	13
2.1 Introduction	13
2.2 Solar Photovoltaic Systems	14
2.3 Artificial Intelligence	14
2.4 Machine Learning	14

2.4.1	Deep Learning	15
2.4.2	Convolutional Neural Networks	15
2.5	Computer Vision	16
2.5.1	Image Classification	17
2.5.2	Object Detection	17
2.6	Internet of Things	18
2.7	Panel Detection	18
2.7.1	Fault Detection and Analysis	18
2.7.1.1	Hotspot and Crack Detection	19
2.7.2	Performance Degradation Recognition	19
2.7.2.1	Challenges in Solar Panel Detection	19
2.8	Initial Models Architecture	19
2.8.1	Computer Vision Model Architecture	20
2.8.2	Machine Learning Model Architecture	21
2.8.3	Guiding Principles	22
2.9	Related Work	22
2.9.1	Vision-Based Inspection of PV Modules	22
2.9.1.1	Thermal Infrared Inspection	23
2.9.1.2	Broader Vision Literature	23
2.9.2	IoT-Enabled Solar Monitoring Systems	24
2.9.3	Machine Learning Based Anomaly Detection from Arduino Sensor Nodes	24
2.9.4	Embedded Monitoring and Edge Platforms	25
2.9.5	Related Work Comparison and Evaluation	26
2.10	Autonomous Monitoring within the Overall System	27
2.11	Required Skills	29
2.12	Conclusion	30
3	DATASET COLLECTION AND SPECIFICATIONS	31
3.1	Introduction	31
3.2	Available Datasets	32
3.3	Dataset Specifications	32
3.3.1	Image Dataset Specification	33
3.3.2	Numerical Dataset Specification	34
3.4	Dataset Collection	35
3.4.1	Numerical Datasets	35
3.4.1.1	Preprocessing Workflow	36
3.4.2	Images Datasets	37
3.4.2.1	Preprocessing Workflow	37
3.5	Dataset Limitations	38
3.5.1	Image Dataset Limitations	38
3.5.2	Numerical Dataset Limitations	39
3.6	Conclusion	40
4	SYSTEM REQUIREMENTS AND SYSTEM DESIGN	41
4.1	Introduction	41
4.2	Information Gathering Methods	41
4.2.1	Literature Review	42
4.2.2	Stakeholder Survey	43

4.2.3	Stakeholder Interview	44
4.3	Requirement Specification	46
4.3.1	Functional Requirements	46
4.3.2	Non-Functional Requirements	48
4.3.3	Data Requirements	49
4.3.3.1	User Requirements	49
4.3.3.2	Site Requirements	50
4.3.3.3	Panel Requirements	50
4.3.3.4	Mission Requirements	51
4.3.4	Software Requirements	51
4.3.5	Hardware Requirements	52
4.4	Initial Design	53
4.4.1	Entity-Relationship Diagram	53
4.4.2	Database Schema	54
4.4.3	Class Diagram	55
4.4.4	Use Case Diagram	58
4.4.5	Flow Chart	65
4.4.6	Sequence Diagrams	66
4.4.7	State Diagram	69
4.4.8	Computer Vision Analysis State Diagram	70
4.4.8.1	Computer Vision Data-flow Diagram	73
4.5	Prototype Design	75
4.5.1	Software Design	75
4.5.1.1	SolarSense User Interface	76
4.5.1.2	Dashboard Page	77
4.5.1.3	Panels Details Interface	79
4.5.1.4	Missions Interface	80
4.5.1.5	Admin Panel Page	81
4.5.2	Chip Design	82
4.6	Near Future Work	84
4.7	Conclusion	84
5	IMPLEMENTATION	85
5.1	Introduction	85
5.2	System Overview	86
5.3	Activities and Actions of the Development	86
5.4	Tools and Technologies	87
5.5	AI Model Components Implementation	91
5.5.1	CV Model Components Implementation	92
5.5.1.1	Component 1: Image Classification	92
5.5.1.2	Data Augmentation	93
5.5.1.3	Classification Results	93
5.5.1.4	Model Pipeline	94
5.5.2	ML Model Components Implementation	94
5.5.2.1	Component 1: Performance Regression & Prediction	95
5.5.2.2	Component 2: Sequence Classification	95
5.5.2.3	Telemetry Prediction Results	96

5.5.2.4	Model Pipeline	97
5.6	Front-end Implementation	98
5.7	Back-end Implementation	104
5.7.1	Database Implementation	105
5.7.2	API Implementation	106
5.8	Chip Implementation	108
5.9	Conclusion	110
6	TESTING	111
6.1	Introduction	111
6.2	Purpose, Scope, and Objectives	112
6.2.1	Purpose	112
6.2.2	Scope	112
6.2.3	STP Objectives	113
6.3	System Components Testing	114
6.4	Types of Testing	114
6.4.1	Component Testing	116
6.4.2	Unit Testing	116
6.4.3	Integration Testing	117
6.5	System Testing	117
6.6	Software Testing	118
6.6.1	Front-End Testing	118
6.6.2	Back-End Testing	119
6.7	Black-Box Functional Testing	120
6.8	AI Model Testing and Evaluation	121
6.8.1	Computer Vision Model Testing	122
6.8.2	Machine Learning Model Testing	126
6.9	Hardware Testing	128
6.9.1	Chip Testing	128
6.9.1.1	Sensor Data Acquisition Testing	129
6.9.1.2	Power Calculation Validation	129
6.9.1.3	Data Transmission Testing	130
6.9.1.4	Real-Time Data Validation	131
6.10	Usability Testing	131
6.10.1	Participants	132
6.10.2	Environment of the Test	132
6.10.3	Evaluation Tasks	133
6.10.4	Objective Measure Analysis	133
6.10.5	Subjective Measure Analysis	134
6.11	Conclusion	136
7	CONCLUSION AND FUTURE WORK	137
7.1	Introduction	137
7.2	Challenges and Limitations	138
7.3	Required Learned Skills	141
7.4	Future Work	144
7.5	Conclusion	145
	References	145

Appendices	151
1 Stakeholder Survey Questions	151
2 Stakeholder Interview Questions	154
3 Usability Testing Survey	157

List of Tables

1.1	Project Timeline and Milestones	10
2.1	Related Work for Solar Panel Fault Detection and Monitoring . .	27
2.2	Autonomous Monitoring within the Overall System	29
3.1	Solar Panel Datasets Overview	32
4.1	User Requirements, Description, and Examples.	49
4.2	Site Requirements, Description, and Examples.	50
4.3	Panel Requirements, Description, and Examples.	51
4.4	Mission Requirements, Description, and Examples.	51
4.5	Register Use Case Description	60
4.6	Log in Use Case Description	61
4.7	Profile Edit Use Case Description	62
4.8	Manage Panels Use Case Description	62
4.9	View Panel Details Use Case Description	63
4.10	Upload Panel Image Use Case Description	63
4.11	Access Control Use Case Description	64
4.12	Dashboard View Use Case Description	64
4.13	Logout Use Case Description	65
6.1	Types of Testing Applied in SolarSense	115
6.2	Summary of Testing Strategy Results	118
6.3	Summary of Software Testing Results	119
6.4	Black-Box Testing Execution Summary	120
6.5	Black-Box Functional Testing Results	121
6.6	Summary of AI Model Testing Results	122
6.7	Comparison Between the Previous and Updated YOLOv8 Pipelines	123
6.8	Computer Vision Model Classes	124
6.9	Comparison of Computer Vision Testing Results Before and After Pipeline Change	124
6.10	Machine Learning Pipeline Test Coverage	127
6.11	Machine Learning Power Prediction Results	127
6.12	Hardware Testing Evidence from Live Solar Panel Input	130
6.13	Task Evaluation: Participants' Number of Clicks	134
6.14	Task Evaluation: Participants' Duration	134

List of Figures

1.1	Renewable Energy Capacity Targets as Percentage of Total Energy Mix	2
1.2	Iterative Waterfall Phases	7
1.3	CPCS498 Gantt Chart	11
1.4	CPCS499 Gantt Chart	11
2.1	CNN Architecture [Ansari, 2020]	16
2.2	Computer Vision Architecture [GeeksforGeeks, 2025]	21
2.3	Machine Learning Architecture [Hardonag, 2024]	22
3.1	Example of Lighting Variation Problem	39
3.2	Example of Steep Angle Problem	39
3.3	Example of Too Far Distances Problem	39
4.1	SolarSense Entity-Relationship Diagram	54
4.2	SolarSense Relational Database Schema	55
4.3	SolarSense Class Diagram	57
4.4	SolarSense System Use Case Diagram	59
4.5	SolarSense Flowchart Diagram	66
4.6	Sequence Diagram for The Embedded Chip	68
4.7	Sequence Diagram for The Camera	69
4.8	Chip State Diagram	70
4.9	Computer Vision Analysis State Diagram	72
4.10	Chip Data-flow Diagram	73
4.11	Computer Vision Data-flow Diagram	75
4.12	Sign In Page	77
4.13	Sign Up Page	77
4.14	Profile Page	77
4.15	Reset Password Page	77
4.16	Dashboard Part 1	78
4.17	Dashboard Part 2	79
4.18	Panels Details	80
4.19	Panel Details Overview	80
4.20	Panel Details Activity	80
4.21	Panel Details records	80
4.22	Inspections Interface	81
4.23	Panel Images	81
4.24	CV Results	81
4.25	Admin Page	82
4.26	SolarSense Chip Design	83
5.1	LSTM Model Performance	96

5.2	Goodness of Fit	97
5.3	Test Set of Metrics	97
5.4	SolarSense Site Map	98
5.5	Sign In Page	100
5.6	Sign Up Page	100
5.7	Profile Page	100
5.8	Reset Password Page	100
5.9	Dashboard Part 1	101
5.10	Dashboard Part 2	101
5.11	Panels Details	102
5.12	Panel Details Overview	102
5.13	Panel Details Activity	102
5.14	Panel Details Records	103
5.15	Panel Images	103
5.16	CV Results	104
5.17	implemented SolarSense database in Supabase	106
5.18	API Implementation in PostgreSQL Part 1	107
5.19	API Implementation in PostgreSQL Part 2	107
5.20	API Implementation in PostgreSQL Part 3	108
5.21	API Implementation in PostgreSQL Part 4	108
5.22	SolarSense Chip	109
5.23	ESP32 Serial Monitor Output Showing Real-Time Telemetry Data	110
6.1	Confusion Matrix (Counts)	125
6.2	Normalized Confusion Matrix	125
6.3	Old Confusion Matrix (Counts)	125
6.4	Old Normalized Confusion Matrix	125
6.5	Example of transmitted JSON data and the corresponding API response	130
6.6	Real-time telemetry data displayed on the SolarSense dashboard .	131
6.7	Ease of use results	135
6.8	Satisfaction results	135
1.1	Study or work field	151
1.2	Knowledge about solar PV systems	151
1.3	Knowledge level regarding PV panel maintenance	151
1.4	Manual inspection is time consuming and labour intensive	151
1.5	Micro-cracks and hotspots are difficult to detect manually	151
1.6	Smart monitoring will improve fault detection accuracy	151
1.7	AI, Computer Vision & drones are effective for large-scale solar farms	152
1.8	The proposed system can reduce operational costs	152
1.9	Sensors on each panel help detect faults faster and more accurately	152
1.10	Using a drone for inspection is an effective solution	152
1.11	Data that should appear on the system dashboard	152
1.12	The biggest challenge facing solar systems	152
1.13	Contribution to improving maintenance efficiency and reducing costs	152
1.14	Most important system feature	152
1.15	Additional comments or expectations for SolarSense	153

3.1	Task 1 Part 1 Results	157
3.2	Task 1 Part 2 Results	157
3.3	Task 2 Part 1 Results	157
3.4	Task 2 Part 2 Results	157
3.5	Task 3 Part 1 Results	158
3.6	Task 3 Part 2 Results	158
3.7	Task 4 Part 1 Results	158
3.8	Task 4 Part 2 Results	158
3.9	Task 5 Part 1 Results	159
3.10	Task 5 Part 2 Results	159
3.11	Task 6 Part 1 Results	159
3.12	Task 6 Part 2 Results	159
3.13	Task 7 Part 1 Results	160
3.14	Task 7 Part 2 Results	160
3.15	Task 8 Part 1 Results	160
3.16	Task 8 Part 2 Results	160
3.17	Task 9 Part 1 Results	161
3.18	Task 9 Part 2 Results	161
3.19	Overall User Experience Results Part 1	161
3.20	Overall User Experience Results Part 1	161

List of Abbreviations

A	Ampere.
AC	Alternating Current.
AI	Artificial Intelligence.
AIoT	Artificial Intelligence of Things.
API	Application Programming Interface.
AUROC	Area Under the Receiver Operating Characteristic Curve.
CNN	Convolutional Neural Network.
CSS	Cascading Style Sheets.
CSV	Comma-Separated Values.
CV	Computer Vision.
DC	Direct Current.
DL	Deep Learning.
EL	Electroluminescence.
ER	Entity-Relationship.
GPU	Graphics Processing Unit.
GUI	Graphical User Interface.
GW	gigawatts.
HTML	HyperText Markup Language.
HTTP	HyperText Transfer Protocol.
I	Electric current.
I²C	Inter-Integrated Circuit.
IDE	Integrated Development Environment.
IOS	iPhone Operating System.
IoT	Internet of Things.
IR	Infrared.
IV	Current–Voltage.
JSON	JavaScript Object Notation.
KPI	Key Performance Indicator.
LCD	Liquid Crystal Display.
LoRa	Long Range.
LSTM	Long Short-Term Memory.
lux	A unit of measurement for light intensity.
MacOS	Mac Operating System.
MAE	Mean Absolute Error.
mAP	mean average precision.
MCU	Microcontroller Unit.
ML	Machine Learning.
MLPE	Module-Level Power Electronics.
MLPerf	Machine Learning Performance Benchmark Suite.
MSE	Mean Squared Error.

PdM	Predictive Maintenance.
PV	Photovoltaic.
QC	Quality Control.
R-CNN	Region-based Convolutional Neural Network.
RGB	Red, Green, Blue.
RMSE	Root Mean Squared Error.
SCADA	Supervisory Control and Data Acquisition.
SQL	Structured Query Language.
STP	Software Testing Plan.
UAV	Unmanned Aerial Vehicle.
UI/UX	User Interface and User Experience.
UML	Unified Modeling Language.
USB	Universal Serial Bus.
V	Voltage .
Wi-Fi	Wireless Fidelity.
YOLO	You Only Look Once.

List of Appendices

Appendix 1 Stakeholder Survey Questions	151
Appendix 2 Stakeholder Interview Questions	154
Appendix 3 Usability Testing Survey	157

CHAPTER 1

INTRODUCTION

1.1 Introduction

Solar energy is one of the fastest growing renewable energy sources due to its scalability, cost-effectiveness, and environmental benefits. As the world seeks sustainable alternatives to fossil fuels, solar photovoltaic (PV) systems have become central to achieving clean energy goals. However, the efficiency of these systems is often compromised by faults such as cracks, hotspots, and dust accumulation, which can significantly reduce power output. Traditionally, inspections of PV panels have been carried out manually, requiring field technicians to physically examine panels one by one. This process is time consuming, costly, and prone to human error. Moreover, early-stage faults frequently go undetected, leading to energy losses, system downtime, and shorter panel lifespan. These challenges are amplified in large scale solar farms, where manual inspection becomes impractical and inefficient.

In response to these challenges, the proposed project introduces SolarSense, an intelligent monitoring and diagnostic system that integrates Artificial Intelligence, computer vision, the Internet of Things, and embedded smart chips. This solution transforms inspections from a manual, reactive process into a fully automated, predictive, and real-time operation. By leveraging data driven insights, SolarSense improves fault detection accuracy, enables predictive maintenance, reduces operational costs, and extends panel lifespan.

The aim of this project is not only to replace traditional inspection methods but also to redefine solar energy maintenance by providing a intelligent, scalable and cost effective framework that supports both residential and industrial

scale installations. By developing SolarSense, we seek to contribute to global sustainability efforts while aligning with the Kingdom of Saudi Arabia's Vision 2030 focus on renewable energy innovation.

In this chapter, section 1.2 will illustrate the motivation behind the project. Then, section 1.3 presents the problem statement, and section 1.4 describes the proposed solution. Section 1.5 specifies the aim, while section 1.6 outlines the objectives. Section 1.7 presents the scope of the project, and section 1.8 identifies the target users. Section 1.9 explains the methodology to be adopted, followed by section 1.10, which details the plan and milestones. Section 1.11 presents the Gantt chart illustrating the project timeline, and finally, section 1.12 provides the conclusion of this chapter.

1.2 Motivation

AI and renewable energy are the central pillars of Saudi Arabia's Vision 2030, which aims to generate 50% of the Kingdom's electricity from renewable sources by 2030, including nearly 58.7 GW from solar energy [Alfehaid and Young, 2025]. With this rapid expansion of solar projects, millions of panels will be deployed, creating an urgent need for smarter monitoring and maintenance solutions to ensure reliability and efficiency.

Traditional inspection methods, such as electroluminescence (EL) and manual checks, are time-consuming, costly, and prone to human error. They also present safety risks for technicians working in large-scale solar farms and often fail to detect subtle defects such as micro cracks and hotspots. These limitations highlight the necessity for advanced, automated solutions.

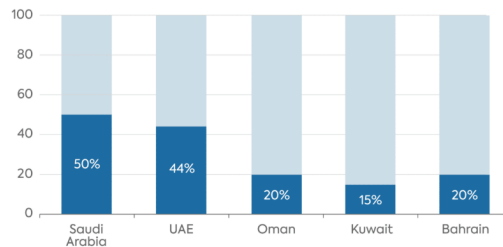


Figure 1.1: Renewable Energy Capacity Targets as Percentage of Total Energy Mix

1.3 Problem Statement

The rapid expansion of PV technology has made it one of the most widely adopted renewable energy sources. Despite this growth, PV systems continue to face critical challenges, including faults, degradation, and variability in energy output. Research indicates that undetected faults in PV arrays, such as partial shading, module cracks, hotspots, or inverter malfunctions, may reduce overall efficiency and cause up to 10% energy loss if not promptly identified. Traditional fault detection approaches, such as manual inspections and electrical measurements, are labor-intensive, slow, and often unreliable, especially in large-scale solar farms [Islam et al., 2023].

In terms of system reliability, most PV installations still rely on reactive maintenance strategies, addressing issues only after failures occur. This results in high repair costs, unplanned downtime, and shorter equipment lifespan. Predictive maintenance, enabled by machine learning, has been shown to reduce operational risks by forecasting equipment failures. However, its effectiveness is limited by challenges such as noisy sensor data, heterogeneous input sources, and limited model interoperability. [Qureshi et al., 2024]

Another pressing challenge is the variable nature of solar energy output, which is influenced by weather conditions, dust accumulation, and irradiance fluctuations. This variability complicates operational monitoring and maintenance planning in PV systems. While AI-based methods have been introduced for PV output forecasting, many remain constrained to short-term horizons and fail to capture long-term degradation patterns [Shaik et al., 2025]. Therefore, there is a need for an integrated intelligent system that combines real-time monitoring, accurate fault detection, predictive maintenance, and robust energy forecasting to ensure sustainable and efficient operation of PV farms.

1.4 Proposed Solution

This project proposes the design and implementation of an AI-driven Solar Monitoring System that integrates CV, ML, and the IoT to optimize the performance of PV installations. The system incorporates panel level monitoring, manually captured panel image inspections, and centralized data analysis to en-

sure reliable detection, validation, and detection of faults in solar panels. In this approach, images of the panels are captured manually by an operator using a standard camera or mobile device and then analyzed by the AI model to identify potential defects. Such an approach is intended to contribute to reduced downtime, improved scalability, and better support for predictive maintenance strategies, which are increasingly emphasized in contemporary solar energy research[Boucif et al., 2025, Gan et al., 2017] .

1.4.1 Panel-Level Monitoring

Each solar panel is associated with an embedded chip, developed during the implementation phase, and connected to sensors capable of measuring voltage, current, light, and temperature. The embedded system performs periodic checks to detect operational anomalies. If an abnormal reading is identified, the chip will generate an alert and transmit a detailed report to the central system for further analysis. Even when no anomaly is detected, routine performance data will still be transmitted, enabling continuous monitoring and data logging. This distributed architecture allows each panel to be independently monitored, thus facilitating early detection of failures and ensuring that the monitoring system remains scalable as the solar farm expands. Such an approach is consistent with recent research on IoT-enabled photovoltaic monitoring, which highlights the importance of distributed sensing for real-time fault detection and predictive maintenance.

1.4.2 Image-Based Inspection

When an anomaly report is received by the central system, a manual image inspection process is initiated. An operator captures images of the specific panel that generated the anomaly report using a camera or mobile device. The captured images enable a detailed visual and RGB inspection of the panel. The images are then transmitted back to the central system for verification and analysis. This mechanism is intended to help reduce false positives generated by sensor readings while providing additional insights into the physical condition of panels through advanced computer vision techniques[Martinez-Gil et al., 2025,

Ansari, 2020, Forsyth and Ponce, 2011].

1.4.3 Centralized Analysis and Dashboard

The central system serves as the integration hub by combining input from panel-level chips and operator-captured images. An intelligent analysis engine processes the data to classify faults and evaluate system performance. The dashboard provides real-time visualization of operational parameters, inspection outcomes, and fault alerts. Additionally, the system is designed to send automated notifications to operators in the event of anomalies. This functionality ensures a comprehensive view of the solar farm, enabling timely interventions and supporting maintenance strategies that are critical for sustainable energy systems[Abdulla et al., 2024, Islam et al., 2023].

1.4.4 System Workflow

The operational workflow of the proposed system can be described as follows. First, the embedded chip continuously monitors panel performance. If an anomaly is detected, an alert is transmitted to the central system. The system then requests an image-based inspection of the affected panel. An operator captures RGB images of the panel, which are relayed back to the central hub. The system integrates the data from the embedded chip and the captured images to generate a final AI result, which is subsequently displayed on the monitoring dashboard. In cases where no anomaly is detected, routine performance reports are still logged for historical analysis and continuous monitoring. This integrated workflow enhances system reliability, reduces operational downtime, and ensures efficient and scalable monitoring of PV installations[Qureshi et al., 2024, Shaik et al., 2025].

1.5 Aim

The project aims to develop an intelligent system for monitoring solar panels to improve their reliability, efficiency, and lifespan. The system leverages machine learning, computer vision, IoT sensors, embedded smart chips, and an interactive dashboard.

1.6 Objectives

The main objective of this project is to develop an intelligent solar panel monitoring system that supports real-time performance tracking, automated fault detection, and predictive maintenance. The specific objectives are as follows:

- To design and implement an integrated solar monitoring system that combines embedded sensing, image-based inspection, CV,ML analysis, and real-time data visualization.
- To develop a lightweight embedded sensing unit capable of collecting panel-level electrical and environmental data in real time.
- To apply ML-based anomaly detection to identify abnormal panel behavior and support early fault diagnosis.
- To use camera-based image inspection to analyze solar panel conditions and verify faults detected from sensor data.
- To implement a user-friendly dashboard that displays live monitoring data, anomaly alerts, and inspection results for system operators.
- To validate the complete system by integrating the embedded sensing unit, sensors, image inspection module, dashboard, and CV/ML models into a scalable solution suitable for solar farm monitoring.

1.7 Scope

The project presents a solution to improve PV system reliability by automating the monitoring and maintenance process. SolarSense employs computer vision, artificial intelligence, and IoT-enabled sensors to detect faults such as cracks, hotspots, dust accumulation, and performance anomalies in solar panels. The system provides real-time alerts and live monitoring insights, reducing downtime and extending panel lifespan.

1.8 Target Users

The target users of SolarSense are solar farm operators who seek to improve system reliability and reduce maintenance costs. The solution is particularly

beneficial for organisations managing large-scale PV installations where manual inspections are inefficient and costly. By providing intelligent, automated monitoring, SolarSense empowers users to ensure maximum energy yield, minimise operational risks, and support sustainable energy management practices. While the system facilitates real-time monitoring and anomaly detection, the responsibility for taking corrective action remains with the system operators.

1.9 Methodology

This project follows an iterative waterfall methodology that ensures systematic progress across all phases of development while allowing refinement at each stage. The methodology is structured into five primary phases: requirement and system analysis, design, system development and implementation, testing, and maintenance. Each phase contributes to achieving the project objectives and builds upon the outcomes of the previous stage [Pal, 2025].

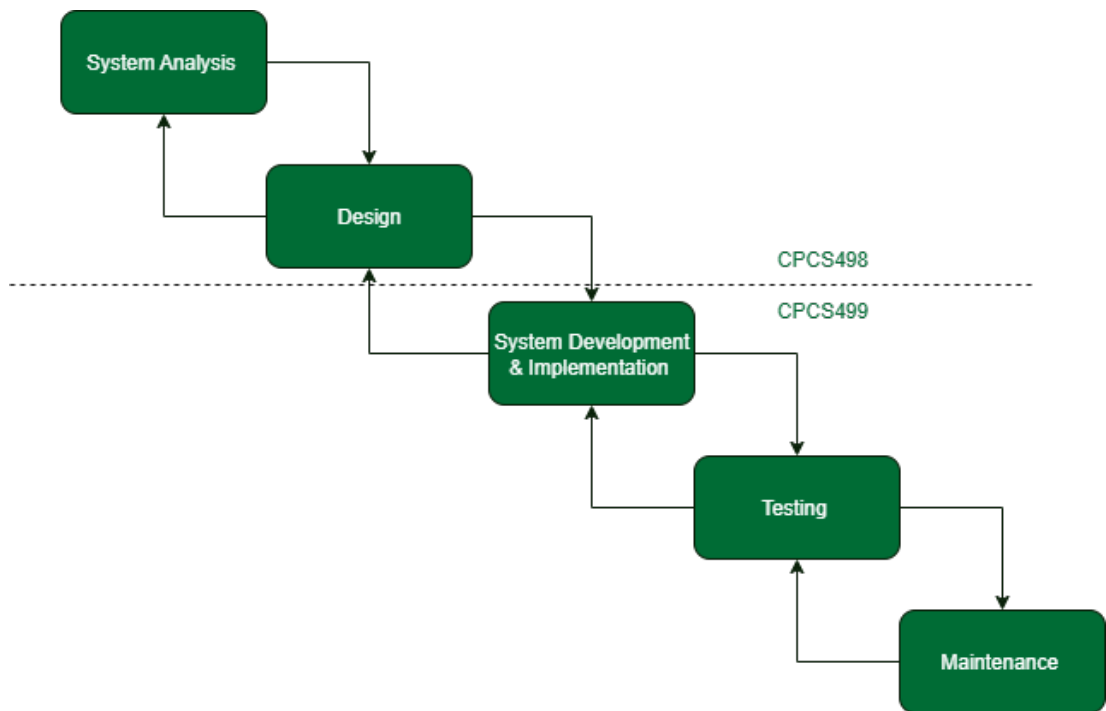


Figure 1.2: Iterative Waterfall Phases

1.9.1 Requirements and System Analysis

The first phase of the project focuses on requirements gathering and analysis, combined with an extensive review of the literature. During this stage, studies

on solar panel monitoring, artificial intelligence-based fault detection, and image-based inspection techniques are examined to establish the theoretical foundation of the system. Both functional and non-functional requirements are identified, including the need for panel-level monitoring, anomaly detection, image-based inspections, and a main dashboard for reporting. In addition, system attributes such as scalability, accuracy, and real-time responsiveness are specified. The outcome of this phase was a detailed specification document of requirements, which defines the objectives, scope, and constraints of the project.

1.9.2 Design

The second phase involved the design of the system architecture and interface. This stage began with the collection and preparation of relevant datasets, including panel parameters such as voltage, current, and temperature, as well as defect imagery depicting issues such as cracks, hotspots, and dust accumulation. These datasets were preprocessed for use in training and validating artificial intelligence models. Alongside data preparation, the system architecture was designed to integrate the embedded chips, IoT communication modules, image-based inspection, and the central monitoring dashboard. Furthermore, the user interface of the dashboard designed to ensure that system operators can easily visualise performance metrics, fault alerts, and inspection reports. The deliverable from this phase were be comprehensive system models and prototypes that serve as blueprints for development.

1.9.3 System Development and Implementation

The third phase focuses on the development and implementation of the system, where both hardware and software components are constructed. The front-end of the system consists of a dashboard that displays real-time monitoring results and fault reports. The back-end handles the integration of databases, APIs, and server logic to process data from panel sensors and image-based inspections. At the same time, artificial intelligence models, including computer vision algorithms and time-series analysis models, were trained on the collected datasets to enable accurate fault detection and anomaly recognition. Hardware development in-

cludes integrating sensing units with solar panels, linking them with sensors, and enabling image capture for computer vision analysis. The development process is iterative, allowing modules to be tested and improved incrementally before full system integration.

1.9.4 Testing and Maintenance

The final phase addressed system testing and maintenance to ensure the solution operates reliably under real-world conditions. Testing began at the unit level, where each module such as chip monitoring, image-based inspection, and dashboard analytics was verified independently. Integration testing confirmed that hardware and software components communicate effectively, while system-level testing evaluated performance, accuracy, and scalability in simulated environments. User acceptance testing was also conducted to ensure the solution met practical requirements. Following future deployment, maintenance became an ongoing activity, including updates to datasets, retraining of artificial intelligence models to improve accuracy, and adjustments to the system interface or performance. This phase guarantees the long-term sustainability and adaptability of the solution.

Table 1.1: Project Timeline and Milestones

Term #	Waterfall Phase	Main Milestone	Tasks	Week #
Term 1 (CPCS498)	System Analysis	Introduction	Problem definition	Week 1
			Suggested solution	Week 2
			Determine aims & objectives	Week 3
			Define scope and target users	Week 4
		Literature Review	Read scientific papers in project domain	Week 5
			Review solar systems, AI, IoT, and image-based monitoring	Week 6
			Analyse related work	Week 7
			Select AI techniques	Week 8
		Requirement Analysis	Requirement specification	Week 9
			Identify functional and non-functional requirements	Week 10
			Determine hardware needs	Week 11
	Design	System Architecture	Develop system workflow	Week 12
	Finalisation	Final Submission & Presentation	Design prototype	Week 13
			Submit Term 1 report and present project progress	Week 14–15
Term 2 (CPCS499)	Development & Integration	Model Training & Integration	Train YOLOv8 model on dataset	Week 1
			Train LSTM model on dataset	Week 2
			Develop the chip	Week 3
			Integrate trained models with back-end	Week 4
			Implement dashboard front-end	Week 5
			Implement dashboard back-end	Week 6
			Connect database and APIs	Week 7
			Connect chip with voltage, current, light, and temperature sensors	Week 8
		System Integration	Prepare image inspection workflow for CV analysis	Week 9
			Test chip-to-platform data flow and image inspection workflow	Week 10
			Integrate chip monitoring and image inspection	Week 11
			Integrate AI model outputs with dashboard	Week 12
	Testing	System Testing	Unit and integration testing	Week 13
	Finalisation	Final Submission & Presentation	Submit final report and conduct project presentation	Week 14–15

1.10 Gantt Chart

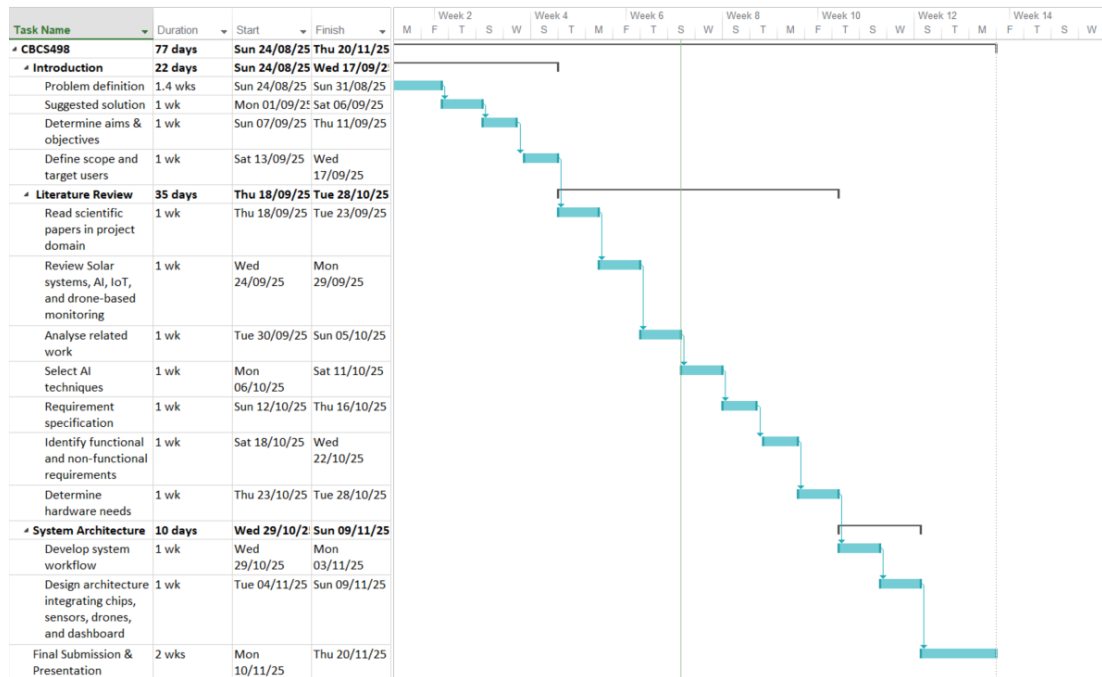


Figure 1.3: CPCS498 Gantt Chart

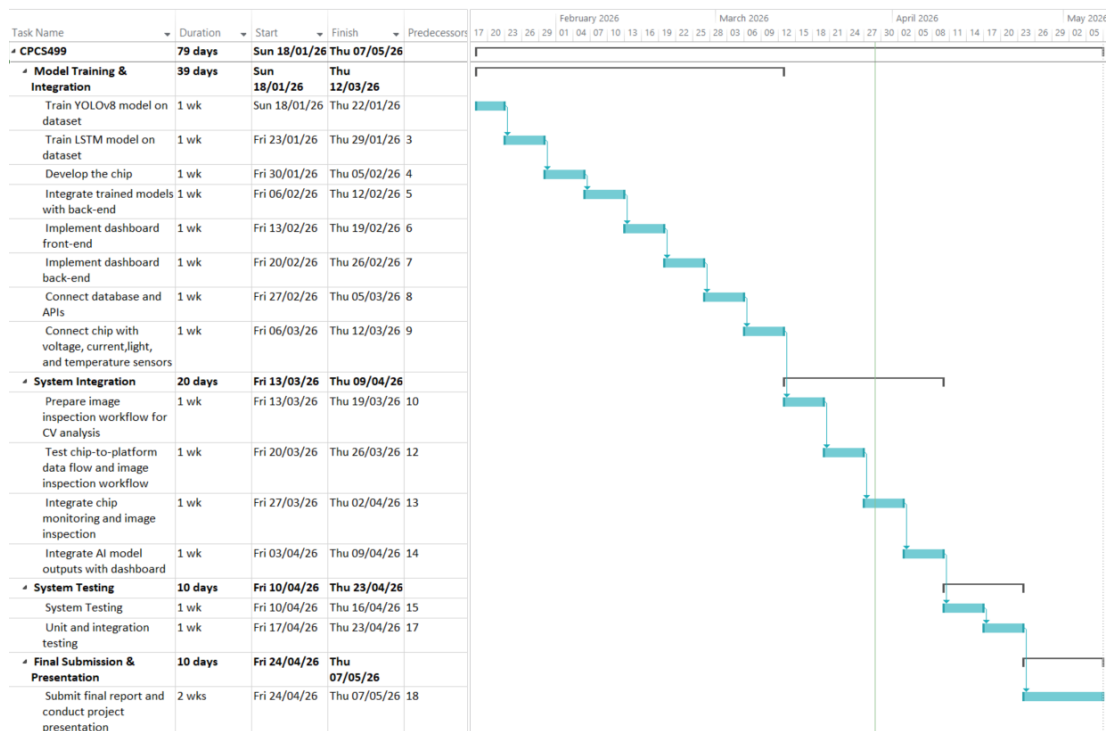


Figure 1.4: CPCS499 Gantt Chart

1.11 Conclusion

This chapter outlined the overall concept of the SolarSense project and introduced its context. This introduced the project's context and highlighted the challenges associated with traditional inspection methods. Then, presenting the proposed solution along with its aims and objectives. In addition, it defined the scope of the project and identified the intended target users. Finally, it described the development methodology, project milestones, and overall project plan to ensure system reliability. The next chapter will provide a literature review of the technologies and strategies that support the implementation of the system.

CHAPTER 2

BACKGROUND & LITERATURE REVIEW

2.1 Introduction

A major part of developing an intelligent solar panel monitoring system is understanding the theoretical foundations and previous research that support the selected technologies. In SolarSense, the system depends on several connected areas, including PV systems, AI, ML, DL, CV, and IoT. These areas provide the technical basis for monitoring solar panel behaviour, analysing sensor readings, identifying visual faults, and supporting maintenance decisions.

For the SolarSense project, the literature review is important because the proposed system combines panel-level telemetry monitoring with image-based inspection. Previous studies related to solar panel fault detection, computer vision inspection, IoT-enabled monitoring, and machine learning-based anomaly analysis were reviewed to identify suitable methods and limitations. This helped establish the relationship between the existing research and the design decisions adopted in the project.

By reviewing these technologies and related systems, the chapter supports the selection of the main components used in SolarSense and clarifies how they contribute to the overall monitoring workflow. This chapter outlines the background and literature review of the project. Section 2.2 presents solar photovoltaic systems; Sections 2.3 to 2.6 describe the main intelligent technologies; Section 2.7 discusses panel detection and fault analysis; Section 2.8 explains the initial model architectures; Section 2.9 evaluates related work; Section 2.10 discusses autonomous monitoring within the system; Section 2.11 presents the required skills; and Section 2.12 concludes the chapter.

2.2 Solar Photovoltaic Systems

PV systems convert sunlight into electricity through solar cells, typically made of silicon semiconductors. These systems have gained prominence as a clean and renewable alternative to fossil fuels due to their scalability, low operational costs, and reduced environmental impact [Shaik et al., 2025]. However, challenges such as intermittency, soiling, hot spots, and component degradation affect their efficiency [Martinez-Gil et al., 2025]. Recent research highlights the integration of AI, ML, and IoT for optimising solar energy production by enabling predictive maintenance, power output forecasting, and automated fault detection [Abdulla et al., 2024, Islam et al., 2023]. Such advancements ensure that PV systems contribute significantly to global renewable energy targets.

2.3 Artificial Intelligence

AI refers to the field of computer science that focuses on creating machines capable of simulating human cognitive functions such as reasoning, problem-solving, learning, and perception. The primary goal of AI is to enable systems to perform tasks that typically require human intelligence, thereby improving efficiency and decision-making in complex environments [Shaik et al., 2025]. Over the past decades, AI has advanced from symbolic approaches to data-driven methodologies, becoming a foundational technology in diverse domains including healthcare, robotics, and renewable energy systems [Boucif et al., 2025].

2.4 Machine Learning

ML is a branch of artificial intelligence that enables computers to learn patterns from data and improve performance without explicit programming. It is commonly grouped into four categories: supervised learning involves learning a mapping from labeled inputs to target outputs for classification or regression; unsupervised learning discovers structure in unlabeled data, such as clusters or anomalies; semi-supervised or self-supervised learning leverages large unlabeled datasets with few or no labels to learn useful representations; and reinforcement learning optimizes sequences of actions by maximizing reward through interaction with an environment. In practice, techniques such as decision trees, support vec-

tor machines, random forests, and gradient boosting are widely used across predictive maintenance, fault detection, and energy forecasting [Qureshi et al., 2024, Nazeer, 2018, Shaik et al., 2025].

In this system, only supervised and unsupervised learning were employed. Typical inputs include SCADA variables (voltage, current, temperatures, irradiance) and panel-level sensor data. The implemented pipeline extracted simple features on the embedded node, fits a lightweight supervised regression model to estimate expected behaviour, and then applies an unsupervised detector to residual windows to flag anomalies; neighbour comparisons across panels will be used to reduce false alarms. This reported clear metrics such as accuracy and AU-ROC for anomaly flags, and mean absolute error for any forecasts, consistent with prior PV studies [Islam et al., 2023, Abdulla et al., 2024]. As a senior project, it started with interpretable models (e.g., random forest or gradient boosting) and only added complexity where it brought practical gains [Aghaei et al., 2025].

2.4.1 Deep Learning

DL is a subfield of ML that utilises multi-layered Artificial Neural Networks to model and process complex, high-dimensional data. Unlike traditional ML algorithms, DL architectures can automatically learn hierarchical feature representations, making them particularly powerful in image recognition, natural language processing, and speech analysis [Ansari, 2020]. DL has been proven to outperform traditional ML approaches in tasks requiring large-scale data processing and high levels of abstraction [Islam et al., 2023].

2.4.2 Convolutional Neural Networks

CNNs are a specialized class of deep learning models developed primarily for image and spatial data analysis. They use convolutional layers to automatically learn spatial features from raw input data, followed by pooling and fully connected layers for classification or prediction. Because CNNs can effectively capture local patterns and hierarchical visual features, they have become widely used in computer vision applications such as object recognition, medical image analysis, and photovoltaic fault detection [Forsyth and Ponce, 2011, Ansari, 2020]. In

renewable energy research, CNNs have been applied to analyze thermal and visual imagery for detecting defects in solar panels, showing strong performance in automated fault identification [Islam et al., 2023].

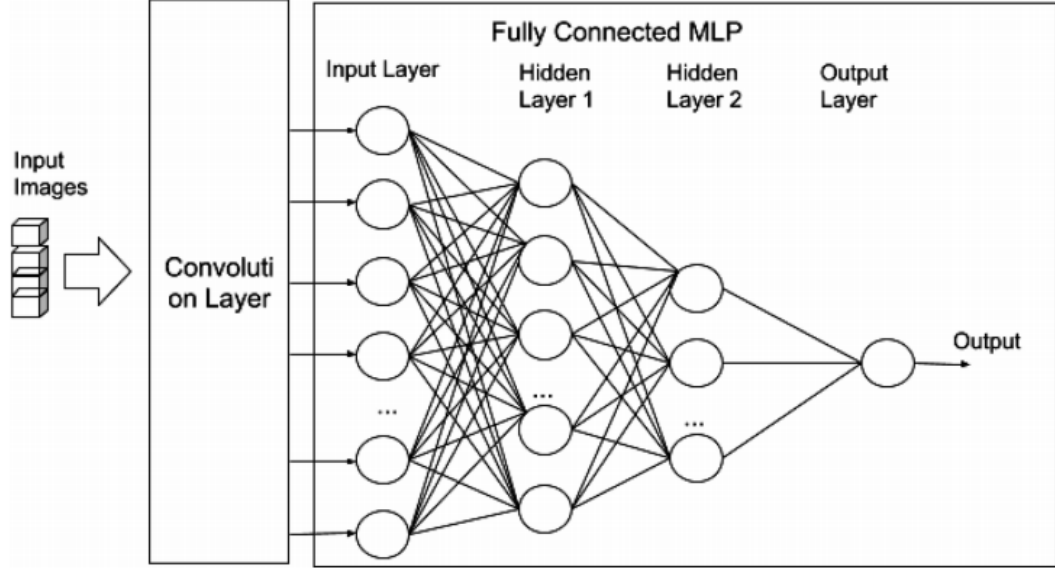


Figure 2.1: CNN Architecture [Ansari, 2020]

2.5 Computer Vision

CV is a field within AI that focuses on teaching machines how to interpret and analyze visual information. Instead of relying on human eyes, CV uses cameras and sensors to capture images or video, and then applies algorithms and models to recognize patterns, detect objects, or classify conditions.

In recent years, CV has become one of the most practical AI technologies because of its wide use in areas like medical imaging, autonomous vehicles, industrial automation, and environmental monitoring. What makes CV powerful is its ability to process large amounts of visual data in real time and identify details that may be overlooked by humans.

For solar energy applications, CV offers valuable capabilities in monitoring panel performance and identifying physical defects. By combining standard cameras, RGB imaging, and deep learning algorithms, CV can automatically spot cracks, hotspots, or other anomalies across thousands of panels with high accuracy. Recent reviews highlight that advancements in UAVs and AI techniques particularly ML and DL have opened new opportunities for solar farm monitoring,

with the top fault detection methodologies relying on deep learning models such as CNNs and YOLO variations [Polymeropoulos et al., 2024]. In this project, CV is applied mainly through image classification to categorize panel conditions and object detection to locate the exact position of faults.

2.5.1 Image Classification

Image classification is one of the fundamental tasks in computer vision, where a model assigns an input image to one of several predefined categories. In the context of photovoltaic systems, this task enables the automatic distinction between normal solar panels and panels affected by defects such as cracks, hotspots, dust accumulation, or snow coverage.

The classification model is trained using annotated datasets of solar panel images. After training, it can classify newly captured images automatically, providing a fast and reliable way to assess panel condition.

This capability is important in the SolarSense system because it serves as an initial step in fault identification, allowing the system to determine the general condition of a panel before more detailed analysis is carried out.

2.5.2 Object Detection

While image classification determines the category of a defect, object detection provides additional information by identifying the exact location of the defect within the image. Object detection models, such as YOLO and R-CNN, generate bounding boxes around abnormal regions, including cracks, hotspots, or other visible faults.

In SolarSense, object detection is applied to solar panel images to highlight defective areas and indicate the position of the fault. This makes the inspection process more actionable, as maintenance teams can focus directly on the affected region instead of examining the entire panel or array manually.

By combining image classification and object detection, the proposed system achieves both accurate fault recognition and precise fault localization, thereby improving the overall effectiveness of photovoltaic monitoring and maintenance.

2.6 Internet of Things

IoT refers to a network of interconnected devices that collect, share, and analyse data through sensors, communication protocols, and cloud platforms. IoT enables real-time monitoring, predictive analytics, and automated control of complex systems. When integrated with AI, IoT forms the Artificial Intelligence of Things (AIoT), which enhances decision-making through intelligent insights derived from massive streams of data [Boucif et al., 2025]. In the solar energy domain, IoT devices such as sensors, microcontrollers, and UAV are used for monitoring energy output, detecting faults, and automating panel cleaning processes. This integration ensures reliability, optimises system performance, and supports sustainability goals [Martinez-Gil et al., 2025].

2.7 Panel Detection

Solar panel detection is a branch of intelligent monitoring in PV systems. It is the task of identifying and localising solar panels within images or video streams, which is essential for fault analysis, performance tracking, and maintenance planning. With the global expansion of solar energy, automated panel detection has become a crucial step to ensure efficiency, safety, and reliability of PV installations [Islam et al., 2023]. Its main task is to detect and locate panels accurately under varying environmental and operational conditions. Due to the wide deployment of PV farms, solar panel detection has been attracting more attention in both academic and industrial research [Qureshi et al., 2024].

2.7.1 Fault Detection and Analysis

Fault detection in PV systems refers to identifying conditions that may impair the efficiency of solar panels. Automated systems such as SolarSense employ a combination of IoT sensors and RGB imaging to monitor voltage, current, temperature, and surface anomalies [Agrawal and Ahuja, 2021]. The process of fault detection can be divided into four stages:

1. Data Acquisition: collecting electrical, environmental, and imaging data.
2. Pre-processing: filtering and normalizing multimodal signals.

3. Feature Extraction: identifying attributes such as hotspots, shading, or cracks.
4. Classification and Prediction: applying deep learning models to diagnose and predict faults.

2.7.1.1 Hotspot and Crack Detection

Hotspots are localized regions of abnormal heating, often caused by shading, cracks, or defective cells, which can accelerate degradation and increase fire hazards [Qureshi et al., 2024]. Using RGB imaging with CNN models, SolarSense can detect hotspots in early stages. Micro-cracks, though invisible to the naked eye, can be captured through pattern recognition in RGB images, allowing timely intervention and efficiency preservation.

2.7.2 Performance Degradation Recognition

Performance degradation refers to the gradual decline in PV panel efficiency due to factors like dust accumulation or soiling. SolarSense identifies degradation by correlating electrical outputs with environmental and imaging data. For example, dust detection through computer vision can help reduce downtime and maintaining energy output.

2.7.2.1 Challenges in Solar Panel Detection

Despite its advancements, solar panel detection faces key challenges:

- Data Diversity: variations in imaging (RGB) and sensors quality.
- Environmental Conditions: weather, seasonal shifts, and lighting inconsistencies.
- Occlusion: debris or shading can hide panel surfaces and trigger false alarms.

2.8 Initial Models Architecture

The proposed project integrates two main artificial intelligence subsystems. The first is the CV model, which processes solar panel imagery to detect and localize defects. The second is a set of machine learning models that analyze operational

and environmental data to identify anomalies and forecast performance trends, thereby supporting predictive maintenance. Together, these subsystems form an integrated framework in which inspection outputs are transformed into actionable maintenance insights.

2.8.1 Computer Vision Model Architecture

The computer vision subsystem begins with the collection of solar panel images from available imaging sources. These inputs then undergo preprocessing operations such as image enhancement, deblurring, and normalization to improve their quality and suitability for analysis. Once preprocessed, a YOLOv8-based detection model extracts visual features and identifies possible forms of panel damage. To improve the reliability of detections, post-processing techniques are applied to refine results and reduce false positives. The detected faults are then linked with location-related metadata when available, to support fault localization within the solar farm. Finally, the processed results are reported for both real-time monitoring and later review, ensuring that each detected issue is documented in a clear and interpretable manner.

This architecture is designed with an emphasis on robustness against environmental variability, computational efficiency for practical deployment, and modularity to support future extensions such as segmentation or severity assessment. In addition, the outputs are intended to remain interpretable by presenting confidence scores and relevant metadata alongside the visual detections.

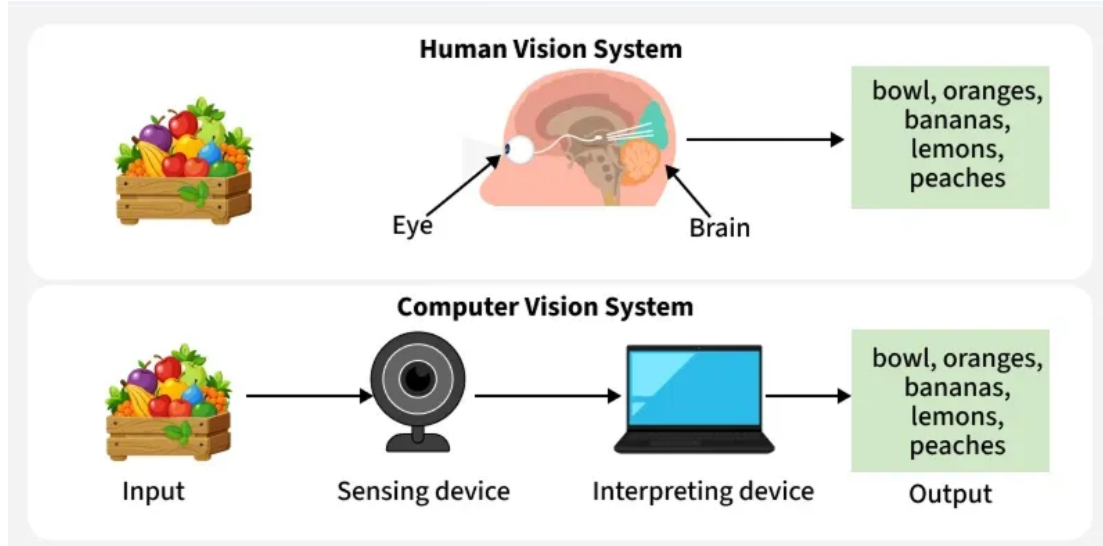


Figure 2.2: Computer Vision Architecture [GeeksforGeeks, 2025]

2.8.2 Machine Learning Model Architecture

The machine learning subsystem is centered on the integration of computer vision outputs with operational plant data. Once detection and localization results are produced by the CV model, they are combined with sensor measurements such as temperature, voltage, current, and power generation metrics. This unified dataset is then processed through feature engineering techniques to derive meaningful temporal, environmental, and operational indicators. Based on these features, forecasting models are used to predict short-term performance patterns, while anomaly detection methods are applied to identify irregular system behavior. The same input data can also support prioritization models that estimate the severity of detected issues and their potential impact on energy yield, allowing maintenance activities to be ranked more effectively.

The design of this subsystem is intentionally data-centric, allowing the use of both classical machine learning and deep learning methods depending on dataset size, quality, and complexity. Particular emphasis is placed on interoperability and explainability so that operators can understand why anomalies are flagged and why certain faults are prioritized. Through its integration with CV outputs, the machine learning subsystem ensures that visual detections are treated not as isolated observations, but as contextual signals that strengthen predictive maintenance decisions.

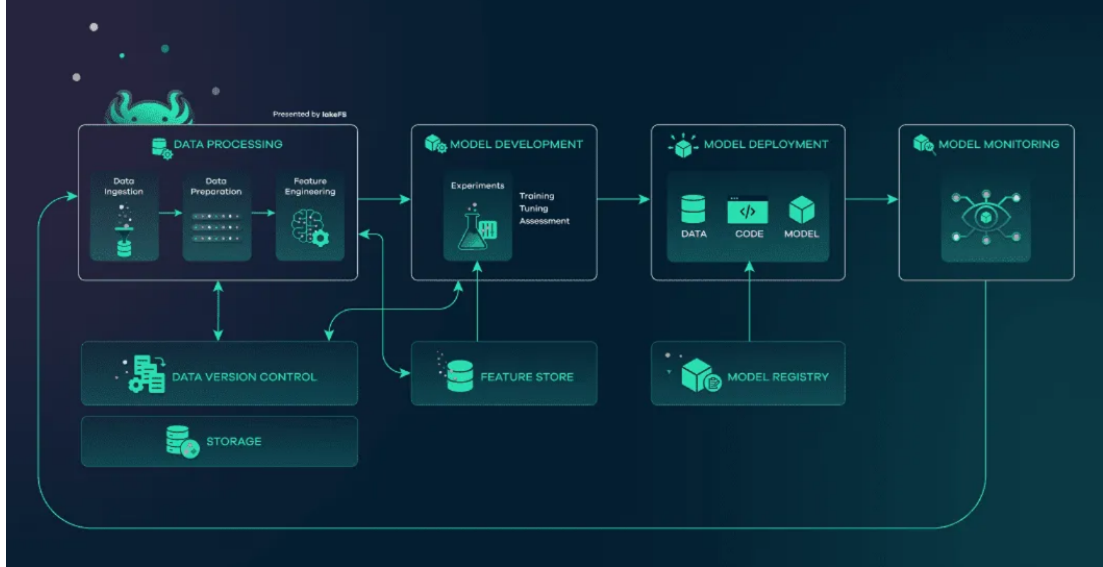


Figure 2.3: Machine Learning Architecture [Hardonag, 2024]

2.8.3 Guiding Principles

Both the CV and ML subsystems are designed according to the principles of modularity, scalability, and adaptability. Each subsystem can evolve independently while still contributing to a unified decision-support framework. The architectures are intended to accommodate additional datasets, sensing modalities, or analytical techniques as the project develops, while continuing to provide outputs that are transparent and explainable. Throughout the design process, attention is also given to ethical and regulatory considerations, particularly with respect to data management, system reliability, and privacy.

2.9 Related Work

Solar panel inspection and monitoring are vital to ensure long-term efficiency and reliability of photovoltaic systems. Researchers have explored both computer vision-based inspection techniques and IoT-enabled monitoring approaches. Each direction has demonstrated unique strengths but also faces limitations in terms of scalability, automation, and fault generalization [Salazar and Macabebe, 2016].

2.9.1 Vision-Based Inspection of PV Modules

Beyond thermal infrared, the broader vision literature has encompassed electroluminescence for cell-scale defect characterisation, visible and multispectral

imagery for context and soiling assessment, and UAV-enabled pipelines that integrate detection with geospatial mapping and maintenance workflows. Across these modalities, deep learning detectors and segmenters have been adapted to PV textures, although performance has remained sensitive to acquisition conditions and domain shift, with reviews calling for standardised datasets, unified metrics, and cross-site validation to ensure transferability to practice [Islam et al., 2023, Polymeropoulos et al., 2024, Aghaei et al., 2025]

2.9.1.1 Thermal Infrared Inspection

[Salazar and Macabebe, 2016] investigated the use of infrared thermography to detect hotspots in PV modules. Their method used thermal image segmentation and clustering to identify overheated cells. The technique is advantageous because it provides direct physical evidence of localized heat anomalies without dismantling the module. However, the method did not employ AI, which limited its ability to classify multiple types of faults beyond temperature-related issues. Consequently, problems such as dust accumulation, shading, or micro-cracks remain undetected.

In another study, [Kirubakaran et al., 2022] applied infrared thermal imaging for PV fault diagnosis. Their system analyzed thermal patterns to identify anomalies across modules. While this method provided detailed visualization, it was heavily dependent on controlled imaging conditions and still lacked AI-based generalization across diverse environmental situations.

2.9.1.2 Broader Vision Literature

Research on computer vision inspection for photovoltaic systems has largely centred on thermal infrared and EL, with RGB or multispectral imagery used for context or mapping. Surveys consistently report that one-stage detectors (e.g., YOLO variants) achieve strong hot spot localisation on thermal frames, while EL enables micro-crack, shunt and potential-induced degradation diagnosis at cell scale [Islam et al., 2023]. Representative studies include thermal UAV detection pipelines that adapt YOLO to PV textures, reporting high mean average precision and real-time feasibility, yet often on private, site-specific datasets that limit generalizability [Zheng et al., 2022, Xie et al., 2024]. EL classification

work demonstrates near 90 percent accuracy in laboratory settings but requires specialised capture and typically lacks outdoor validation [Deutsch et al., 2019]. Multispectral UAV efforts that co-register RGB and long-wave IR improve localisation and support IV curve verification, although they introduce registration errors and additional operational complexity [Duan et al., 2024]. Recent reviews emphasise that public, standardised datasets and unified evaluation metrics are still scarce, which complicates cross-site benchmarking and technology transfer to operations and maintenance (O&M) practice [Polymeropoulos et al., 2024, Aghaei et al., 2025].

2.9.2 IoT-Enabled Solar Monitoring Systems

[Asnil et al., 2023] developed a real-time IoT-based monitoring system for photovoltaic modules. Their approach integrated sensors for voltage, current, temperature, light, and power, transmitting data via cloud platforms for real time tracking. The system successfully improved preventive maintenance, but it was limited to descriptive monitoring. Without AI, it could not distinguish between different types of anomalies such as shading, dust, or cracks.

Similarly, [Sutikno et al., 2021] proposed an IoT-enabled PV parameter monitoring framework using NodeMCU, sensors, and ThingSpeak cloud integration. Their method enhanced the scalability and accessibility of PV farm monitoring. However, it remained descriptive and did not include intelligent analytics for automated fault classification or root-cause diagnosis.

2.9.3 Machine Learning Based Anomaly Detection from Arduino Sensor Nodes

The system instrumented selected modules with Arduino-class sensor nodes that sample electrical and environmental quantities at panel level (e.g., DC voltage and current, panel surface temperature, ambient temperature and a simple irradiance proxy). The node performs lightweight pre-processing on device, including timestamping, outlier rejection and minute-level aggregation, and then transmits records to the central dashboard via a gateway. A store-and-forward buffer preserves data during connectivity loss so that alerts

and periodic reports remain consistent in adverse conditions. This architecture aligns with AIoT practices for PV monitoring, where edge nodes push reliable telemetry to cloud services for analytics and maintenance decision support [Ait Abdelmoula et al., 2023, Boucif et al., 2025].

The anomaly engine in this system applies a hybrid detection strategy that combines physics-aware baselines with machine learning. First, estimate the expected panel behaviour using a simple temperature- and irradiance-conditioned model (e.g., a gradient-boosting regressor or shallow neural network trained to predict current and voltage), or a neighbour-median reference when irradiance is missing. then compute residuals and robust scores (e.g., rolling median absolute deviation) and feed multivariate windows into an unsupervised detector such as Isolation Forest or an autoencoder to flag departures that are unlikely under normal operation. The output is a calibrated anomaly score, a human-readable root-cause hint (e.g., “current deficit at normal temperature”), and a severity level with hysteresis to avoid alert flapping. The dashboard receives real-time alerts when thresholds are crossed, and it also receives a periodic report that summarises uptime, number and duration of anomalies, drift in residuals, and recommended follow-ups. This design is consistent with predictive-maintenance literature that stresses anomaly detection, data quality control and operational reporting, while addressing known gaps in external validity by using neighbour consensus and simple physics baselines rather than site-specific thresholds alone [Abdulla et al., 2024, Aghaei et al., 2025, Salazar and Macabebe, 2016].

2.9.4 Embedded Monitoring and Edge Platforms

Prior research highlights two main approaches relevant to PV monitoring. The first involves module-level nodes installed directly on solar panels to collect data and transmit it securely to a nearby gateway. Existing studies show that compact embedded boards can operate in outdoor environments and provide useful panel health information [Chaychian et al., 2017]. The second approach is edge computing, in which a small computer is deployed near the solar array to buffer incoming data and perform basic analytics before forwarding summaries to the cloud. This architecture can improve response time and system

resilience, particularly in environments with limited or unstable internet connectivity [Abdulla et al., 2024].

In addition, reviews discuss the role of module-level power electronics (MLPE), which enable per-panel telemetry and support more detailed diagnostics. However, these systems also increase complexity, and their practical benefits depend on site-specific operating conditions [IEA-PVPS, 2021]. For hardware selection, benchmarking initiatives such as MLPerf provide useful latency and energy efficiency measurements across computing devices, which can help shortlist potential accelerators for embedded analytics. Nevertheless, such benchmarks remain task-agnostic, meaning that PV-specific evaluation is still necessary to verify reliability under field conditions such as heat, dust, and limited power availability [Hardonag, 2024].

Based on this literature, the proposed system adopts a simplified embedded and edge-based design. A monitoring chip was installed at the panel level to measure voltage, current, panel temperature, and a basic proxy for irradiance. performed lightweight feature extraction, ran a compact model, and transmitted status updates alerts to a nearby gateway.

During the initial phase, computationally intensive processing remained on the ground gateway or in the cloud in order to reduce device-level complexity. This design follows the main ideas presented in the related literature while maintaining a level of simplicity and feasibility appropriate for a senior project. The practical implementation stage began after the design phase and initial experimental validation are completed.

2.9.5 Related Work Comparison and Evaluation

From the reviewed literature, non-AI inspection and monitoring approaches provide useful but limited solutions. Computer vision methods such as thermal imaging offer physical fault detection but require manual interpretation and cannot generalize across multiple fault types. IoT sensor-based systems ensure real-time monitoring of PV performance but, without AI integration, cannot diagnose or classify the causes of performance degradation.

This comparative evaluation highlights a significant research gap: while

traditional methods are reliable in specific domains, their lack of intelligent automation and scalability prevents them from meeting the requirements of fully autonomous PV fault detection and monitoring systems.

Table 2.1: Related Work for Solar Panel Fault Detection and Monitoring

Related Research	Reference	Method / Data	Result	Limitation
Infrared Thermography for Hotspot Detection	[Salazar and Macabebe, 2016]	Thermal images + segmentation	Accurate hotspot localization	Cannot detect dust, cracks, or shading; no AI classification
Infrared Thermal Imaging for PV Diagnosis	[Kirubakaran et al., 2022]	Infrared thermal imaging	Identified anomalies across modules	Sensitive to environmental conditions; lacks AI
IoT-Based PV Monitoring	[Asnil et al., 2023]	IoT sensors + cloud-based monitoring	Continuous real-time monitoring	No AI; cannot distinguish fault causes
IoT Parameter Monitoring System	[Sutikno et al., 2021]	NodeMCU + ThingSpeak integration	Scalable monitoring framework	Descriptive only; lacks automated diagnostics
ST-YOLO: Defect Detection for PV Modules in Thermal Imagery	[Xie et al., 2024]	YOLOv8 variant (SC-Conv + triplet attention) on thermal IR images of PV arrays	mAP at 0.5 $\approx$ 96.6%; precision improved over baseline; parameters reduced by $\sim$ 15%	Private single-site dataset; generalisation to other farms and cameras remains unproven
Lightweight Hot-Spot Fault Detection Model of PV Modules	[Zheng et al., 2022]	Improved YOLOv5 (ShuffleNetV2, FPN) on $\sim$ 5.6k thermal image frames	mAP $\approx$ 98.1%; 49 FPS; 3.71M parameters	Possible frame correlation; limited weather diversity; evaluated on one farm
Automatic Classification of Defective PV Module Cells in EL Images	[Deitsch et al., 2019]	CNN vs SVM on 1,968 EL cell crops (mono-/poly-Si)	CNN 88.42% (cell level); SVM 82.44%	Laboratory EL only; no outdoor validation; taxonomy differs from O&M practice
PV Panel Faults via Multispectral Imaging	[Duan et al., 2024]	Thresholding + morphology on co-registered RGB + LWIR mosaics; IV-curve verification	IV-verified detections; accuracy drops on small or weak hotspots	Band registration errors; extra effort for IV testing; no aggregate accuracy reported
Vision-Based Monitoring Review	[Polymeropoulos et al., 2024]	Review of RGB/IR inspection pipelines, detectors, segmenters, and metrics	Synthesises CV methods and KPIs	Notes lack of unified datasets and benchmarks
AI in PV Fault Identification: Systematic Review	[Islam et al., 2023]	Systematic review across imaging and electrical signals	Catalogues AI techniques and trends	Dataset scarcity; cross-site domain shift
Autonomous Intelligent Monitoring of PV Systems	[Aghaei et al., 2025]	Multidisciplinary review (, IV, IR/EL, AI)	Roadmap for PV monitoring with AI	Calls for standard public datasets and validation
PV Systems O&M: Review and Future Directions	[Abdulla et al., 2024]	O&M strategies, KPIs, and PdM trends	Identifies PdM opportunities and gaps	Limited real-world prognostics; little prescriptive optimisation
Sustainable Edge Computing Framework for PV Monitoring	[Ait Abdelmoula et al., 2023]	Edge SBC + IoT stack near array; buffering + analytics	Shows edge acquisition and processing feasibility	Scalability and maintainability challenges; few PV-specific benchmarks
Trusted Monitoring for Smart Solar Panels	[Chaychian et al., 2017]	On-panel embedded prototype; secure telemetry	Demonstrates on-panel sensing and secure data path	Early prototype; limited power and thermal analysis
Performance of New PV System Designs (MLPE Overview)	[IEA-PVPS, 2021]	Survey of microinverters and DC optimisers with per-panel telemetry	Enables granular diagnostics (per-panel MPPT and monitoring)	Added system complexity; benefits depend on site context
MLPerf Inference / Tiny Benchmark Suites	[Hardonag, 2024]	Standard latency, throughput, and power benchmarks for SoCs/MCUs	Supports comparative device ranking to shortlist accelerators	Task-agnostic; requires PV-specific workload validation

2.10 Autonomous Monitoring within the Overall System

Autonomous monitoring in the proposed system has been positioned as a cross-cutting capability that fuses electrical telemetry, on-panel sensing, and image-based inspection using uploaded panel images to maintain performance and safety across photovoltaic assets. Module-level nodes collected current, voltage, light, and temperature together, after which lightweight preprocessing is executed on

the edge gateway to ensure buffering and resilience during connectivity loss. These data streams are then combined with image-based findings, so that complementary fault signatures such as hotspots, soiling, and micro-cracks can be captured with broader coverage than any single modality alone [Islam et al., 2023, Duan et al., 2024].

Anomaly detection has been designed to follow a hybrid strategy. Expected behaviour is estimated using physics-aware baselines conditioned on temperature, and the resulting residuals are supplied to machine learning detectors for robust outlier scoring. This design choice has been motivated by recent operations and maintenance reviews that emphasise data quality control, explainable alerts, and the need for methods that generalise across sites rather than relying on fixed thresholds [Abdulla et al., 2024]. Image-processing pipelines contribute localised evidence for hotspot and surface defects, while electrical time-series data provide continuous coverage for slow drifts and intermittent faults.

End-to-end automation is completed through a central dashboard where severity-calibrated alerts, root-cause hints, and recommended actions are presented to operators. The dashboard aggregates uptime, anomaly counts, and residual trends into weekly and monthly reports to support preventive maintenance planning. By coupling edge processing with cloud analytics, the overall system has been structured to achieve low-latency responses in the field while retaining the scalability required for larger photovoltaic deployments. This architecture aligns with emerging roadmaps for autonomous PV monitoring that call for multi-modal sensing, AI-assisted diagnostics, and validated workflows integrated with operations and maintenance practice [Aghaei et al., 2025, Islam et al., 2023].

Table 2.2: Autonomous Monitoring within the Overall System

Aspect	Approach in proposed system	Purpose	Key strengths	Considerations	Reference
Module-level sensing	On-panel nodes (V, I, panel temp, irradiance proxy) with time-stamping and outlier rejection	Continuous panel-level telemetry	Fine granularity; detects slow drifts and intermittent faults	Sensor calibration; outdoor robustness	[Islam et al., 2023, Abdulla et al., 2024]
Edge gateway	Buffering, minute-level aggregation; basic QC; runs lightweight models when offline	Low-latency alerts; resilience to connectivity loss	Reduced data loss; faster responses	Limited compute/power at edge	[Abdulla et al., 2024]
Periodic imaging	RGB and thermal image analysis integrated with the monitoring workflow	Localise hotspots, soiling, and visible defects	Spatial coverage; visual evidence	Image quality and environmental sensitivity	[Duan et al., 2024, Islam et al., 2023]
Hybrid anomaly detection	Physics-aware baselines $\rightarrow$ residuals $\rightarrow$ ML (Isolation Forest/autoencoder)	Robust detection across sites	Explainable alerts; fewer false alarms	Model drift; needs periodic recalibration	[Islam et al., 2023, Abdulla et al., 2024]
Fusion & decision layer	Combine electrical residuals with image findings	Stronger fault confirmation and triage	Multimodal confidence; fewer missed faults	Synchronisation of data and time alignment	[Aghaei et al., 2025]
Alerts & reporting	Severity-calibrated alerts, root-cause hints, weekly/monthly KPIs	Actionable O&M decisions	Practical workflows; preventive maintenance	Threshold tuning to avoid alert fatigue	[Abdulla et al., 2024]
Scalability & roll-out	Edge + cloud architecture; panel $\rightarrow$ string $\rightarrow$ farm scaling	Support small- to utility-scale sites	Flexible deployment	Requires device management and updates	[Aghaei et al., 2025]

2.11 Required Skills

The implementation of the SolarSense system requires a combination of technical skills to develop, integrate, and validate its components effectively. The most important skills include:

- **AI & ML Skills:** The ability to build and train ML models for detecting solar panel faults such as cracks, hotspots, dust accumulation, and performance anomalies.
- **Computer Vision:** Knowledge of image processing and CV techniques to analyse RGB and thermal images for fault identification and localisation.
- **Embedded Systems & IoT:** Skills in developing and programming embedded devices with sensors for real-time data collection, communication, and local analysis.
- **Dashboard & Platform Development:** The ability to design and implement a user-friendly dashboard for real-time monitoring, alerts, and predictive maintenance insights.
- **Data Analysis & Preprocessing:** Skills in handling sensor and image datasets, including cleaning, preprocessing, feature extraction, and

preparing data for model training and evaluation.

- **System Integration & Validation:** Knowledge of integrating all system components, including AI models, sensors, edge devices, and the monitoring platform, into a reliable and scalable solution, as well as validating overall system performance.

2.12 Conclusion

This chapter has outlined the theoretical foundations and prior research that support the development of intelligent solar monitoring systems. It introduced the key technologies AI, ML, DL, CNN, CV, IoT, and PV systems that form the backbone of fault detection and predictive maintenance. It also examined related work, comparing computer vision-based inspections and IoT-enabled monitoring, and identified their strengths and shortcomings.

From the reviewed literature, it is clear that while traditional inspection methods can detect specific types of faults, they lack the scalability and automation required for modern solar farms. Similarly, IoT-based monitoring systems provide real-time data but require AI integration to achieve automated fault classification and decision-making. These insights underscore the importance of developing a hybrid system, like the proposed SolarSense , which integrates CV, ML, and IoT into a cohesive and scalable solution.

CHAPTER 3

DATASET COLLECTION AND SPECIFICATIONS

3.1 Introduction

A major part of developing any ML, DL, or CV based system is the collection and preparation of a relevant and representative dataset. These data support training, validation, and testing of the analytical models used in SolarSense. For the SolarSense project, the accuracy and efficiency of the proposed system depend on high-quality datasets that represent real-world solar panel operating conditions. The project aims to develop an intelligent monitoring framework capable of detecting dust accumulation, hotspots, snow coverage, and other surface faults on PV modules, while analyzing sensor data such as temperature, current, and voltage to predict power generation and assess performance. Two complementary datasets are employed. The first is an RGB image dataset of PV modules captured using standard cameras under various surface and environmental conditions to support CV-based fault and condition classification. The second is a numerical sensor dataset containing time-series readings of environmental and electrical parameters for performance modeling and fault correlation. By integrating visual and numerical data, the system connects physical surface conditions with performance metrics, enabling accurate prediction, diagnosis, and optimization of solar panel operation.

This chapter outlines the datasets used in the SolarSense project. Section 3.2 presents available datasets; Section 3.3 details specifications; Section 3.4 describes preprocessing; Section 3.5 discusses constraints; and Section 3.6 concludes the chapter.

3.2 Available Datasets

To develop and evaluate the SolarSense system, two main types of datasets were utilized: image-based and numerical based datasets. These datasets were carefully selected from publicly available sources to ensure diversity, reliability, and compatibility with the project’s objectives.

The image datasets contain a variety of clean and faulty solar panel images captured under different environmental and lighting conditions. They are used to train the computer vision models to detect cracks, dust, and other surface anomalies.

The numerical datasets include real sensor readings such as voltage, current, and power generation values recorded over time. These are essential for training the anomaly detection models that operate within the embedded chip to monitor panel performance in real time.

Table 3.1: Solar Panel Datasets Overview

Dataset	Dataset Type	Dataset Source	Number of the Images/Numeric rows	Resolution/sampling rate	Reference
Solar Panel Images Clean and Faulty Images	Images	Kaggle	891	various; mode 720×960 (n=58); range 149×110–6240×5376	[pythonafroz, 2023b]
Solar panel clean and faulty images	Images	Kaggle	907	various; mode 720×960 (n=64); range 149×110–6240×5376	[pythonafroz, 2023a]
Solar Power Generation Data	numerical	Kaggle	142917	15 minute(s)	[rxhulshxrmx, 2025]
Solar Panels Performance	numerical	Kaggle	32005	Unknown	[anikannal, 2020]
Solar PV Panel Detection	Images	Roboflow Universe	1586	various; multiple resolutions (not explicitly specified)	[whereareyousolarpanel, 2023]

3.3 Dataset Specifications

The data used in this project comprise two complementary components designed to support the analysis, modeling, and optimization of PV systems. The first component focuses on visual imagery representing the physical and environmental conditions of PV modules, while the second component consists of numerical sensor records capturing electrical and environmental parameters that influence energy generation. Together, these datasets enable the development of robust ML and CV models capable of detecting faults, classifying surface conditions, predicting power output, and assessing system performance under varying real-world conditions. The following subsections describe the specific characteristics and structural requirements for each dataset type.

3.3.1 Image Dataset Specification

The visual dataset used in the SolarSense computer-vision pipeline consists of colour RGB images of solar panels categorized into image-level surface condition classes. The implemented model is designed to classify the overall condition of a panel image rather than precisely localize defects using bounding boxes. The final dataset classes include Clean, Dusty, Electrical-damage, Physical-Damage, and Snow-Covered.

- Each image shall be assigned to one clearly defined surface-condition class, such as Clean, Dusty, Electrical-damage, Physical-Damage, or Snow-Covered. These labels are used for image-level classification.
- Images shall be readable, correctly labelled, and representative of the visible condition of the solar panel surface. Corrupt or unusable files should be removed before training.
- Images should be captured or selected under varied lighting, viewing angles, and environmental conditions where possible, in order to improve the model's ability to generalize to real inspection images.
- Beside duplicate images, it should be identified and removed using perceptual hashing to reduce data leakage and prevent the model from overfitting to repeated or highly similar samples.
- Images shall be converted to RGB format and resized to the input size required by the training pipeline. In the implemented pipeline, images are resized to 640×640 pixels.
- The dataset shall be divided into training, validation, and testing subsets. In the implemented pipeline, the split is 70% training, 15% validation, and 15% testing.
- Data augmentation shall be applied only to the training subset. Augmentation techniques may include random resized cropping, rotation, affine transformation, horizontal flipping, brightness and contrast adjustment, blur, noise, fog, rain, and colour variation.

- Validation and testing images shall remain unaugmented to provide a fair evaluation of the model on unseen data.
- Since the current dataset uses image-level labels, the model output represents the overall panel condition. Precise defect localization is not provided by the final classification-first pipeline.

3.3.2 Numerical Dataset Specification

The numerical dataset used in the SolarSense machine-learning pipeline consists of time-series telemetry records that describe the electrical and environmental behaviour of the monitored solar panel. Each record is indexed by a timestamp and contains the main sensor readings required by the implemented model: voltage, current, and temperature. The power output is derived from the telemetry readings by multiplying voltage by current, and this calculated power value is used as the main prediction target for performance analysis.

- Each telemetry record shall include a timestamp, voltage reading, current reading, and temperature reading. These fields represent the required input features for the implemented machine-learning model.
- The timestamp shall be used to order the telemetry records chronologically, allowing the data to be converted into sequential time windows for time-series prediction.
- Power output shall be calculated from the measured voltage and current values using the relationship:

$$Power = Voltage \times Current$$

This derived power value is used as the main target variable for model training and evaluation.

- The dataset shall support regression-based prediction, where the model learns to predict expected power output from recent telemetry patterns.
- For model preparation, the telemetry records shall be sorted by timestamp,

cleaned to remove unusable or incomplete records, and scaled using standard normalization techniques.

- The implemented pipeline uses voltage, current, and temperature as input features. Other variables, such as irradiance, humidity, or energy yield, are not used as final model input features in the current implementation.
- The time-series data shall be transformed into fixed-size sequential windows. In the implemented model, a window size of 20 time steps is used to prepare the input sequences for the LSTM model.
- The dataset shall be divided into training, validation, and testing subsets to evaluate the model on unseen telemetry records and reduce overfitting.
- Anomaly detection shall be performed indirectly by comparing the predicted power output with the actual calculated power. Large prediction errors are treated as possible abnormal panel behaviour.
- Measurement units and variable definitions should be documented clearly to support reproducibility and correct integration with the backend telemetry API and database schema.

3.4 Dataset Collection

This section documents the two complementary modalities available in the project’s data, which are numerical telemetry describing the environmental and electrical behavior of photovoltaic PV plants, and image data representing panel conditions.

3.4.1 Numerical Datasets

Time-stamped records are available from weather sensors and generation exports. The weather files include a timestamp field and plant or device keys alongside meteorological measurements such as ambient temperature, module temperature, and an irradiation or irradiance field. The generation files include a timestamp field and plant or device keys alongside electrical measurements such as DC power and AC power, as well as energy counters for daily or total yield[anikannal, 2020, rxhulshxrmx, 2025].

- Keys present: DATE_TIME, PLANT_ID, SOURCE_KEY.
- Weather fields examples: AMBIENT_TEMPERATURE, MODULE_TEMPERATURE, IRRADIATION .
- Generation fields examples: DC_POWER, AC_POWER, DAILY_YIELD, TOTAL_YIELD.

3.4.1.1 Preprocessing Workflow

The following preprocessing steps are applied according to the implemented numerical-data pipeline:

1. **Data loading:** Telemetry records are loaded either from the database or from a CSV file. The expected fields are timestamp, voltage, current, and temperature.
2. **Timestamp parsing and ordering:** If a timestamp column is present in the CSV file, it is parsed as a datetime value. All records are then sorted chronologically by timestamp to preserve the time-series order.
3. **Power calculation:** If the power column is not already available, power is calculated from the measured voltage and current values using:

$$Power = Voltage \times Current$$

This calculated power value is used as the main prediction target.

4. **Feature selection:** The implemented model uses voltage, current, and temperature as input features. Other variables, such as humidity, DC power, AC power, and energy yield, are not used as final input features in the current model.
5. **Data validation:** Records with missing or unusable required values are excluded from model processing. At runtime, the backend also rejects telemetry records with missing, non-numeric, null, or non-finite voltage, current, or temperature values.

6. **Train–validation–test split:** The dataset is divided chronologically into 80% training, 10% validation, and 10% testing subsets. This split preserves the temporal order of the telemetry records.
7. **Scaling:** A `StandardScaler` is fitted on the training feature values and then applied to the validation and testing subsets. A separate scaler is also fitted for the target variable and saved for production inference.
8. **Sequence creation:** The scaled telemetry records are converted into fixed-length time-series windows. In the implemented pipeline, each input sequence contains 20 previous time steps.
9. **Model-ready output:** The final processed data consists of sequential input windows containing voltage, current, and temperature values, paired with the target power value for supervised LSTM regression.

3.4.2 Images Datasets

Three image datasets of solar panels were available for use in the project. Within the available datasets, per-image class labels (e.g., clean, faulty) are provided in the Kaggle datasets and an image dataset is available from Roboflow [pythonafroz, 2023a, pythonafroz, 2023b, whereareyousolarpanel, 2023].

3.4.2.1 Preprocessing Workflow

1. **Curation and integrity checks:** Image files are checked for readability, and unusable or corrupt files are excluded from the dataset. The dataset is organized into class folders representing the implemented image-level classes.
2. **Near-duplicate removal:** Near-duplicate images are identified using perceptual hashing. In the implemented pipeline, difference hashing is used to compare images and remove highly similar samples before dataset splitting.
3. **Resolution and colour space:** Images are read, converted to RGB, and resized to the model input size. In the implemented pipeline, images are standardized to 640×640 pixels.

4. **Classification dataset preparation:** The dataset is prepared as an image classification dataset rather than a bounding-box detection dataset. Each image is assigned to one class folder, such as Clean, Dusty, Electrical-damage, Physical-Damage, or Snow-Covered.
5. **Split strategy and leakage control:** The unique original images are split into training, validation, and testing subsets using a 70%, 15%, and 15% split. Splitting is performed after duplicate removal to reduce the risk of similar images appearing across different subsets.
6. **Data augmentation:** Data augmentation is applied only to the training subset. The augmentation process includes transformations such as random resized cropping, rotation, affine transformation, horizontal flipping, brightness and contrast adjustment, gamma correction, blur, noise, fog, rain, and colour variation.
7. **Evaluation integrity:** Validation and testing images are kept unaugmented so that model performance is measured on unseen original images rather than augmented copies of the training data.

3.5 Dataset Limitations

The SolarSense project utilizes two primary datasets: an image dataset for visual fault detection using computer vision models, and a numerical dataset collected from sensors for real-time performance analysis. Each dataset presents specific limitations that can influence the accuracy, generalization, and reliability of the developed models. This section outlines the major constraints observed during data preprocessing and training.

3.5.1 Image Dataset Limitations

The image dataset consists of RGB images of solar panels captured using standard cameras under various environmental conditions. While this dataset provides valuable information for detecting surface defects such as cracks and hotspots, certain limitations were identified that may affect model performance. To ensure data quality and reduce noise, the following conditions were excluded from

training and testing

- **Lighting Variations:** Extreme brightness or shadows in images can confuse the model when detecting cracks or hotspots. Images captured under overexposed or underexposed conditions were removed to maintain consistency.



Figure 3.1: Example of Lighting Variation Problem

- **Camera Angle and Distance:** The angle and distance between the camera and the solar panels significantly affect image clarity. Images taken from steep angles or from excessively far distances were excluded, as they may lead to inaccurate fault localization



Figure 3.2: Example of Steep Angle Problem



Figure 3.3: Example of Too Far Distances Problem

3.5.2 Numerical Dataset Limitations

The numerical dataset is derived from sensor readings collected at the panel level, including voltage, current, temperature, and irradiance data. This dataset plays a critical role in monitoring electrical performance and detecting anomalies. However, several data related limitations were identified during preprocessing and model development:

- **Missing or Incomplete Data:** Some sensor readings, such as voltage or current, were missing due to communication interruptions between sensors and the embedded chip. These missing values can lead to biased model predictions and were handled using interpolation or removal techniques.

Example: Missing temperature readings during high-load periods.

- **Outliers and Abnormal Patterns:** Unexpected spikes or drops in sensor values may occur due to electrical interference, sensor malfunction, or temporary shading. These outliers must be detected and smoothed during preprocessing.

Example: A sudden drop in voltage caused by temporary connection loss

3.6 Conclusion

This chapter presented the datasets used in the SolarSense project, focusing on two complementary components: RGB image datasets representing the visual condition of PV modules and numerical time-series datasets capturing environmental and electrical parameters. Together, these datasets form the basis for developing ML, DL, and CV-based models for fault detection, power prediction, and performance evaluation. Key challenges such as missing values, inconsistent resolutions, and irregular sampling were addressed through preprocessing, synchronization, and normalisation. The next chapter presents the system requirements and design architecture of the SolarSense framework and explains how these datasets are integrated into the overall model pipeline.

CHAPTER 4

SYSTEM REQUIREMENTS AND SYSTEM DESIGN

4.1 Introduction

Engaging all system components during development is crucial to achieving an effective and dependable design. The SolarSense system integrates embedded sensors on PV panels with DL and CV techniques for automated fault detection and performance evaluation, while end users interact through a centralized dashboard that displays real-time metrics, alerts, and inspection updates. The design and implementation of the SolarSense framework therefore focus on intelligent monitoring of PV panels using both panel-level sensing and image inspections.

This chapter outlines the key stages involved in developing the SolarSense system. Section 4.2 describes the information gathering methods, including the literature review, stakeholder survey, and expert interview. Section 4.3 specifies the system requirements, covering functional, non-functional, data, software, and hardware aspects. Section 4.4 presents the initial design models, such as the Entity–Relationship diagram, database schema, class diagram, use case diagram, flowchart, sequence diagrams, state diagrams, and data-flow diagrams. Section 4.5 introduces the prototype design, encompassing the software user interface and the hardware chip module. Section 4.6 highlights potential near future developments.

4.2 Information Gathering Methods

Collecting reliable information is an essential step in defining and shaping the requirements of the proposed system. It ensures that the solution addresses real needs, reflects practical challenges, and is grounded in accurate understanding

rather than assumptions. In this project, information was gathered from multiple sources to build a clear picture of the problem context and the feasibility of the system. Academic resources and published studies were reviewed to understand existing approaches in solar monitoring, fault detection, and the integration of AI and IoT technologies. Additionally, feedback from potential stakeholders was collected to explore expectations and usability considerations. To complement this, direct interviews were conducted with a domain expert in renewable energy systems, providing valuable insights into real world operational challenges, sensor reliability concerns, and effective monitoring practices. Together, these methods provided a strong foundation for defining system requirements, refining the project scope, and guiding the design decisions of SolarSense.

4.2.1 Literature Review

The literature review served as a foundational step in the development of the SolarSense system. A structured understanding of previous research related to solar panel monitoring, embedded sensing, and AI-based fault detection was established through this review. Existing studies were examined in order to understand how issues such as panel soiling, hotspots, and micro-cracks have been addressed, and to identify the technologies commonly used for fault detection and analysis, including RGB imaging, machine learning models, and IoT-based monitoring frameworks.

Through the literature review, different technical approaches were assessed and their advantages and limitations were compared. This process supported the identification of suitable methods for the project, particularly the integration of embedded sensors for real-time panel-level monitoring with image-based inspection techniques. Although the use of drones was considered during the survey and requirements-gathering stage as a possible method for image acquisition, a more flexible approach based on image capturing without drone dependency was ultimately adopted. This decision was made in order to allow inspection images to be acquired through different means according to availability and operational needs, while still supporting computer vision analysis within the system. In addition, prior work highlighted the importance of performance metrics such as

panel temperature, voltage, and current trends in evaluating photovoltaic system health.

The literature review also guided dataset selection and preparation. Publicly available datasets containing both clean and defective solar panel images, as well as real performance measurement data, were examined. The size, diversity, and resolution of these datasets were considered in order to support the selection of training data aligned with the objectives of the system.

Overall, the literature review contributed to shaping the technical direction of the project, establishing the rationale behind the system design choices, and identifying the most relevant research methodologies to adopt. The detailed literature review is presented in Chapter 2, starting from Section 2.1.

4.2.2 Stakeholder Survey

This survey aims to gather insights and practical experiences from professionals in the field of solar panel maintenance. Responses were collected from domain practitioners and engineers to assess perceived challenges with manual inspection, the expected benefits of smart monitoring, and preferences for panel-level sensing, drone-assisted inspection, and dashboard content. Participation was voluntary and questions combined multiple-choice ratings with targeted feature selections to enable quantitative summarization. The results are intended to inform requirement prioritization and interface design for the SolarSense prototype.

- **60%** of participants come from the Renewable Energy field, while the remaining **40%** are equally distributed between Computer Engineering/AI and Architecture backgrounds (**20%** each).
- **100%** of participants indicated that they already have knowledge about solar PV systems.
- Regarding PV maintenance experience, **40%** rated their knowledge as Moderate, while **20%** rated it as Basic, **20%** as Good, and **20%** as Excellent. No participant selected None.
- **80%** of participants agreed that manual PV inspection requires significant time and effort (ratings 4–5), whereas **20%** gave a lower rating (rating 2).

- **80%** believed that detecting micro-cracks and hotspots manually is difficult (ratings 4–5), while **20%** selected rating 1.
- **80%** strongly agreed that implementing a smart monitoring system would improve fault detection accuracy (rating 5), while **20%** gave a moderate rating (rating 3).
- **60%** of respondents considered the use of AI, computer vision, and drones highly effective for large-scale solar farms (rating 5), while **20%** selected rating 4 and **20%** selected rating 2.
- **80%** of participants expected the proposed system to reduce operational costs (ratings 4–5), whereas **20%** selected rating 1.
- **100%** of participants supported deploying sensors at the panel level in order to improve the speed and accuracy of fault detection.
- When asked about drone-based inspection, **80%** stated that its effectiveness depends on the system size, while **20%** answered Yes. No participant selected No.
- For dashboard design, **100%** of participants selected fault alerts, **80%** selected panel temperature, energy output, and fault location, and **60%** selected fault type. In addition, **20%** suggested displaying current-related data, and another **20%** requested a feature related to panel safety monitoring.

4.2.3 Stakeholder Interview

To support the development of the proposed system and to ensure that the design aligns with real operational needs in the solar energy field, a professional interview was conducted with an Associate Professor at King Abdulaziz University, who is also the Director of the Center of Research Excellence in Renewable Energy and Power Systems. The interview was intended to gather expert insights related to solar panel performance monitoring, common system failures, and practical considerations for implementing sensor-based fault detection. The interview questions and responses are provided in Appendix 2.

The interview began with a presentation of the project concept. The expert confirmed that the most common issues in solar panels include hot-spots, partial shading, and dust accumulation, and explained that in current solar farms these problems are usually detected through SCADA performance monitoring, with visual or thermal inspections performed only when irregularities appear.

A question was raised as to whether monitoring should be performed at the string level or panel level, and the expert noted that panel-level monitoring can be more effective when detailed fault localization and predictive maintenance are required, which aligns well with the project goals.

Regarding electrical parameters, it was emphasized that voltage and current are primary indicators of panel performance, while temperature, irradiance, and humidity measurements play a supporting role in diagnosing performance variations caused by environmental conditions. It was also noted that sensor accuracy is critical, and that low-quality sensors may lead to misleading fault detection results.

From the hardware perspective, the expert highlighted challenges such as:

- Extreme rooftop temperatures, which can damage electronics if not properly protected.
- The need for physical stability and secure mounting to avoid accidental damage during maintenance.
- Ensuring stable power availability, with the use of small energy storage recommended to handle early morning and sunset periods.

In terms of communication, it was noted that Wi-Fi may be suitable, but that care must be taken to handle data loss or unstable connectivity.

Overall, the interview provided validation for the panel-level monitoring approach and offered practical guidance for selecting sensors, designing protective hardware enclosures, and managing real-time data transmission. The expert feedback is expected to directly assist in refining the system architecture and experimental plan.

4.3 Requirement Specification

This section specifies the behavioral and quality expectations for SolarSense so that design, implementation, and verification activities can be guided by explicit, testable statements. The requirements have been structured to reflect the three principal subsystems of the solution, namely panel-level embedded sensing, mission inspection, and a central monitoring dashboard. Each requirement has been written to be unambiguous, observable, and traceable to stakeholder needs gathered during the review activities.

4.3.1 Functional Requirements

The functional requirements define what SolarSense shall do in operation across data acquisition, server-side analytics, automated inspection creation, image-based defect analysis, and operator decision support. They describe the capture of electrical and environmental measurements at the panel level, the execution of anomaly detection on the backend server, the automatic creation of inspections for anomalous panels, the upload and analysis of panel images, and the presentation of unified monitoring information through a central dashboard.

- User Authentication and Profile Management

- R1. SolarSense shall provide user registration, login, logout, and password reset functionality through a secure authentication service.
- R2. SolarSense shall allow users to view and update their profile information including display name.

- Site and Panel Management

- R3. SolarSense shall allow operators to register solar sites and add, update, or remove individual panels within each site, with each panel tracking a health status of OK, Warning, or Fault.
- R4. SolarSense shall provide a per-panel detail view showing telemetry record count, fault count, inspection history, and the latest sensor readings.

- Panel-Level Data Acquisition

- R5. SolarSense shall record voltage, current, and temperature readings for each solar panel at fixed time intervals through a REST telemetry endpoint and compute power output as the product of voltage and current.
- R6. SolarSense shall perform anomaly detection on the backend server using an LSTM neural network model to identify abnormal power output deviations from predicted values.
- R7. SolarSense shall store sensor readings with timestamps and panel identifiers, and persist anomaly detection results including predicted power, prediction error, and severity classification.
- Automated Inspection Creation
- R8. SolarSense shall automatically create an inspection task for any panel where the ML anomaly scan detects abnormal readings, provided no open inspection already exists for that panel.
- Image-Based Inspection
- R9. SolarSense shall allow operators to upload RGB images of the target panels through the web interface for each inspection task.
- R10. SolarSense shall analyze uploaded panel images using a YOLOv8 computer vision model to classify defects into the following categories: Dusty, Electrical Damage, Physical Damage, Snow Covered, or Clean.
- R11. SolarSense shall store inspection results with defect type, confidence score, bounding box coordinates, and model version, and allow operators to dismiss incorrect classifications.
- Central Monitoring Dashboard
- R12. SolarSense shall integrate data from both telemetry anomaly detection and image-based inspections to provide a unified view of panel health and fault history.

- R13. SolarSense shall display real-time monitoring data, panel status indicators, power output trends, and recent anomalies through a responsive web dashboard accessible on desktop and mobile devices.
- R14. SolarSense shall provide role-based access control with administrator and operator roles, where administrators can manage user accounts and roles, and operators can manage inspections and view analytics.

4.3.2 Non-Functional Requirements

The non-functional requirements define the quality attributes that shall govern the user experience and operational robustness of SolarSense. Emphasis is placed on availability so that services remain reachable during routine operation and across common device and browser configurations. Usability expectations ensure that interfaces are consistent, terminology is clear, and guidance is accessible to users with varying technical backgrounds. Performance criteria ensure that views load responsively and analytical outcomes appear promptly.

- Availability

- R1. The system shall remain reachable during normal operation without unexpected interruptions.
- R2. The web dashboard shall be accessible on common desktop and mobile browsers with a responsive layout.

- Usability

- R3. A clear and consistent interface shall be provided with straightforward navigation and terminology.
- R4. Helpful labels, plain language, and concise guidance shall support users with limited technical background.

- Performance

- R5. Dashboard views shall open quickly and remain responsive while displaying key indicators and monitoring data.

R6. Telemetry data and anomaly results shall refresh automatically at short intervals without requiring manual page reloads.

R7. Uploaded images shall display with clarity suitable for reviewing defect detection results.

4.3.3 Data Requirements

The specification of data requirements constituted a fundamental stage in the design of the database for the proposed solar panel monitoring and anomaly detection system. After identifying the functional requirements through literature review, stakeholder consultation, and system analysis, the data needed to support authentication, site and panel organisation, inspection management, telemetry monitoring, and anomaly detection were defined. This stage was necessary to ensure that all essential information would be stored consistently and could support secure access, operational traceability, and analytical processing.

4.3.3.1 User Requirements

User-related data are required to support authentication, role-based access control, profile management, and accountability for system actions such as image uploads and administrative operations. In the implemented design, user information is logically treated as a single user domain, although it is maintained across the authentication, profile, and role-management structures. The required attributes include a unique user identifier, display name, email address, role, and account creation timestamp. Together, these attributes enable effective user administration and basic traceability of user activity. Table 4.1 summarises the principal user attributes and example values.

Table 4.1: User Requirements, Description, and Examples.

Data Field	Description	Example
user_id	Unique identifier assigned to each user.	2f1b8bc0-6c3e-4b2e-9a5b-2a6a9cba9c01
name	Display name used to identify the user within the system.	Aisha Al-Mutairi
email	Email address used for authentication and communication.	aisha@gmail.com
role	Permission level assigned to the user; valid values include <i>admin</i> and <i>operator</i> .	operator
created_at	Timestamp indicating when the user account or profile record was created.	2025-05-15T10:30:00+03:00

4.3.3.2 Site Requirements

The Site entity represents the physical solar installation at which one or more photovoltaic panels are deployed. This entity is used to organise the monitored infrastructure at the highest structural level and to store the basic identification and geographical information associated with each installation. Each site is defined by a unique identifier, a site name, optional latitude and longitude coordinates, and a creation timestamp. Within the database design, the site acts as the parent entity for all panels installed at that location. Table 4.2 presents the site attributes and example values.

Table 4.2: Site Requirements, Description, and Examples.

Data Field	Description	Example
id	Unique identifier assigned to the site.	550e8400-e29b-41d4-a716-446655440000
name	Name or label assigned to the solar installation site.	NEOM Solar Farm - Phase 01
location_lat	Optional latitude coordinate of the site location.	28.1054
location_lng	Optional longitude coordinate of the site location.	35.1237
created_at	Timestamp indicating when the site record was created.	2025-03-01T08:00:00+03:00

4.3.3.3 Panel Requirements

The Panel entity contains the essential information required to identify and monitor each solar panel within the system. Each panel is associated with a specific site and serves as a central entity linked to telemetry records, inspection missions, inspection results, and detected faults. The panel record includes a unique identifier, a foreign key referencing its parent site, an optional panel label, an optional serial number, a status value indicating its current operational condition, and an optional deletion timestamp used for soft deletion. The permissible panel status values are *OK*, *WARNING*, and *FAULT*. Table 4.3 summarises the principal attributes of the panel entity.

Table 4.3: Panel Requirements, Description, and Examples.

Data Field	Description	Example
id	Unique identifier assigned to the panel.	1b6f0534-c5bc-4faf-91d6-2cae03fb6fb7
site_id	Foreign key referencing the site in which the panel is installed.	550e8400-e29b-41d4-a716-446655440000
label	Optional descriptive label used to identify the panel within the site.	Panel A-12
serial_number	Optional manufacturer or equipment serial number of the panel.	SN-2025-00481
status	Current operational condition of the panel; valid values are <i>OK</i> , <i>WARNING</i> , and <i>FAULT</i> .	OK
deleted_at	Optional timestamp used to indicate that the panel has been soft deleted.	NULL

4.3.3.4 Mission Requirements

The Mission entity represents an inspection activity conducted for a specific panel. Each mission is linked to one panel and is used to record the progress of inspection operations and to establish the connection between the panel, uploaded mission images, and subsequent inspection results. The mission record includes a unique identifier, a foreign key referencing the inspected panel, a status value, and a creation timestamp. In the current design, the canonical mission status values are *OPEN* and *COMPLETED*. This entity supports operational traceability by linking inspection activities with the related monitoring and analysis outputs. Table 4.4 presents the mission attributes and corresponding example values.

Table 4.4: Mission Requirements, Description, and Examples.

Data Field	Description	Example
id	Unique identifier assigned to the mission.	024f515c-1068-42aa-9ef0-a73b2ce663c3
panel_id	Foreign key referencing the panel being inspected.	1b6f0534-c5bc-4faf-91d6-2cae03fb6fb7
status	Current state of the mission; valid values are <i>OPEN</i> and <i>COMPLETED</i> .	COMPLETED
created_at	Timestamp indicating when the mission record was created.	2025-05-18T06:30:00+03:00

4.3.4 Software Requirements

The SolarSense system relies on four main software components: the sensor data acquisition layer, the backend application programming interface, the intelligent fault detection modules, and the web-based monitoring dashboard. Sensor readings such as voltage, current, and temperature are collected from the hardware layer and transmitted to the backend for storage and analysis. The backend is

implemented using Python and FastAPI, providing API endpoints for telemetry, panels, missions, faults, inspection results, and user profiles.

The intelligent analysis component includes both machine learning and computer vision modules. The telemetry anomaly detection model is implemented in Python using TensorFlow/Keras together with scikit-learn for data preprocessing and prediction support. The image-based fault detection component uses YOLOv8 through the Ultralytics framework, with OpenCV available as a fallback image-processing mechanism when needed.

The monitoring dashboard is implemented as a web application using React and Vite, enabling operators and administrators to access system data through modern web browsers without requiring additional installation. The frontend also integrates with Supabase services for authentication and related client-side functionality. To ensure proper synchronization between the hardware layer, backend services, intelligent analysis modules, and dashboard interface, the system requires a stable network connection.

4.3.5 Hardware Requirements

The SolarSense prototype utilizes a specialized suite of hardware components designed for high-precision monitoring and real-time data transmission via wireless connectivity. The core of the system is the ESP32 microcontroller, which provides high-speed processing and integrated Wi-Fi capabilities. This allows the hardware module to establish a direct connection with the centralized dashboard for automated telemetry updates. To capture environmental and electrical metrics, the system integrates a voltage sensor module for panel output and an ACS712 (5A) current sensor. The 5A variant was specifically selected to maintain high sensitivity and accuracy for the solar panels used in the prototype. Environmental monitoring is supported by a BH1750 light intensity sensor to show how much sunlight is reaching the panel, and a TMP36 temperature sensor to monitor the surrounding thermal conditions.

The design includes a solar panel coupled with a dedicated battery and a battery charger module. This configuration allows the system to harvest energy while maintaining a stable power supply for the electronics. A 16×2 LCD display

with an I²C interface is incorporated to provide immediate local visual feedback of the measured parameters directly on the hardware unit. All components are interconnected using a breadboard and jumper wires to facilitate prototyping and testing of the hardware setup.

Furthermore, a computing device such as a laptop or workstation is required to interface with the ESP32 through a USB connection. This device is responsible for collecting sensor data via serial communication and transmitting the data to the system database for storage and analysis. For the computer vision subsystem, a standard camera is required to take pictures for analysis. For accessing the monitoring dashboard, operators may use any personal device such as a laptop, tablet, or smartphone running Windows, MacOS, Linux, Android, or IOS, as no specialized hardware capabilities are required beyond basic internet connectivity.

4.4 Initial Design

This section presents the initial design of the SolarSense system by illustrating how the system components interact and how data flows between them. The design phase helps define the system structure before implementation, ensuring clarity in functionality and communication between modules. It begins with the Entity-Relationship Diagram to outline the main data entities and their relationships. Then, the Database Schema shows how this design is translated into tables and keys. The Class Diagram represents the system's software structure, while the Use Case Diagram clarifies how different users interact with the system. The Flow Chart and Sequence Diagram describe the operational logic and step by step execution of core processes. Together, these diagrams provide a clear blueprint for system development and integration.

4.4.1 Entity-Relationship Diagram

The ER diagram illustrates the database structure of the proposed solar panel monitoring and inspection system. Each site contains one or more panels, and each panel has a status that can be classified as ok, warning, or fault. Every panel is associated with multiple telemetry records that store measurements such

as voltage, current, temperature, light, and timestamp, and it may also be associated with multiple fault records that store the fault type, confidence value, and detection time. In addition, each panel may have multiple missions, where each mission has a status such as open or completed and a creation time. Each mission contains one or more mission images, and each image stores metadata such as content type, width, height, and upload time. These images are uploaded by users, where each user has attributes such as name, email, role, and account creation date. The uploaded mission images are then analysed to generate inspection results, where each result records the inspection status, such as pass or fail, together with the inspection time. Overall, the database structure supports integrated management of sites, panels, telemetry monitoring, fault recording, inspection missions, uploaded images, inspection results, and user interaction.

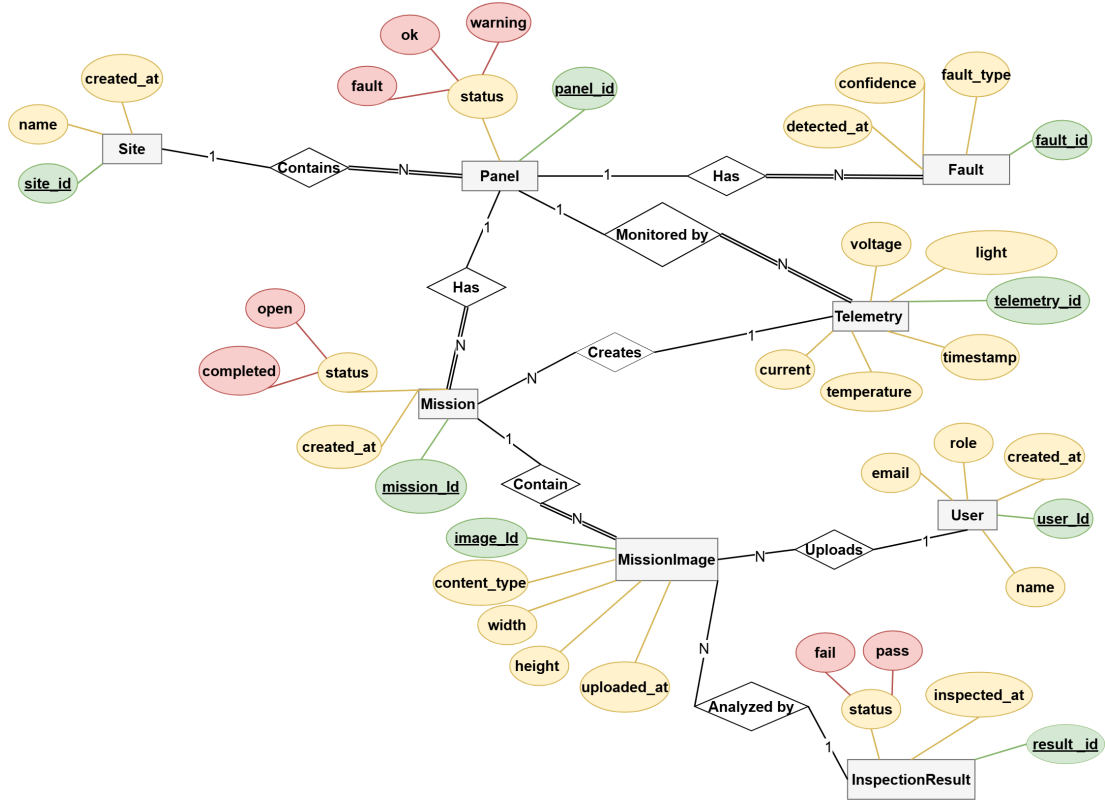


Figure 4.1: SolarSense Entity-Relationship Diagram

4.4.2 Database Schema

The ER diagram was converted into a relational database schema to organize how data is stored and connected within the system. The schema includes tables such as Site, Panel, Fault, Telemetry, Mission, MissionImage, InspectionResult, and

User. Each table uses a primary key to uniquely identify records, while foreign keys link related tables and maintain data integrity. This structure allows the system to store panel information, sensor readings, inspection images, and fault detection results in a single organized database that supports monitoring and decision-making.

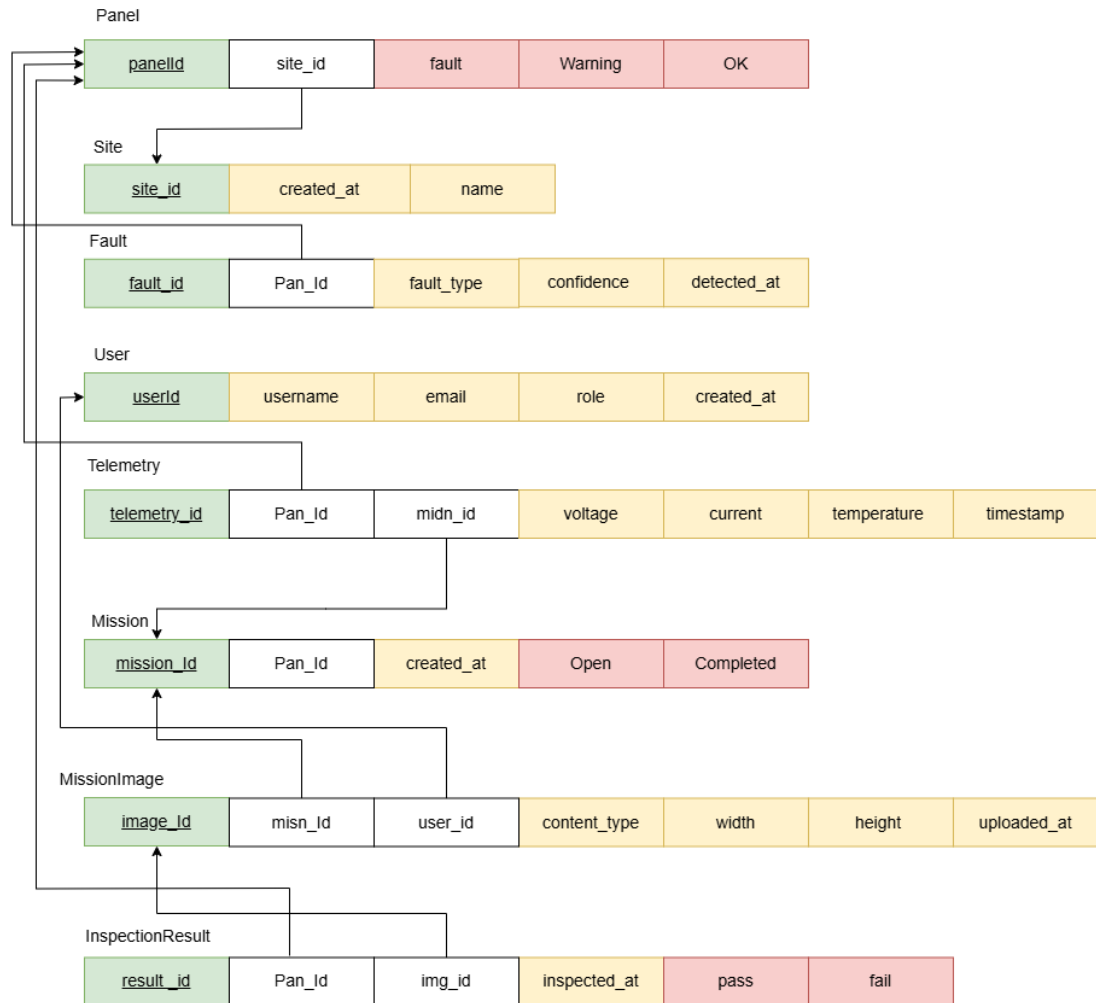


Figure 4.2: SolarSense Relational Database Schema

4.4.3 Class Diagram

The diagram illustrates the structural design of the proposed solar panel monitoring and inspection system. Each site contains zero or more panels, and each panel stores attributes such as its label, serial number, status, and deletion time. Every panel is linked to one telemetry entity through the monitoring relationship, where telemetry stores measurements and analytical attributes including voltage, current, temperature, light, timestamp, predicted power, prediction er-

ror, error percentage, anomaly severity, and analysis time, and also includes an operation for calculating actual power. Each panel may also have zero or more fault records, where each fault stores the fault type, confidence value, and detection time. In addition, each panel may be inspected through zero or more missions, where each mission stores its identifier, status, and creation time. Each mission may contain zero or more mission images and may also produce zero or more inspection results. Every mission image stores metadata such as storage path, storage key, content type, width, height, uploading user, and upload time, and each image is uploaded by exactly one user. The user entity stores attributes such as name, role, and account creation date, and also includes operations for authentication and profile updating. Finally, each mission image may be analysed into zero or more inspection results, where each result stores the related mission, panel, and mission image, together with the inspection status, defect type, confidence value, bounding-box data, notes, inspection time, and model version. Overall, the system structure supports the integrated management of sites, panels, telemetry monitoring, fault records, inspection missions, uploaded images, inspection analysis, and user interaction.

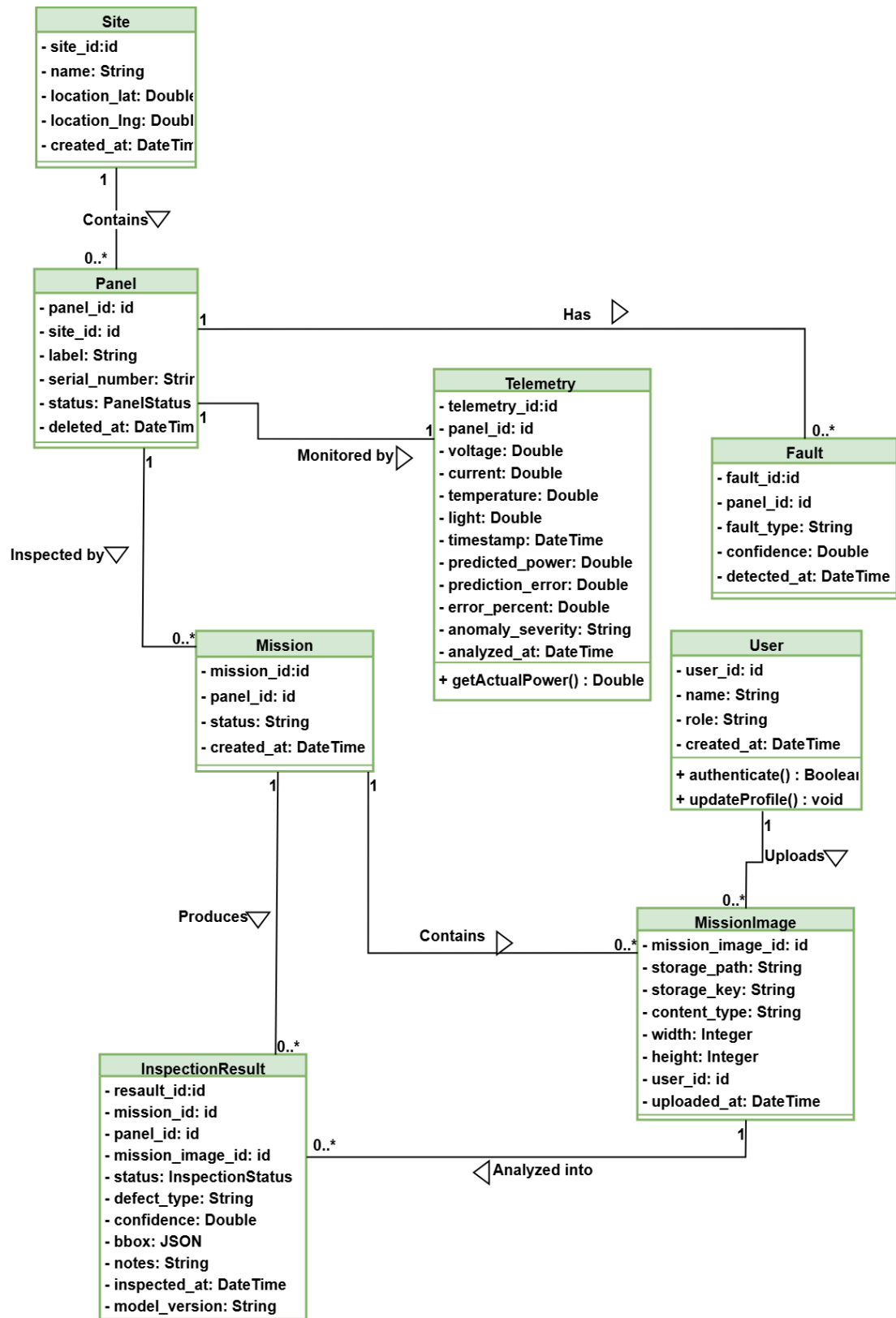


Figure 4.3: SolarSense Class Diagram

4.4.4 Use Case Diagram

The use case diagram for the SolarSense system illustrates the interactions between the Operator, Administrator, and the various external hardware and software components. Both actors are generalized as Users who can perform core functions such as Profile Edit, Dashboard View, and Site Registration.

A key architectural feature of the system is the delegation of security to an external Authentication system. The use cases for Register, Log in, and Log out are associated with this external actor to ensure secure identity management. The Registration process includes mandatory steps for entering user details and choosing roles, while the Log in process is extended by an optional Password Reset function.

The system's primary technical value lies in its automated data acquisition and analysis. Solar panel telemetry is collected by the external Module Sensor via the Sensor Record use case, which feeds into a mandatory Anomaly Detect process and is stored in the Data Store. Simultaneously, the Camera actor provides visual data through the Capture Panel Images use case. This process includes an Image Analyze function which is essential for identifying defects. Users can choose to dismiss specific analysis results through the Result Dismiss extension, or the system can automatically extend the Data Store process to create a formal maintenance record via the Inspection Create use case.

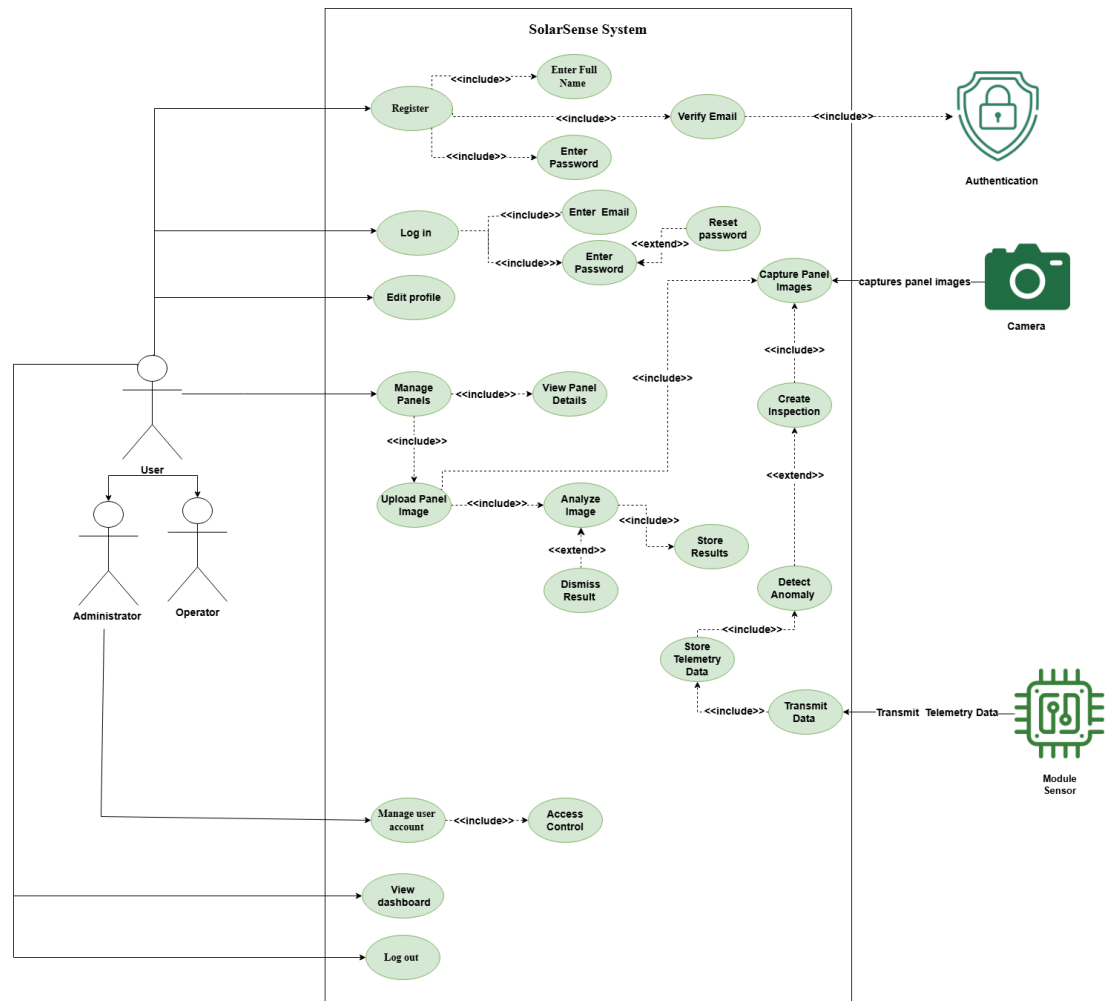


Figure 4.4: SolarSense System Use Case Diagram

Table 4.5: Register Use Case Description

Use Case Name	Register
Actors	Unregistered User
Triggering Event	User selects the "Sign Up" option from the website.
Scenario	A new user creates an account in the system by providing their personal details to gain access to the platform's features.
Related Use Cases	Log in.
Flow of Activities	System: 1. Display the registration form. User: 2. Enter personal details (e.g., name, email address, password). 3. Submit the registration form. System: 4. Validate the provided information. 5. Save the new user account in the database. 6. Display a successful registration message and redirect to the login page.
Special Requirements	- Email address must follow a valid format and be unique. - Password must meet the system's security policy (e.g., minimum length, complexity).
Pre-conditions	The user does not currently have an active account associated with their email address.
Post-conditions	- A new user profile is successfully created and stored in the database. - The user is able to log in to the system.
Exception Conditions	- Invalid email format. - Email address already registered. - Passwords do not match. - Missing required fields. - Database connection failure.

Table 4.6: Log in Use Case Description

Use Case Name	Log in
Actors	Registered User
Triggering Event	User selects the "Log in" option and attempts to access the system.
Scenario	A registered user authenticates themselves by providing their credentials to gain access to the system's secured features and personal dashboard.
Related Use Cases	Register, Profile Edit.
Flow of Activities	System: 1. Display the login form. User: 2. Enter credentials (email and password). 3. Submit the login form. System: 4. Validate the credentials against the records in the database. 5. Establish an active, secure user session. 6. Redirect the user to the main dashboard or home page.
Special Requirements	- Passwords must be securely verified without exposing plain text. - Session tokens should be generated securely.
Pre-conditions	The user already has a valid, registered account and is currently logged out.
Post-conditions	- The user is successfully authenticated. - An active user session is established. - The user gains access to role-based system features.
Exception Conditions	- Incorrect email or password. - Account is locked or suspended. - Database connection failure.

Table 4.7: Profile Edit Use Case Description

Use Case Name	Profile Edit
Actors	Authenticated User
Triggering Event	User selects the "Edit Profile" or "Account Settings" option from the menu.
Scenario	A logged-in user views and modifies their personal profile information such as their name or password to keep their account up-to-date and secured.
Related Use Cases	Log in.
Flow of Activities	System: 1. Retrieve the user's current profile data from the database. 2. Display the profile settings form with populated data. User: 3. Edit the desired fields (e.g., name). 4. Click the "Save Changes" button. System: 5. Validate the newly entered data. 6. Update the user profile in the database. 7. Display a success notification confirming the changes.
Special Requirements	- Any change to the email address may require secondary email verification.
Pre-conditions	The user must be authenticated and currently logged into their account.
Post-conditions	- The user's profile information is updated in the database. - The updated information is immediately reflected across the user's dashboard.
Exception Conditions	- Invalid data format (e.g., constraint validation in password field). - Missing required fields. - File size too large (if uploading a profile picture). - Database connection failure.

Table 4.8: Manage Panels Use Case Description

Use Case Name	Manage Panels
Actors	Administrator, Operator
Triggering Event	User selects the "Manage Panels" option for a registered site.
Scenario	The user manages solar panels in a selected site by adding, editing, or removing panels and updating their configurations.
Related Use Cases	Register Site, View Panel Details, Upload Panel Image.
Flow of Activities	System & User: 1. Display selected site details and list of panels. 2. User chooses to add, edit, or remove panels. 3. Display panel information form. 4. Validate entered panel data. 5. Save new or updated panel information in database. 6. Update panel list and recalculate site capacity.
Special Requirements	- Panel ID must be unique within a site. - Panel location coordinates must be valid. - System should maintain history of panel modifications.
Pre-conditions	User must be logged in and a site must already be registered.
Post-conditions	- Panel information is stored or updated in database. - Panel appears in the panel list. - Panel becomes available for monitoring and inspection.
Exception Conditions	- Invalid panel data. - Duplicate panel ID within site. - Database error during save operation. - Network connectivity failure.

Table 4.9: View Panel Details Use Case Description

Use Case Name	View Panel Details
Actors	Administrator, Operator
Triggering Event	User selects a panel from the panel list or dashboard.
Scenario	The user views detailed information about a solar panel including telemetry data, inspection results, and fault history.
Related Use Cases	Manage Panels, Upload Panel Image, Image Analyze, Sensor Record.
Flow of Activities	System & User: 1. Retrieve panel information from database. 2. Retrieve telemetry data and inspection results. 3. Retrieve panel images and fault history. 4. Display panel details including performance metrics and charts. 5. Allow user to navigate to related operations or export data.
Special Requirements	- Real-time telemetry display. - Visualization of performance trends. - Support exporting panel data to PDF or CSV. - Responsive design for different devices.
Pre-conditions	User must be logged in and the panel must exist in the system.
Post-conditions	- Panel information is displayed. - User can access related actions such as inspection or image upload.
Exception Conditions	- Panel not found in system. - No telemetry data available. - Image loading failure. - Database query timeout.

Table 4.10: Upload Panel Image Use Case Description

Use Case Name	Upload Panel Image
Actors	Operator, Administrator, Camera (External Hardware)
Triggering Event	Operator captures an image or the system requests visual confirmation of a sensor anomaly.
Scenario	The Operator uploads captured images of solar panels to the system, triggering an automated AI analysis to detect physical surface faults.
Related Use Cases	Capture Panel Images, Image Analyze, View Panel Details.
Flow of Activities	User: 1. Navigate to the specific panel detail page. 2. Select the upload image option. 3. Include: Capture panel image via external camera hardware. 4. Confirm upload to the central server. System: 5. Include: Invoke Image Analyze use case to process the file. 6. Update the panel status based on the analysis results.
Special Requirements	- Supports high-resolution RGB formats. - Automated metadata tagging for timestamp and Panel ID.
Pre-conditions	Panel must exist in the database; User must be authenticated.
Post-conditions	Analysis result is stored and data is visible on the dashboard.
Exception Conditions	- Image file format not supported. - Network failure during file transmission.

Table 4.11: Access Control Use Case Description

Use Case Name	Access Control
Actors	Administrator, External Authentication System
Triggering Event	Administrator accesses the user management or site security module.
Scenario	The Administrator manages permissions and roles to ensure system security and data integrity.
Related Use Cases	Manage User Account, Log in, Site Registration.
Flow of Activities	System: 1. Verify Administrator credentials via the External Authentication System. 2. Provide access to user management and security tools. Administrator: 3. Include: Perform Manage User Account tasks such as updating roles or deactivating users. 4. Extend: Initiate Site Registration if a new farm site needs deployment. System: 5. Update permissions and log security changes.
Special Requirements	- Multi-factor authentication support. - Encrypted storage for access tokens.
Pre-conditions	User is logged in with high-level Administrator privileges.
Post-conditions	User roles and site permissions are updated successfully.
Exception Conditions	- External Authentication system is unreachable. - Session timeout during administrative tasks.

Table 4.12: Dashboard View Use Case Description

Use Case Name	Dashboard View
Actors	Administrator, Operator
Triggering Event	User selects the main monitoring tab or successfully logs into the portal.
Scenario	Displays a high-level summary of system performance, active alerts, and historical trends for the monitored solar panels.
Related Use Cases	Trend View, Log in, View Panel Details.
Flow of Activities	System: 1. Retrieve current telemetry from the database. 2. Include: Generate Trend View to visualize historical performance data. 3. Display active anomaly notifications and site status. User: 4. Review system health metrics and click on specific panels to View Panel Details.
Special Requirements	- Real-time data refresh capability. - Mobile-responsive layout for field operations.
Pre-conditions	Telemetry data must be present in the Data Store; User must be authenticated.
Post-conditions	User is informed of the overall health and yield of the solar farm.
Exception Conditions	- Database query error. - No sensor data available for the selected period.

Table 4.13: Logout Use Case Description

Use Case Name	Logout
Actors	Authenticated User
Triggering Event	User selects the "Logout" option from the system menu or profile dropdown.
Scenario	The user securely ends their session, ensuring that no unauthorized access to their account can occur from the current device.
Related Use Cases	Log in, Session Timeout
Flow of Activities	System & User: 1. User clicks the "Logout" button or link. 2. System invalidates the user's session or authentication token. 3. System clears any session-related data from the client (e.g., cookies, local storage). 4. System redirects the user to the login page or public landing page. 5. System displays a confirmation message indicating successful logout.
Special Requirements	- All session tokens must be securely invalidated. - Sensitive data must be cleared from the client.
Pre-conditions	The user must be authenticated and currently logged in.
Post-conditions	- The user is logged out of the system. - No further access to authenticated features is possible without logging in again.
Exception Conditions	- Session already expired or invalid. - Network or server error during logout process. - Failure to clear session data from client.

4.4.5 Flow Chart

Figure 4.5 illustrates the operational flow of the proposed system. The process begins when the embedded chip installed on the solar panel continuously monitors the panel's electrical and operational behaviour. The collected measurements are then processed by the machine learning anomaly detection module to determine whether abnormal behaviour is present. If no anomaly is identified, the normal output is sent directly to the central system. In contrast, if an anomaly is detected, an inspection process is automatically created, and panel images are uploaded for further inspection. After that, both the inspection data and the sensor-based output are forwarded to the central system, where the different sources of information are combined and analysed. Finally, the integrated results are displayed on the dashboard, enabling users to monitor the panel condition and review the system output in a clear and centralised manner.

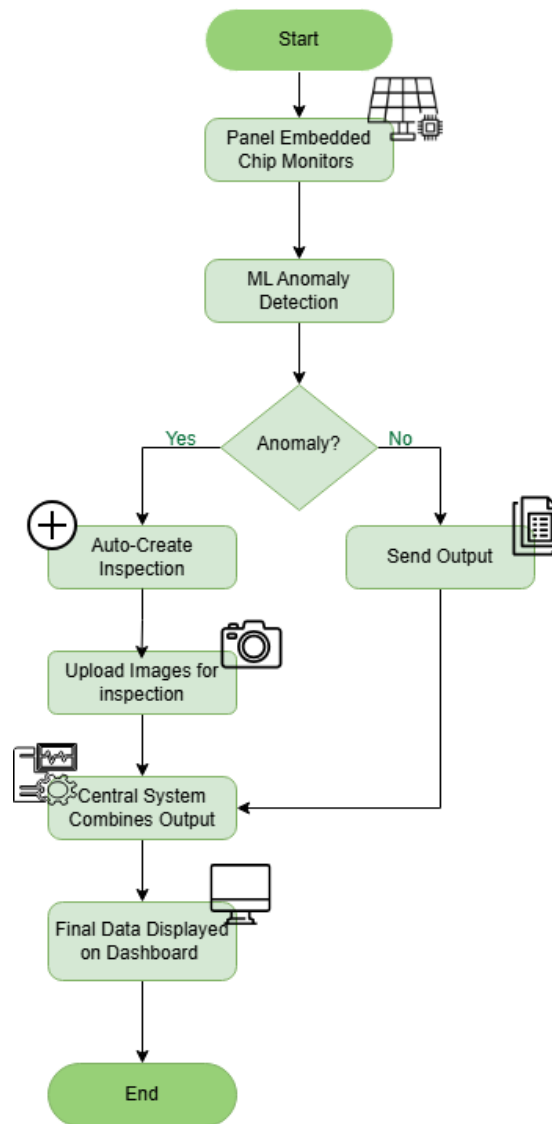


Figure 4.5: SolarSense Flowchart Diagram

4.4.6 Sequence Diagrams

The sequence diagram illustrates how the main components of SolarSense interact during the real-time monitoring process. The embedded chip performs periodic sensor readings in a continuous loop, where voltage, current, temperature, irradiance, and power values are recorded. These readings, together with any anomaly alert, are then sent to the GUI, which stores the panel data in the database. When the operator logs in and opens the dashboard, the GUI requests the latest panel data from the database, and the database returns the relevant key performance indicators and panel status information. If an anomaly is detected, the system displays an alert to the operator, who may then

create a mission for further inspection while continuing to observe the real-time monitoring view. Overall, this sequence diagram clarifies the communication flow between the operator, GUI, database, and embedded chip during continuous monitoring and anomaly handling within the SolarSense system.

Figure 4.6 illustrates how the operator, GUI, database, and embedded chip interact during panel monitoring in the SolarSense system. The embedded chip performs periodic sensor readings in a continuous loop, where voltage, current, temperature, irradiance, and power values are recorded and then sent together with any anomaly alert to the GUI. The GUI stores the received panel data in the database and, when the operator logs in and opens the dashboard, requests the latest panel data from the database in order to retrieve the relevant KPIs and panel status information. If an anomaly is detected, the system displays an alert to the operator and allows a mission to be created for further inspection, while the real-time monitoring view continues to be shown. Therefore, the diagram summarises the real-time flow of monitoring data, anomaly handling, and operator interaction within SolarSense.

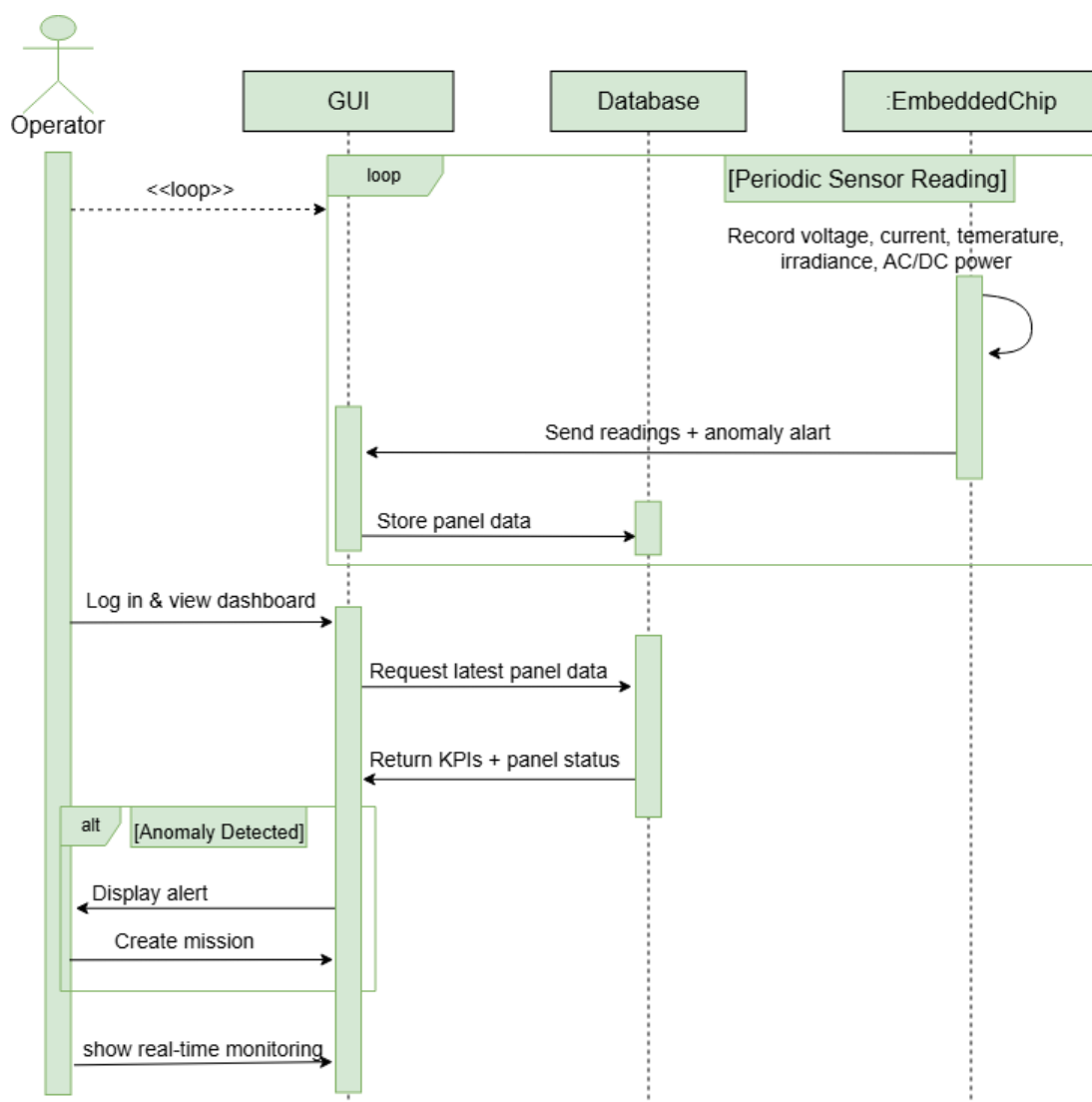


Figure 4.6: Sequence Diagram for The Embedded Chip

Figure 4.7 sequence diagram shows how camera inspections are carried out In SolarSense, the operator uploads RGB images of solar panels through the interface using a regular camera. The interface sends the image data to the dashboard, which forwards it to the CV model for analysis. The CV model processes the images and returns the detected defects, including the defect category, confidence scores, and bounding boxes. The dashboard then stores the inspection results in the database and sends the analysis results back to the interface, where the inspection findings are displayed to the operator.

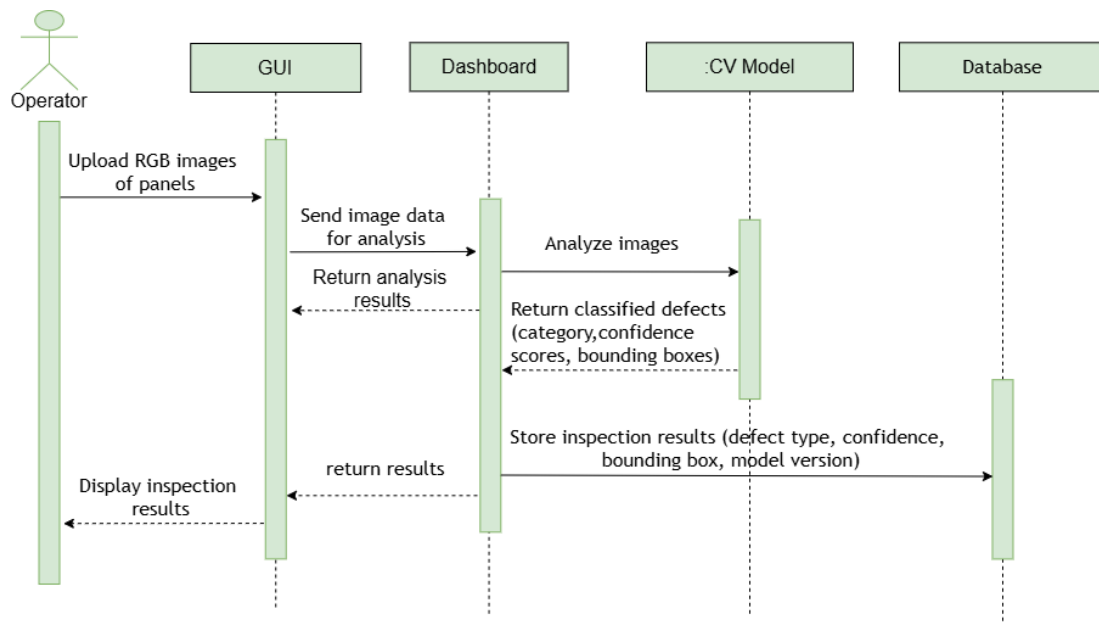


Figure 4.7: Sequence Diagram for The Camera

4.4.7 State Diagram

The state diagrams illustrate how SolarSense components transition between operational states based on internal events and system triggers. Each diagram focuses on one subsystem and shows how it responds to sensor readings, operator actions, or mission progress. The first diagram represents the embedded chip workflow, while the second diagram demonstrates the drone mission lifecycle. Together, they provide a clear view of how each component behaves, reacts to events, and moves between states to support autonomous monitoring and inspection.

The state diagram in Figure chip stat diagram summarises the behaviour of the SolarSense panel-level embedded module during continuous monitoring. After configuration with panel components, the module repeatedly reads sensors, runs the ML model, and either sends normal data when no anomaly is found or transmits an alert and anomaly data when an anomaly is detected. It then waits for the next sampling interval before returning to the read-sensors state and repeating the cycle.

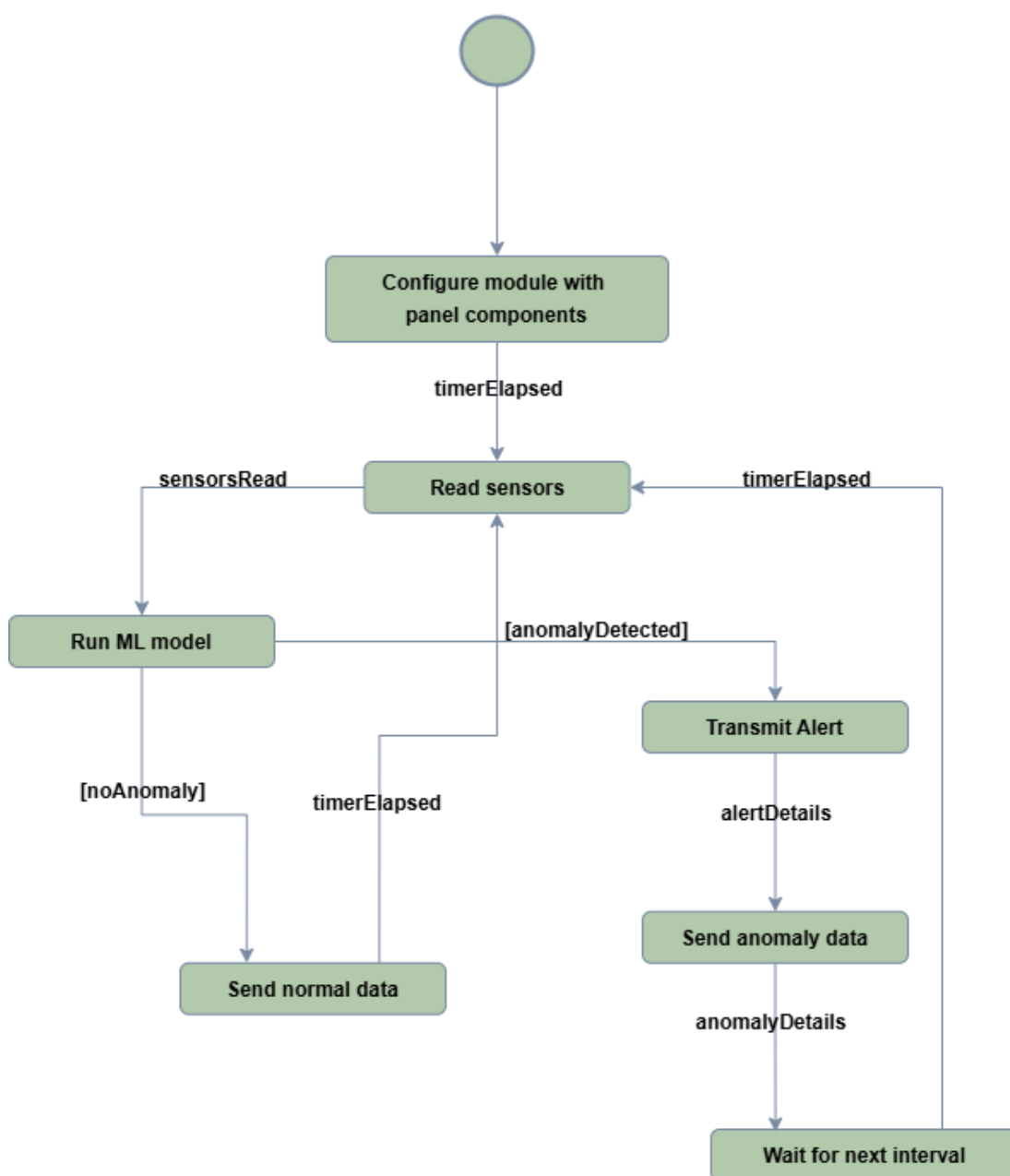


Figure 4.8: Chip State Diagram

4.4.8 Computer Vision Analysis State Diagram

The state diagram in Figure 4.9 illustrates the sequential behaviour of the SolarSense inspection subsystem during a manual or triggered visual check. This process transitions through several operational states to ensure that high-resolution visual evidence is captured and correctly processed by the AI

backend:

- Idle: The initial state where the system or operator waits to initiate the inspection process.
- Capturing: Triggered by the operator pressing capture, the camera hardware acquires the RGB image of the target solar panel.
- Uploading: Once the image is taken, the module transitions to transmitting the data to the centralized server.
- Analyzing: The system invokes the YOLOv8 computer vision model to identify defects such as cracks, dust, or shading.
- Result: The analysis is completed, and the specific fault classification is generated.
- Reporting: The system formats the diagnostic findings into a visual report for the dashboard.
- Stored: The final state where all data is saved to the database, after which the system cycles back to Idle for the next panel.

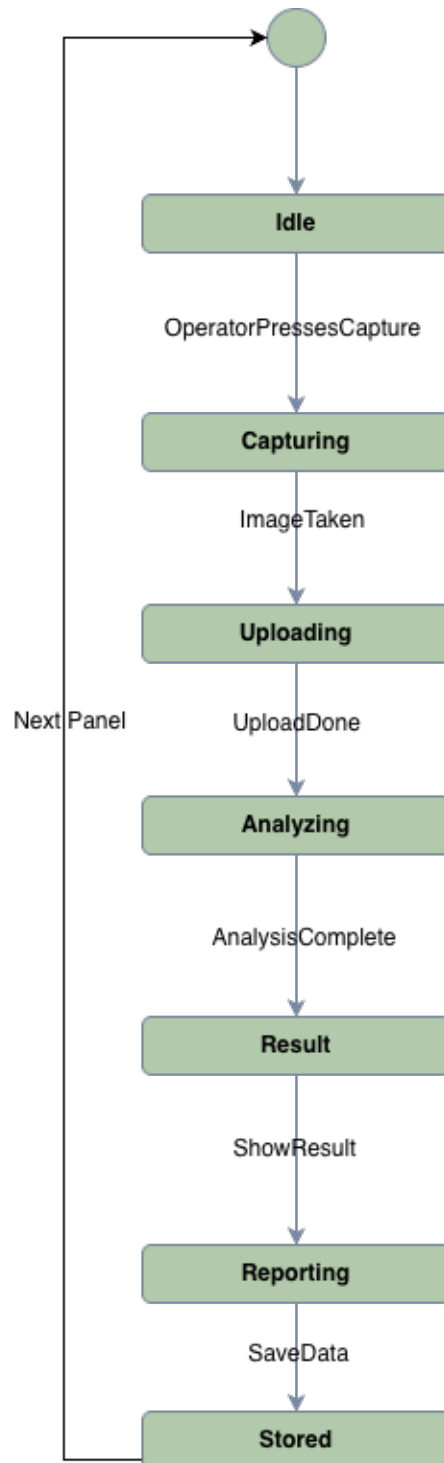


Figure 4.9: Computer Vision Analysis State Diagram

The Chip Data-flow Diagram shows how sensor information is collected, processed, and delivered to the monitoring dashboard. Solar panel sensors generate Sensor Data, stored in D1 before entering Process 1.1, where it is filtered into a Data Packet. This packet is saved in D2 and then analyzed in Process 1.2. Normal results create a Normal Entry in D3, while abnormal results produce

an Anomaly Package that moves to Process 1.3 for alert creation. The resulting Alert Packet is sent through Process 1.4, generating an Alert Message for the dashboard. The dashboard can also access Record History from D3 for reviewing past alerts.

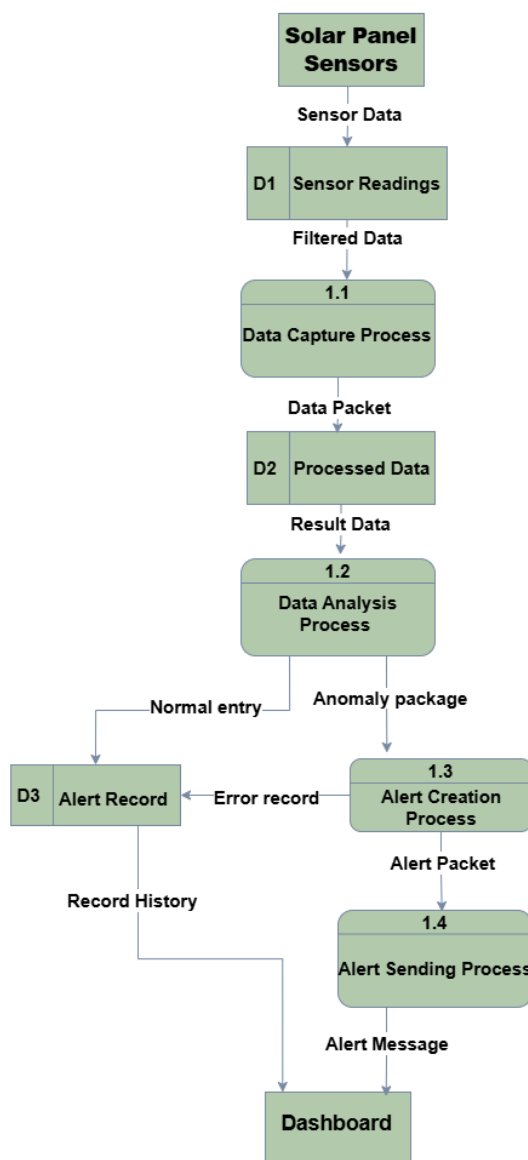


Figure 4.10: Chip Data-flow Diagram

4.4.8.1 Computer Vision Data-flow Diagram

The Computer Vision Data-flow Diagram (Figure 4.11) illustrates the progression of visual data as it moves from the physical solar site to the monitoring interface. The workflow begins when the Operator initiates a capture, triggering Process 1.1 (Capture Image). The resulting image file is stored in the Image Database (D1).

Once the upload is verified through Process 1.2 (Upload Image), the system triggers Process 1.3 (Computer Vision Analysis), which accesses the raw image from D1 to perform fault detection. The outcomes of this analysis, such as defect type and confidence scores, are written to the Analysis Results (D2) data store. These results are then retrieved by Process 1.4 (Generate Dashboard Report) to create a visual summary for the user. Finally, the processed information is committed to long-term storage via Process 1.5 (Store Data), ensuring a complete historical record is available before the cycle returns to the Ready Panel state.

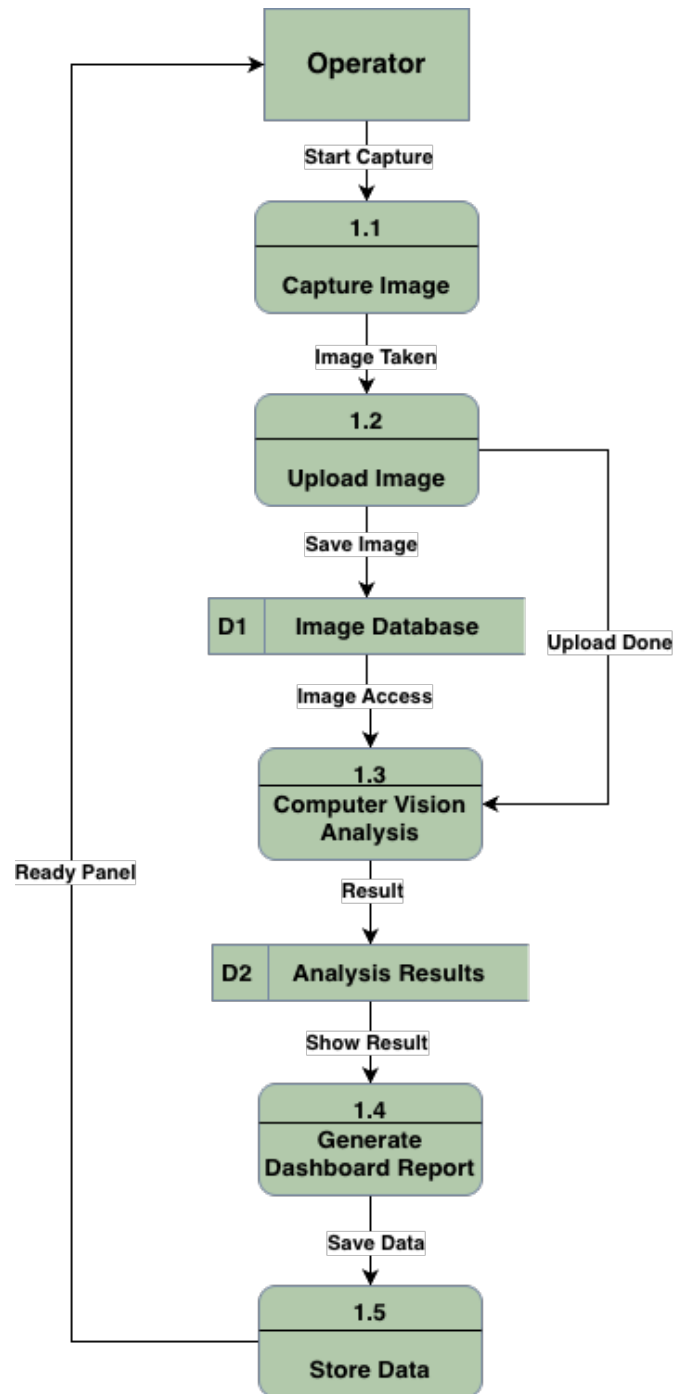


Figure 4.11: Computer Vision Data-flow Diagram

4.5 Prototype Design

This section views the software and hardware prototypes for SolarSense system.

4.5.1 Software Design

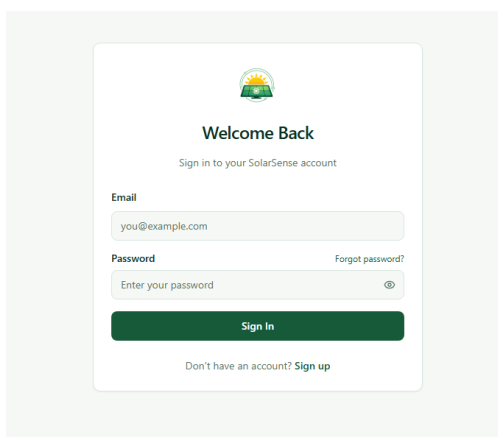
The overall software design has been derived directly from the presented user interfaces, so that service boundaries and data flows reflect real operator tasks.

A layered, modular architecture is adopted. The presentation layer provides the authenticated web experience shown in the Sign In/Sign Up, Dashboard, Detailed Panel, Alerts, and capturing Missions views. The application layer orchestrates domain services that correspond to these views: Monitoring (real-time KPIs, time-series charts, and panel grid), Panel Analytics (per-panel signals, CV/ML comparisons, residuals, anomaly scores, and confidence), Alerting (event ingestion, triage, updates, assignment, and notes), Mission Control (calendar scheduling and status tracking), and Identity/Access (profile and role management). The data and integration layer maintains telemetry, images, findings, and workflow records, while streaming endpoints are used to push live updates to the dashboard and detail pages. Navigation and component structure in the interfaces have been used to define API contracts and entity relationships. Dashboard cards and charts map to read optimized endpoints; the panel page requires correlated queries over electrical, environmental, and model outputs; the alert pop-up relies on idempotent update APIs to ensure safe workflow changes; and the missions calendar uses paged and date-scoped queries. Cross-cutting concerns are addressed consistently: role-based access governs admin actions, audit fields capture all state transitions, and consistent status and severity vocabularies are enforced across alerts, panels, and missions. This design ensures that the implemented system remains traceable to user needs, scalable for real-time operation, and maintainable as additional models or devices are incorporated.

4.5.1.1 SolarSense User Interface

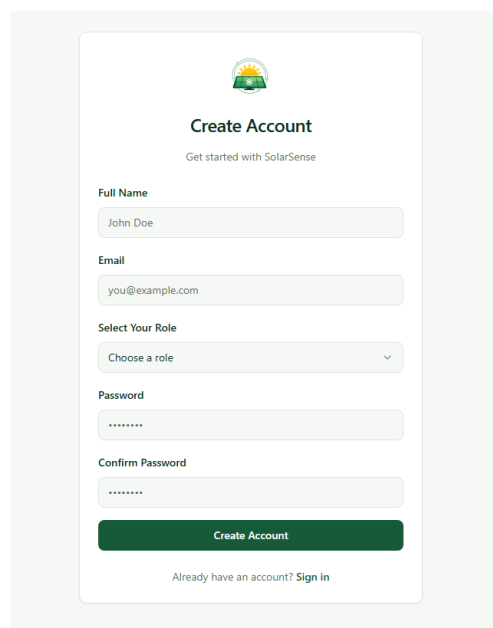
The SolarSense user interface, as shown in the provided screens, was designed to support the main user-account operations through a simple and consistent visual style. The sign-in page provides a clear authentication form with fields for email and password, together with options for password recovery and account registration, while the sign-up page follows the same card-based layout and allows new users to create an account by entering their full name, email address, selected role, password, and password confirmation. In addition, the reset password page supports account recovery through a single email input field that allows the user to request a password reset link, together with an option to return to the login page. The profile page was designed to allow users to view and manage

their personal account information, including full name, email address, role, and membership date, and it also provides account-management functions such as profile editing and password reset under a dedicated security section. Overall, these interfaces reflect a consistent and user-friendly design approach that supports account access, registration, recovery, and profile management within the SolarSense system.



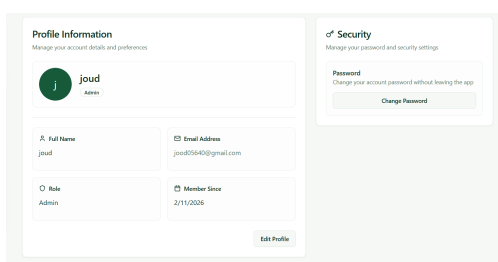
The Sign In page features a central white card on a light gray background. At the top of the card is the SolarSense logo (a green sun with a gear inside). Below the logo, the text "Welcome Back" is displayed in bold, followed by "Sign in to your SolarSense account" in a smaller font. The form includes an "Email" field with the placeholder "you@example.com", a "Password" field with the placeholder "Enter your password", and a "Forgot password?" link. A green "Sign In" button is positioned below the password field. At the bottom of the card, there is a link: "Don't have an account? Sign up".

Figure 4.12: Sign In Page



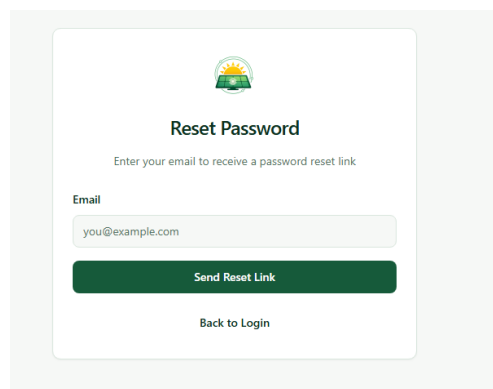
The Sign Up page features a central white card on a light gray background. At the top of the card is the SolarSense logo. Below the logo, the text "Create Account" is displayed in bold, followed by "Get started with SolarSense" in a smaller font. The form includes a "Full Name" field with the placeholder "John Doe", an "Email" field with the placeholder "you@example.com", a "Select Your Role" dropdown menu with the placeholder "Choose a role", a "Password" field with the placeholder "*****", and a "Confirm Password" field with the placeholder "*****". A green "Create Account" button is positioned below the confirm password field. At the bottom of the card, there is a link: "Already have an account? Sign in".

Figure 4.13: Sign Up Page



The Profile page is divided into two main sections: "Profile Information" and "Security". The "Profile Information" section on the left includes a user profile card for "joud" (Admin) with a green circular avatar. Below this, there are fields for "Full Name" (joud), "Email Address" (joud0540@gmail.com), "Role" (Admin), and "Member Since" (2/11/2026). An "Edit Profile" button is at the bottom right of this section. The "Security" section on the right includes a "Password" field with the placeholder "Change your account password without leaving the app" and a "Change Password" button.

Figure 4.14: Profile Page



The Reset Password page features a central white card on a light gray background. At the top of the card is the SolarSense logo. Below the logo, the text "Reset Password" is displayed in bold, followed by "Enter your email to receive a password reset link" in a smaller font. The form includes an "Email" field with the placeholder "you@example.com". A green "Send Reset Link" button is positioned below the email field. At the bottom of the card, there is a link: "Back to Login".

Figure 4.15: Reset Password Page

4.5.1.2 Dashboard Page

The SolarSense overview page was designed to provide Operators with a consolidated summary of the current operational state of the monitored solar panels. As shown in the interface, the page presents key status indicators at the top,

including the numbers of active, warning, fault, and total panels, followed by a power output chart that visualises the latest available telemetry history. In addition, a recent anomalies section highlights detected issues together with panel identifiers, site names, severity labels, confidence information, and timestamps, while an open inspections section allows users to track pending inspection tasks and access them directly. The lower part of the page includes a panel health overview, where panels are displayed visually by site and colour-coded according to their status, with filtering options for site and panel condition. Overall, this interface was designed to support rapid system awareness by combining monitoring data, anomaly alerts, inspection activity, and panel health status within a single dashboard view.

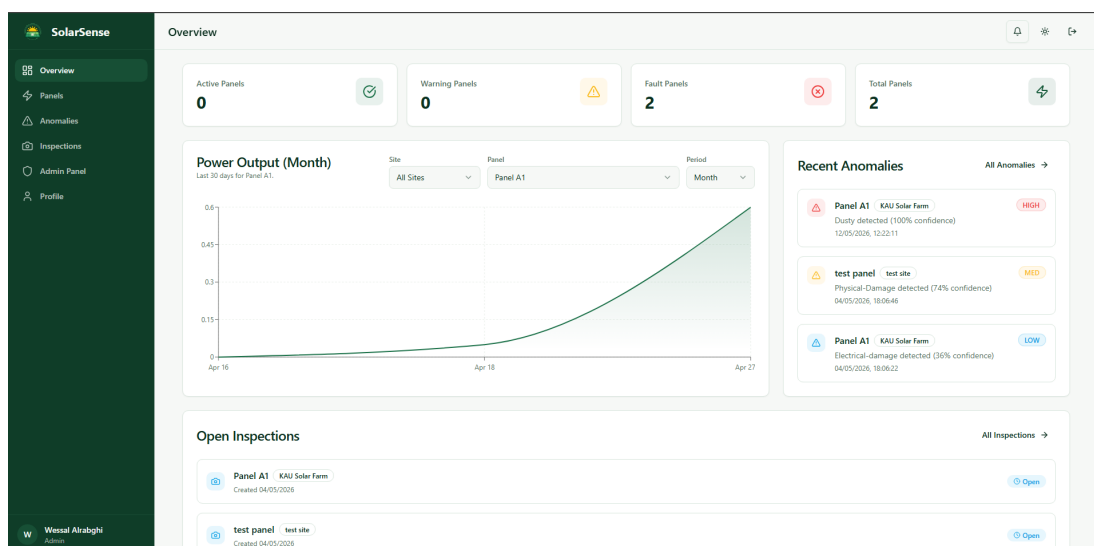


Figure 4.16: Dashboard Part 1

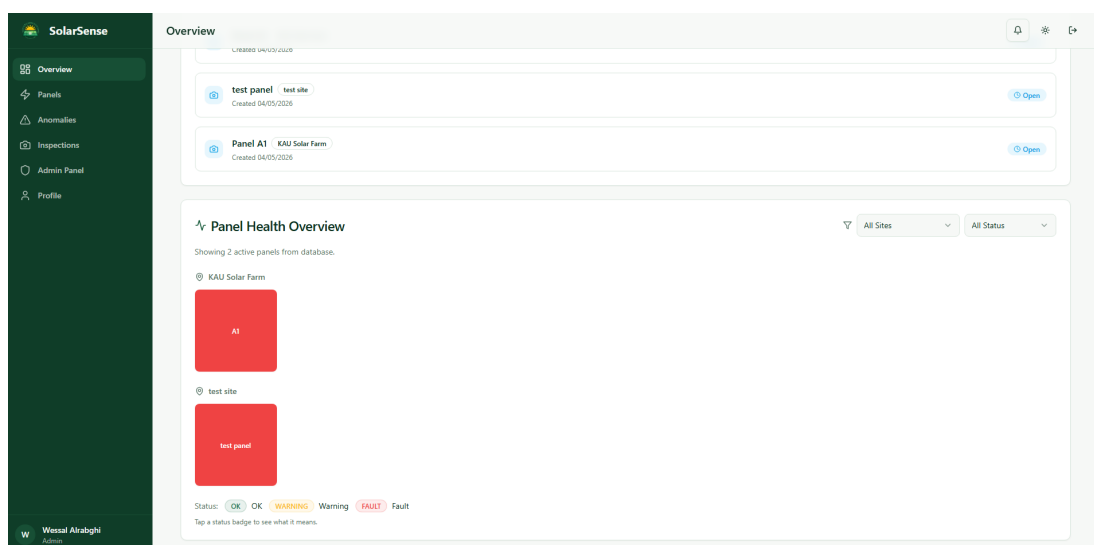


Figure 4.17: Dashboard Part 2

4.5.1.3 Panels Details Interface

The SolarSense panels page was designed to provide a structured view of all registered solar panels within the system. As shown in the interface, the page displays the panels in a tabular format with key attributes including panel label, serial number, site name, and current status, while also providing filtering and search options to help users locate specific panels efficiently. Each panel entry includes a details action that opens a dedicated information window containing more complete panel data, such as the panel identifier, site identifier, status, label, serial number, and associated site name. In addition, the details view presents summary information related to the selected panel, including the number of telemetry records, fault records, and missions, as well as the latest telemetry readings and the most recent mission information. Overall, this interface was designed to support efficient panel management by combining quick panel listing, status visibility, filtering capabilities, and access to detailed operational information within a single workflow.

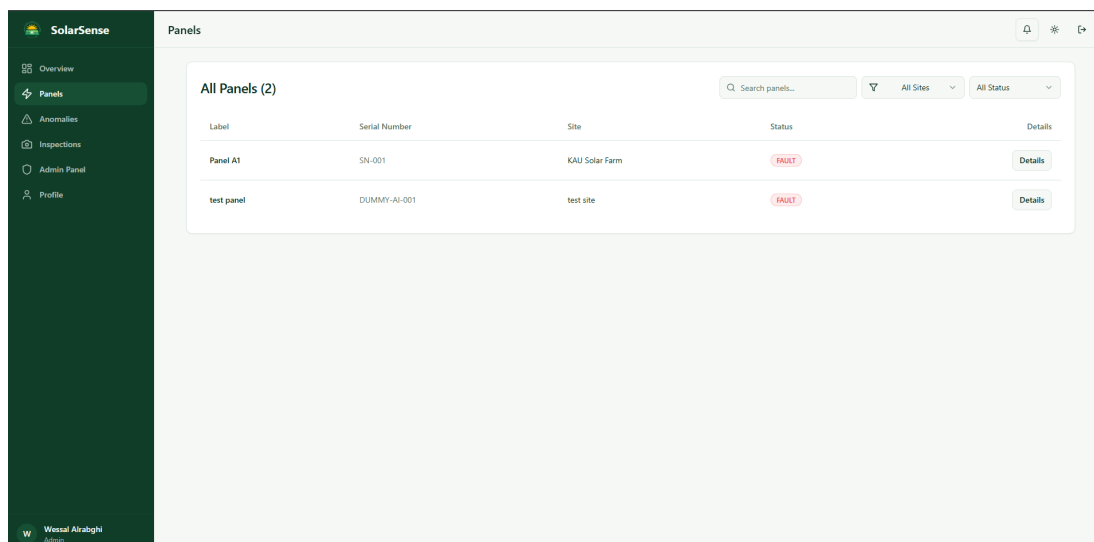


Figure 4.18: Panels Details

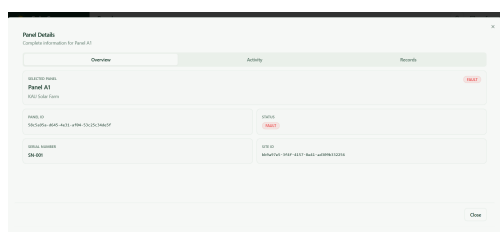


Figure 4.19: Panel Details Overview

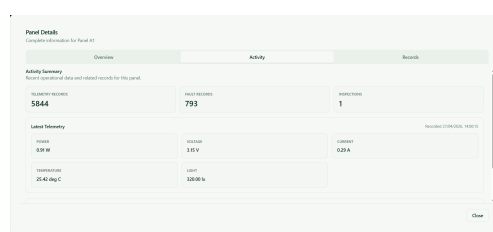


Figure 4.20: Panel Details Activity

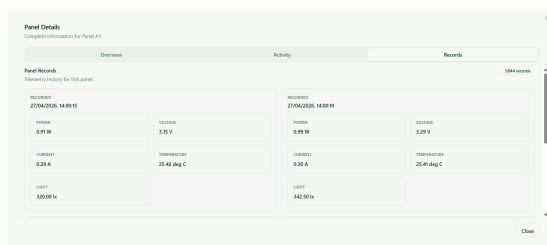


Figure 4.21: Panel Details records

4.5.1.4 Missions Interface

The SolarSense inspections page was designed to support the management and review of inspection missions in a structured and operationally clear manner. As shown in the interface, the page presents all inspections in a tabular format that includes the inspection identifier, related panel, site, status, and creation date, together with filtering options and a button for creating a new inspection. Each inspection entry also provides action controls for viewing details and managing the inspection workflow. When an inspection is opened, a detailed view is presented

showing the inspection status, associated site, creation time, and the uploaded inspection images. In addition, the interface includes a dedicated AI analysis section in which detected defects are summarised and displayed with their fault type, confidence value, detection time, and recommended action. The detailed results also include lower-level computer vision outputs, such as raw detections, model name, bounding-box coordinates, confidence scores, and review actions such as dismissal or re-analysis. Overall, these interfaces were designed to support the full inspection lifecycle by combining inspection tracking, image review, AI-based defect analysis, and result validation within a single integrated workflow.

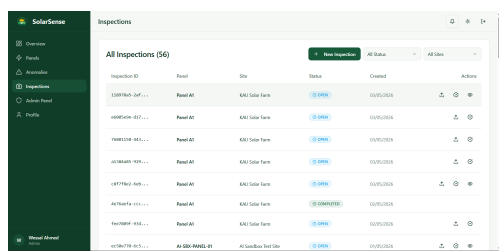


Figure 4.22: Inspections Interface

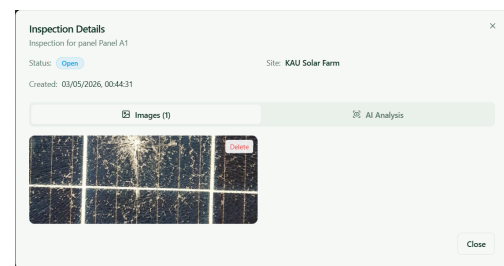


Figure 4.23: Panel Images

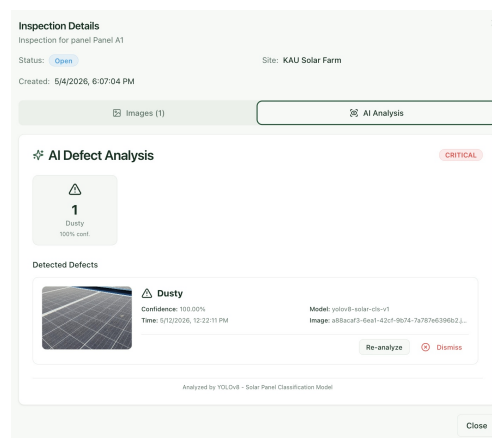


Figure 4.24: CV Results

4.5.1.5 Admin Panel Page

The Admin Panel page was implemented to provide administrator users with a central interface for managing user roles and permissions in SolarSense. The page displays summary cards showing the total number of users, administrators, and operators, allowing the current role distribution to be reviewed quickly. A user management table is also provided, where each registered user is listed with their name, email address, current role, membership date, and role-changing

option. Role labels are visually distinguished to separate administrator and operator accounts, while the drop-down menu allows an administrator to update user permissions directly from the interface. This page supports system security and maintainability by enabling role-based access control and ensuring that administrative responsibilities can be managed without direct database modification.

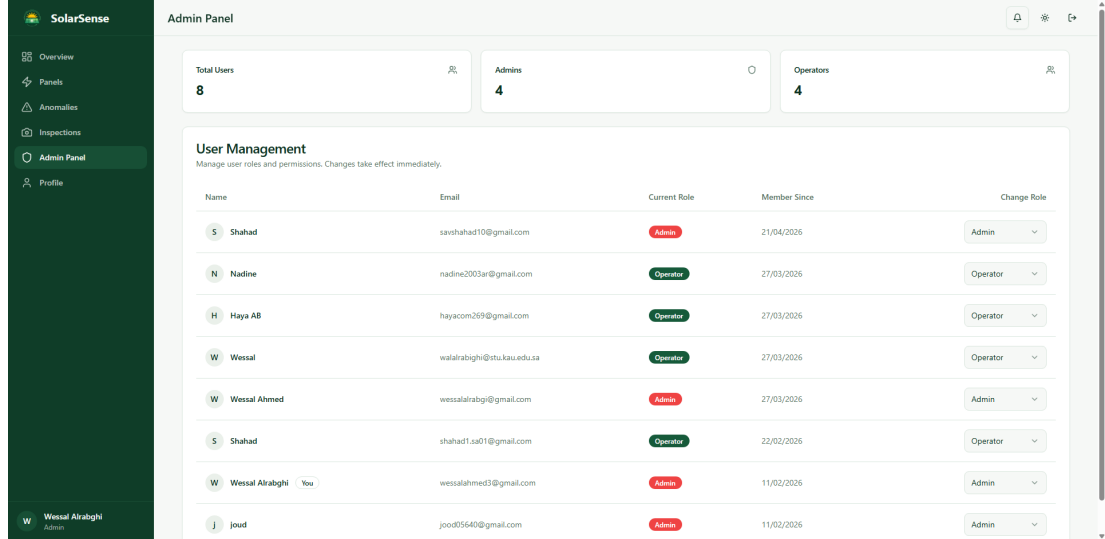


Figure 4.25: Admin Page

4.5.2 Chip Design

The real hardware design of the SolarSense monitoring chip was implemented using an ESP32 development board as the main controller, which reads the panel data, processes the measurements, calculates power, and displays the results. Based on the provided component list and prototype image, the real design includes a 0-25 V Voltage Sensor Module for voltage measurement, an ACS712 20A Range Current Sensor Module for current measurement, a BH1750 Digital Light Intensity Sensor Module for light measurement, and a TMP36GT9Z TMP36GZ temperature sensor for temperature measurement. In addition, a 16×2 I2C LCD was used to display the values in real time, while the power side of the prototype included a solar panel, an 18650 lithium battery, and related connection circuitry. The collected parameters are voltage, current, light intensity, and temperature, and the power value is calculated from the voltage and current readings before the data are transferred to the database for later analysis by the machine-learning model for fault classification.

Not all real components were included in the simulation design because the simulation was intended to validate the embedded logic and sensing workflow, rather than reproduce the full physical power system. For this reason, the first potentiometer was used instead of the real voltage sensor module, and the second potentiometer was used instead of the real ACS712 current sensor, since both provide adjustable analogue inputs that are suitable for testing the ESP32 reading and calculation logic. Likewise, simplified supported components were used to represent the sensing behaviour of the real system, while some physical modules from the prototype, such as the solar panel, battery-related power arrangement, and other practical hardware details, were not fully represented because such elements are either not directly supported in the simulator or are not necessary for validating sensor reading, data processing, and LCD output behaviour. Therefore, the simulation focused on the functional behaviour of the monitoring chip, whereas the real prototype included the complete hardware elements required for actual deployment.[Wokwi, 2026].

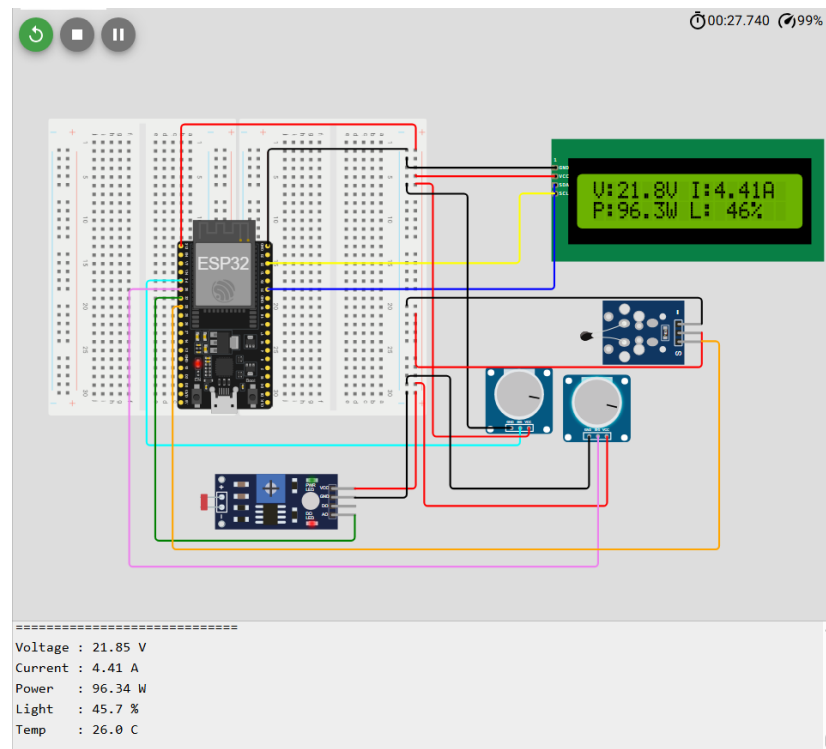


Figure 4.26: SolarSense Chip Design

4.6 Near Future Work

By analyzing the collected requirements through surveys, stakeholder interviews, and literature review, the system's core functionalities were defined. Furthermore, the necessary data were gathered to support model development. The subsequent chapters will discuss the implementation process, model construction, and training procedures, followed by the presentation of testing results and the remaining components of the system as outlined in the work plan.

4.7 Conclusion

This chapter focused on the software development process of the proposed SolarSense system. It outlined the major stages of system design, including the identification of end users, specification of system requirements, and definition of core functionalities. The underlying database structure and user interface were also designed to support efficient operation and user interaction. Additionally, UML diagrams were utilized to illustrate the essential system components and their relationships. Finally, the chapter presented the project's prospective enhancements and future work. The content of this chapter serves as a foundation and reference for implementing the proposed SolarSense system.

CHAPTER 5

IMPLEMENTATION

5.1 Introduction

Implementation is the stage in which the designed SolarSense architecture is transformed into a functional prototype. In this project, the implementation process involved developing and connecting several components, including the AI models, front-end dashboard, back-end services, database, and -based sensing unit. These components were implemented to support the main SolarSense workflow, where telemetry data is collected from sensors, inspection images are processed, analysis results are stored, and monitoring information is presented to the user through the dashboard.

In SolarSense, the implementation stage was important because the system depends on the integration of software, AI, and hardware components. The CV model was implemented to support image-based solar panel condition analysis, while the ML model was implemented to process telemetry data and support performance-related analysis. The front-end was developed to provide user access to monitoring pages, panel details, inspection results, and dashboard visualisations. The back-end and database were implemented to manage data storage, API requests, user-related operations, telemetry records, and communication between the system components. The hardware part was also implemented using an -based chip connected to sensors for collecting real-time readings from the solar panel environment.

This chapter presents the implementation details of the SolarSense prototype. Section 5.2 provides an overview of the implemented system. Section 5.3 describes the development activities and actions. Section 5.4 presents the tools

and technologies used during implementation. Section 5.5 explains the implementation of the AI model components, including the CV and ML components. Section 5.6 describes the front-end implementation, while Section 5.7 presents the back-end and database implementation. Section 5.8 explains the chip implementation, and Section 5.9 concludes the chapter.

5.2 System Overview

The proposed system, SolarSense, is designed to monitor and inspect solar panels by combining image analysis and sensor data. The system dashboard provides several pages, including Overview, which displays a general summary of system status, alerts, and panel health. The Panels page allows the operator to view information about each solar panel and monitor its operational condition. During inspections, the operator uploads RGB images of solar panels using a regular camera through the Inspections page. These images are sent to the dashboard, which forwards them to the CV model to detect possible panel defects such as cracks, dirt, or shading. At the same time, telemetry data collected from the sensor module (voltage, current, and temperature) is processed by the ML model to monitor panel performance and detect anomalies, which are displayed in the Anomalies page. The analysis results from both the CV and ML models are then stored in the database along with inspection details and panel information. Finally, the system displays the results to the operator through the dashboard interface, while administrative controls such as system management are handled through the Admin Panel, and user information can be accessed from the Profile page.

5.3 Activities and Actions of the Development

The SolarSense application consists of four layers. The development process of the four layers of the system is as follow:

1. Front-end layer: Everything that the end user sees and requires to engage with the system is presented in this layer, including the dashboard for uploading missions, reviewing results, and managing inspections.
2. Back-end layer: There are two primary components to this layer:

- **API:** Handles requests for data processing, storage, and retrieval.
 - **Database:** All of the system's data that are preserved and accessible is contained in the database, including missions, faults, telemetry, and user profiles.
3. **ML/CV layer:** The system's most fundamental layer is this one. Computer vision is the first element for image defect detection, and the second element is machine learning for telemetry anomaly analysis.
 4. **Sensor chip layer:** This layer is responsible for collecting real-time telemetry data from the solar panels using a sensor module connected to a microcontroller chip. The chip measures parameters such as voltage, current, and temperature and sends the collected data to the backend system for monitoring and analysis.

The sections that follow will provide further details regarding the evolution of each layer.

5.4 Tools and Technologies

The development and implementation of the **SolarSense** system utilized a variety of specialized tools and technologies across computer vision, machine learning, embedded systems, and full-stack development:

1. Programming Languages:

- **Python:** The primary language for the SolarSense software infrastructure. It is widely used in artificial intelligence, computer vision, and backend development. In this project, Python was used to develop the YOLOv8 detection pipeline, the LSTM prediction models, and the FastAPI backend service [Python Software Foundation, nd, Google, nd, FastAPI, nd, Ultralytics, nd].
- **C++:** The high-performance language used to program the Arduino Uno microcontroller in the hardware prototype. It handles low-level hardware interactions, such as reading voltage, current, temperature,

and light sensor values, and forwarding telemetry data to the software system [Arduino, nd].

- **JavaScript/TypeScript:** Used for the development of the frontend web application. It supports the dynamic rendering of the dashboard, authentication pages, mission management features, telemetry visualization, and communication with backend APIs and Supabase services [MDN Web Docs, nd, Microsoft, ndb, Meta Platforms, Inc., nd, Supabase, nd].
- **SQL:** The language used to interact with the PostgreSQL-based relational database. It was used to define and manage the schema for storing user accounts, solar panel metadata, telemetry records, missions, faults, and inspection results [PostgreSQL Global Development Group, nd].
- **HTML & CSS:** The standard markup and styling languages used to build the web interface. CSS, specifically through the Tailwind CSS framework, was used to ensure that the SolarSense dashboard is responsive, consistent, and visually intuitive for users [Meta Platforms, Inc., nd, Tailwind Labs, nd].

2. Computer Vision & Detection Tools:

- **YOLOv8 (Ultralytics):** A state-of-the-art object detection framework used to identify and classify solar panel conditions and visible faults from inspection images. In the current SolarSense pipeline, it supports classes such as clean, dusty, electrical-damage, physical-damage, and snow-covered panel states [Ultralytics, nd].
- **Albumentations:** A Python library for image augmentation. It was used to expand the training dataset by applying transformations such as RandomResizedCrop, Rotate, Flip, Blur, and GaussNoise, which improves the model's ability to generalize across varied environmental conditions.
- **OpenCV:** An open-source library used for image processing tasks such as reading and resizing images, converting color spaces, supporting

heuristic fallback analysis, and implementing the `dhash_64` algorithm to identify near-duplicate images in the dataset [OpenCV, nd].

3. Machine Learning & Prediction Tools:

- **TensorFlow & Keras:** Open-source libraries used to develop and load the LSTM model. These tools were essential for building the neural network that analyzes time-series telemetry data, including voltage, current, and temperature, in order to predict power output and support anomaly analysis [Google, nd, Keras, nd].
- **Scikit-Learn:** Used for important preprocessing and training-support tasks, including `StandardScaler` to normalize telemetry features and `LabelEncoder` to support class encoding within the computer vision training workflow [scikit-learn developers, nd].

4. Frontend & Design:

- **React:** A JavaScript library for building user interfaces. It serves as the foundation of the SolarSense dashboard, providing a component-based architecture for pages such as overview, panels, anomalies, missions, authentication, and profile management [Meta Platforms, Inc., nd].
- **Vite:** A modern frontend build tool used to develop, bundle, and serve the React application efficiently during both development and production builds [Vite, nd].
- **Netlify:** A cloud platform used to host and deploy the React frontend. It provides a practical environment for publishing the user interface and handling single-page application routing [Netlify, nd].
- **Figma:** A cloud-based design tool used for preparing the UI/UX wireframes and interface designs of the application before and during the development process [Figma, nd].

5. Backend & Database:

- **FastAPI:** A high-performance Python framework used to build the system's RESTful API. It manages the flow of data between the ML models, the database, the image-analysis pipeline, and the user interface [FastAPI, nd].
- **SQLAlchemy:** An Object-Relational Mapping (ORM) tool used to model and manage the backend database entities, including panels, telemetry records, missions, faults, mission images, and inspection results [SQLAlchemy, nd].
- **Supabase (PostgreSQL):** A cloud-based platform used for authentication, storage, and PostgreSQL-backed relational data management. It stores persistent system data, including user accounts, panel information, telemetry logs, mission images, and inspection records [Supabase, nd, PostgreSQL Global Development Group, nd].

6. Hardware & Embedded Systems:

- **ESP32 Development Board:** The main microcontroller used in the hardware prototype. It is responsible for reading sensor inputs, processing the collected values, calculating the panel power, and controlling the display output [Espressif Systems, nd].
- **Voltage Sensor Module:** Used to measure the output voltage of the solar panel and provide a safe scaled signal to the ESP32.
- **Current Sensor Module:** Used to measure the current flowing from the solar panel by converting the sensed current into an analogue voltage signal that can be read by the ESP32.
- **Light Intensity Sensor Module:** Used to measure the surrounding light intensity falling on the solar panel in order to support environmental monitoring.
- **Temperature Sensor:** Used to measure the ambient or panel temperature, which is an important factor affecting solar panel performance.
- **16×2 I2C LCD:** Used to display the measured parameters, including

voltage, current, light intensity, temperature, and calculated power, in real time.

- **Solar Panel:** Acts as the primary monitored energy source in the prototype and provides the electrical input to be measured by the sensing modules.
- **Charging Module:** Used to regulate the charging process of the battery and provide overcharging protection.
- **Lithium Battery:** Used as the energy storage unit to support portable and continuous system operation.

7. Development, Deployment & Data Management:

- **Visual Studio Code:** The primary Integrated Development Environment (IDE) used for writing, organizing, and debugging the application code across the hardware and software layers [Microsoft, ndc].
- **Docker:** Used to containerize the backend service and package the runtime ML and CV artifacts required for deployment [Docker, Inc., nd].
- **Microsoft Azure:** A cloud deployment platform used to host the containerized FastAPI backend, particularly through Azure Container Apps and Azure Container Registry [Microsoft, nda].
- **GitHub:** Used for version control, source code management, collaboration, and maintaining the central project repository throughout development and deployment workflows [GitHub, nd].

5.5 AI Model Components Implementation

The artificial intelligence framework of the SolarSense system is designed as a hybrid architecture that processes both spatial and temporal data to ensure comprehensive solar panel monitoring. This implementation is divided into two primary domains: a CV model, which utilizes the YOLOv8 architecture for real-time defect detection and health state classification from image data, and a ML

model based on LSTM networks for analyzing telemetry sequences. While the CV components focus on identifying visible anomalies such as physical damage or shading, the ML components perform high-accuracy regression to predict future power output and detect internal performance deviations. Together, these components provide an integrated pipeline for automated inspection and proactive maintenance.

5.5.1 CV Model Components Implementation

The artificial intelligence core of the SolarSense system includes a computer vision component responsible for analyzing solar panel images and identifying their surface condition. This section details the implementation of the CV model used to classify solar panel images into predefined health states.

5.5.1.1 Component 1: Image Classification

Image classification is the primary layer of the vision system. Instead of detecting localized bounding boxes, the current implementation classifies the full solar panel image into one of the supported condition categories. This approach was selected because the dataset represents broad panel surface conditions, such as dust, snow, and visible damage, rather than precise object-level annotations.

5.5.1.1.1 You Only Look Once The system uses a YOLOv8 classification model through the Ultralytics framework. YOLOv8 was selected due to its efficient feature extraction, fast inference speed, and suitability for real-time inspection workflows. In this implementation, the model classifies solar panel images into five condition states: *Clean*, *Dusty*, *Electrical-damage*, *Physical-Damage*, and *Snow-Covered*.

5.5.1.1.2 Classification Training The training pipeline was implemented in Python using the *Ultralytics* framework. The following configurations were applied:

- **Image Size:** All input images were resized to 640×640 pixels for consistency.

- **Data Split:** A 70% training, 15% validation, and 15% testing ratio was used.
- **Preprocessing:** Difference Hashing (D-Hash) was implemented to remove duplicate and near-duplicate images before splitting the dataset, reducing redundancy and limiting overfitting.
- **Augmentation:** Data augmentation was applied only to the training set, while the validation and testing sets remained unaugmented to ensure fair evaluation.

5.5.1.2 Data Augmentation

To improve model generalization under different environmental and lighting conditions, the *Albumentations* library was used for data augmentation. The augmentation process included:

- **Geometric Transformations:** Random resized cropping, rotation, affine transformation, and horizontal flipping.
- **Photometric Transformations:** Brightness and contrast adjustment, gamma correction, CLAHE, hue-saturation-value adjustment, Gaussian noise, Gaussian blur, and motion blur.
- **Environmental Effects:** Random rain, fog, and sun flare effects to simulate outdoor inspection conditions.
- **Training Configuration:** Training was conducted for 80 epochs using a batch size of 6 and the AdamW optimizer.

5.5.1.3 Classification Results

The trained CV model was evaluated using the held-out test split. The evaluation measured classification accuracy, precision, recall, F1-score, and inference time. The final model achieved strong performance across the five solar panel condition classes.

- **Accuracy:** 0.84, indicating that the model correctly classified 84% of the test images.

- **Macro Precision:** 0.86, representing the average precision across all five classes.
- **Macro Recall:** 0.85, showing the average ability of the model to correctly identify each class.
- **Macro F1-score:** 0.86, indicating balanced classification performance across the classes.
- **Weighted F1-score:** 0.84, reflecting the overall F1-score while considering the number of samples in each class.
- **Test Samples:** 96 images were used in the held-out test set.

These results indicate that the model can reliably classify most test images and is fast enough for near real-time solar panel inspection.

5.5.1.4 Model Pipeline

The implemented computer vision pipeline follows the sequence below:

1. **Image Ingestion:** A solar panel image is captured or uploaded to the system.
2. **Preprocessing:** The image is resized and prepared for YOLOv8 inference.
3. **Inference:** The trained YOLOv8 classification model performs a forward pass and predicts the most likely panel condition.
4. **Post-processing:** The predicted class label and confidence score are extracted from the model output.
5. **Output:** The classification result is stored in the system and displayed on the user dashboard.

5.5.2 ML Model Components Implementation

The implementation phase of the ML model focuses on processing sequential sensor data to predict the future performance of solar panels. The system utilizes deep learning to analyze temporal patterns in voltage, current, and temperature derived from the SolarSense system architecture.

5.5.2.1 Component 1: Performance Regression & Prediction

The first component handles the numerical analysis of telemetry data, which is collected via the system's Module Sensors. This component identifies trends in power output based on historical sensor readings.

5.5.2.1.1 Time-Series Forecasting Instead of spatial detection, the model performs temporal forecasting. It identifies patterns over a defined **WINDOW_SIZE** of 20 timesteps. This allows the system to identify if current power levels are deviating from historical norms.

5.5.2.1.2 Model Training Training was conducted using a dataset of simulated and real-time telemetry records:

- **Feature Engineering:** Input features include `voltage`, `current`, and `temperature`.
- **Normalization:** A `StandardScaler` was applied to ensure all sensor readings are on a uniform scale with a mean of 0 and a variance of 1.
- **Optimization:** The model used the **Adam optimizer** with a **Mean Squared Error (MSE)** loss function, training for 50 epochs with a batch size of 32.

5.5.2.2 Component 2: Sequence Classification

This component evaluates the state of the solar panel based on predicted telemetry trends and temporal relationships within the data store.

5.5.2.2.1 Proposed Model Architecture LSTM The architecture is a deep sequential model designed to handle temporal dependencies. The model follows the structure:

- **LSTM Layer 1:** 64 units with `return_sequences=True` to pass temporal data to the next layer.
- **Dropout Layer:** A rate of 0.2 to prevent overfitting by randomly deactivating neurons.

- **LSTM Layer 2:** 32 units with `return_sequences=False` to condense the temporal information.
- **Dense Layers:** A 32-unit hidden layer with **ReLU activation** followed by a final linear output layer for regression.

5.5.2.2.2 Sequence Training Results The training pipeline was evaluated by splitting the data into 80% training, 10% validation, and 10% testing. This ensures that the classification of future power states remains accurate across unseen time intervals.

5.5.2.3 Telemetry Prediction Results

The performance of the LSTM model was evaluated using a held-out test set of 288 samples. The added result figures illustrate the comparison between the actual and predicted power values. As shown in the figures, the model achieved strong predictive performance on unseen data, with low prediction error and a close alignment between the predicted and actual telemetry trends.

Figure 5.1 presents the training and prediction performance of the telemetry prediction model. The loss curve shows that both the training and validation losses decreased steadily across the training epochs, indicating that the LSTM model successfully learned the relationship between the input telemetry features and the target power output. The prediction plot compares the actual and predicted power values on the test data. Although small differences can be observed, the predicted curve follows the general trend of the actual power output, showing that the model is capable of estimating solar panel power behavior from unseen telemetry data.

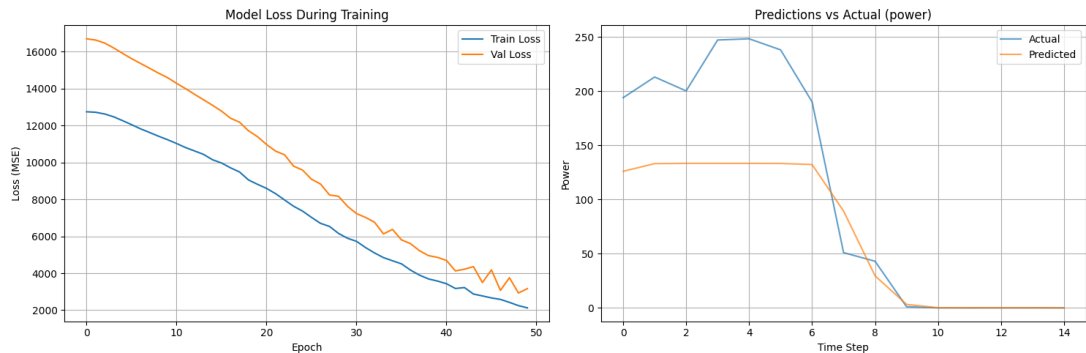


Figure 5.1: LSTM Model Performance

5.5.2.3.1 Final Performance Metrics Based on the test set performance summary, and the goodness of fit figure, the model achieved the following metrics:

- **Mean Squared Error:** 18.9800.
- **Root Mean Squared Error :** 4.3600 W, representing the average prediction error magnitude.
- **Mean Absolute Error :** 2.6200 W, indicating the mean absolute deviation from actual values.
- **R² Score:** 0.9410 (94.10%), meaning the model explains 94.10% of the variance in power predictions.
- **Correlation:** 0.9701, showing a strong linear relationship between predicted and actual values.

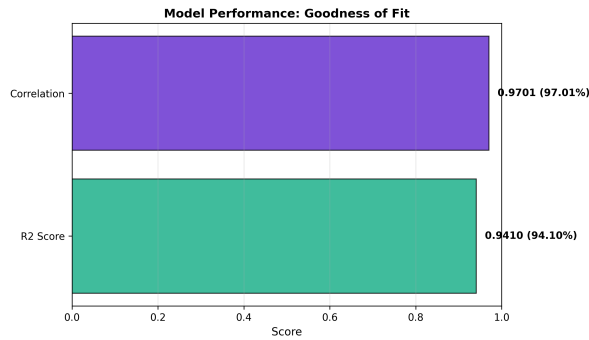


Figure 5.2: Goodness of Fit

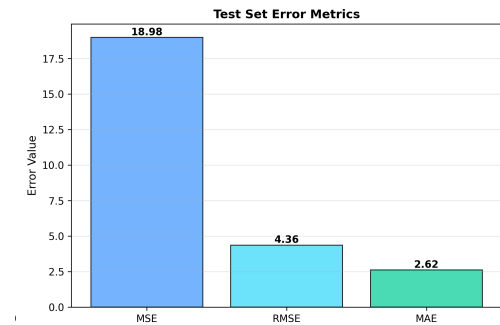


Figure 5.3: Test Set of Metrics

5.5.2.4 Model Pipeline

The integrated ML pipeline follows a structured flow from data collection to insight generation:

1. **Data Acquisition:** Fetching telemetry records (voltage, current, temperature) from the database via `SessionLocal`.
2. **Sequence Creation:** Transforming raw data into windowed sequences of 20 steps.
3. **Preprocessing:** Applying the `StandardScaler` and saving the scaler objects (`.joblib`) for production.

4. **Anomaly Detect:** Using the prediction trends to identify deviations in the Data Store.
5. **Visualization:** Generating comparison plots through the Dashboard View to show Actual vs. Predicted values.

5.6 Front-end Implementation

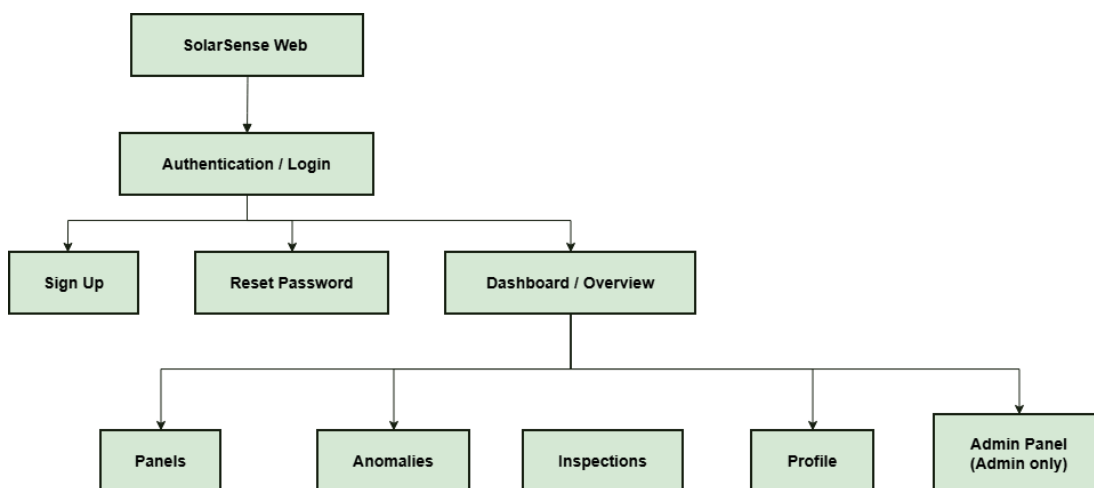


Figure 5.4: SolarSense Site Map

SolarSense is a web-based monitoring and inspection platform that supports the supervision of solar-panel sites and the management of anomaly-detection results. The front-end layer was implemented using React with TypeScript and built through Vite, which provides a modern development environment and efficient rendering performance. In addition, Tailwind CSS and reusable user-interface components were employed to provide a responsive and consistent visual design. The application depends on React Router for navigation between pages and React Query for asynchronous data fetching, caching, and synchronisation with the back-end services.

The front-end architecture was designed according to a component-based approach in order to improve maintainability and reuse. The main layout consists of a sidebar and a header, which provide unified navigation across the application pages. Authentication and authorisation were integrated into the user interface through Supabase, while protected routes and role-based routes were implemented to ensure that each user accesses only the functions permitted for that role. In

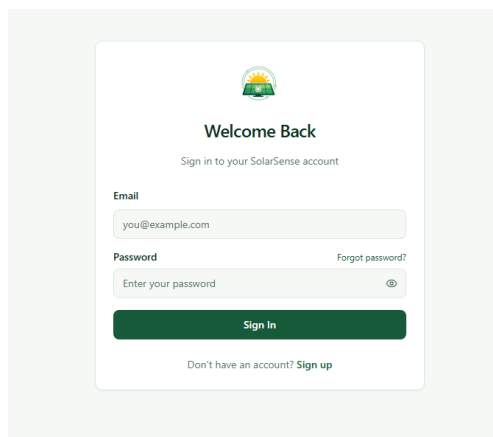
this way, general authenticated users can access the overview and profile pages, whereas operator and administrator roles can access operational pages such as panel management, anomaly monitoring, and inspection handling.

The implemented interfaces in SolarSense are described as follows:

- **Login Page:** This page allows the user to sign in to the system using registered credentials. The authentication process is linked to Supabase authentication services, and access is granted only after successful verification of the user session.
- **Sign Up Page:** This page enables new users to create an account in the system. The collected user information is then associated with the corresponding profile record and role definition used throughout the application.
- **Reset Password Page:** This page supports account recovery by allowing the user to initiate the password reset procedure in case access credentials are forgotten.
- **Overview Page:** This page acts as the main dashboard of the system. It presents summarised information about active, warning, faulty, and total panels, in addition to telemetry charts, recent anomalies, mission queues, and a panel heatmap that helps users understand the global operational state of the monitored sites.
- **Panels Page:** This page displays all registered solar panels in tabular form and supports filtering by site, status, and search terms. It also allows the user to inspect panel details, including the latest telemetry reading, number of anomalies, and mission history.
- **Anomalies Page:** This page combines machine-learning anomalies and computer-vision inspection findings into one monitoring view. It enables filtering by severity, site, and model type, which assists the user in reviewing the most critical abnormal conditions first.
- **Inspections Page:** This page manages inspection missions associated with solar panels. It enables authorised users to create a new inspection, upload

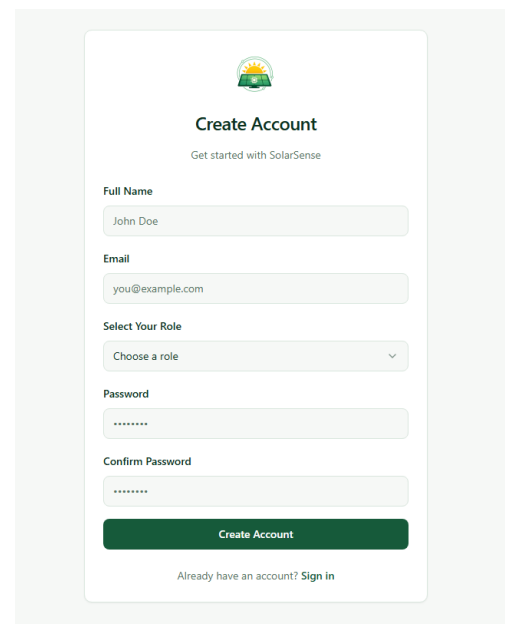
images, review mission details, and update the inspection status once the process is completed.

- **Profile Page:** This page allows the user to view and update personal information such as name and avatar. It also provides account-related information, including role and membership date.
- **Admin Page:** This page is restricted to administrators and is used to manage users and assign roles. Through this interface, the administrator can view user statistics and change the role of each registered account.



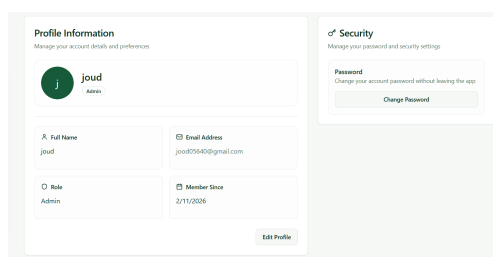
The Sign In page features a central white card on a light gray background. At the top of the card is a logo of a green sun rising over a green field. Below the logo, the text "Welcome Back" is displayed in bold, followed by "Sign in to your SolarSense account" in a smaller font. The form includes an "Email" field with the placeholder "you@example.com", a "Password" field with the placeholder "Enter your password" and a "Forgot password?" link, and a green "Sign In" button. At the bottom, there is a link that says "Don't have an account? Sign up".

Figure 5.5: Sign In Page



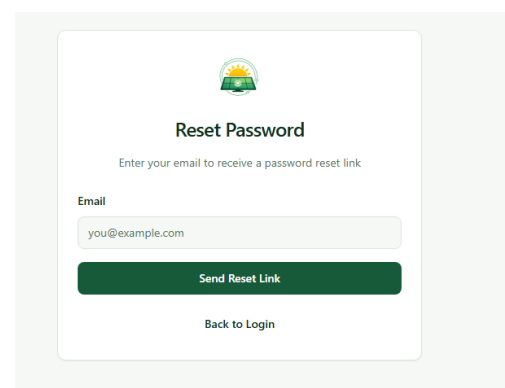
The Sign Up page features a central white card on a light gray background. At the top of the card is a logo of a green sun rising over a green field. Below the logo, the text "Create Account" is displayed in bold, followed by "Get started with SolarSense" in a smaller font. The form includes a "Full Name" field with the placeholder "John Doe", an "Email" field with the placeholder "you@example.com", a "Select Your Role" dropdown menu with the placeholder "Choose a role", a "Password" field with the placeholder "*****", and a "Confirm Password" field with the placeholder "*****". A green "Create Account" button is at the bottom, with a link that says "Already have an account? Sign in" below it.

Figure 5.6: Sign Up Page



The Profile page is divided into two main sections: "Profile Information" and "Security". The "Profile Information" section on the left includes a profile picture placeholder with the name "joud", a "Full Name" field with the value "joud", an "Email Address" field with the value "joud85640@gmail.com", a "Role" dropdown menu with the value "Admin", and a "Member Since" field with the value "2/11/2026". There is an "Edit Profile" button at the bottom right of this section. The "Security" section on the right includes a "Password" field with the placeholder "Change your account password without leaving the app" and a "Change Password" button.

Figure 5.7: Profile Page



The Reset Password page features a central white card on a light gray background. At the top of the card is a logo of a green sun rising over a green field. Below the logo, the text "Reset Password" is displayed in bold, followed by "Enter your email to receive a password reset link" in a smaller font. The form includes an "Email" field with the placeholder "you@example.com" and a green "Send Reset Link" button. At the bottom, there is a link that says "Back to Login".

Figure 5.8: Reset Password Page

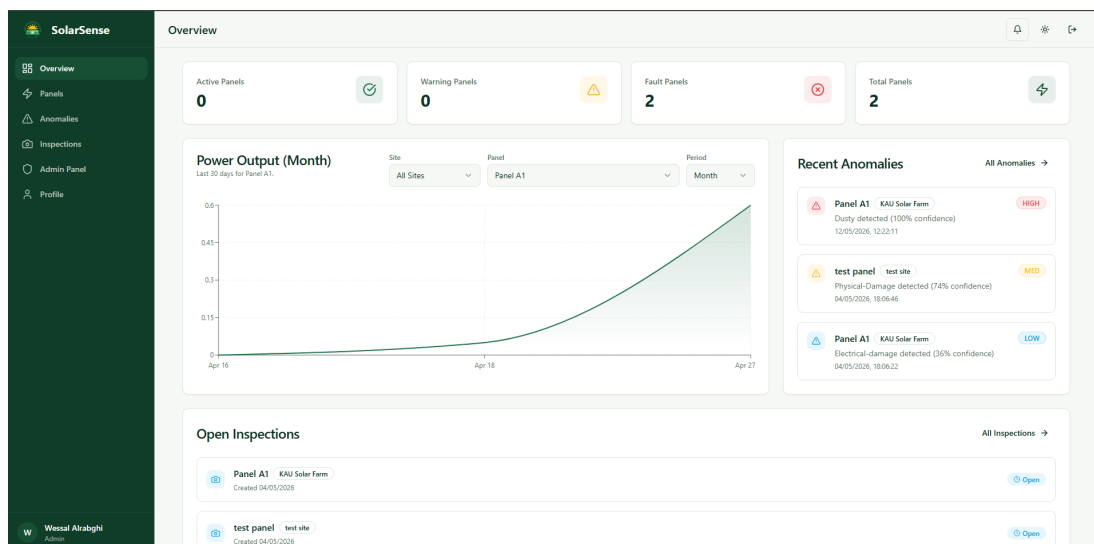


Figure 5.9: Dashboard Part 1

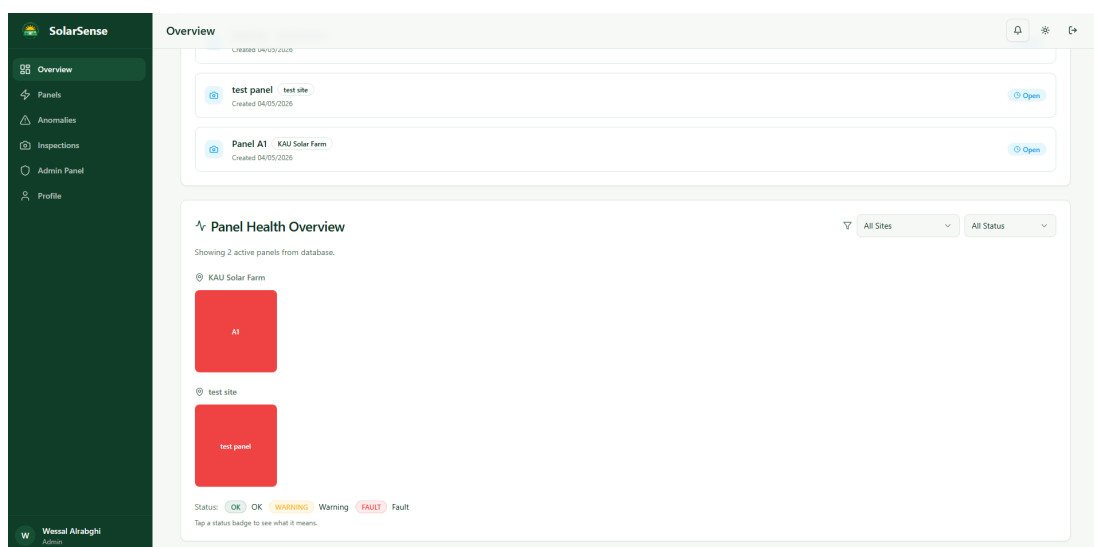
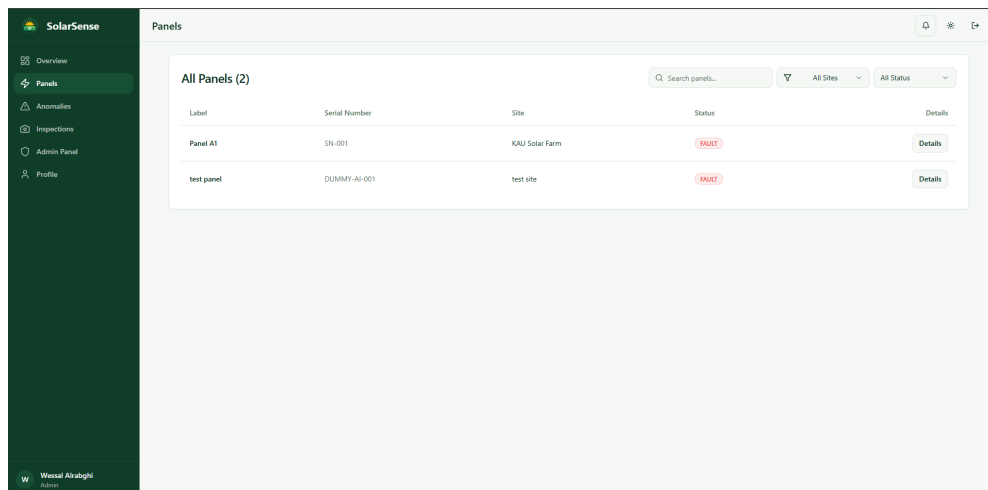


Figure 5.10: Dashboard Part 2



SolarSense

Panels

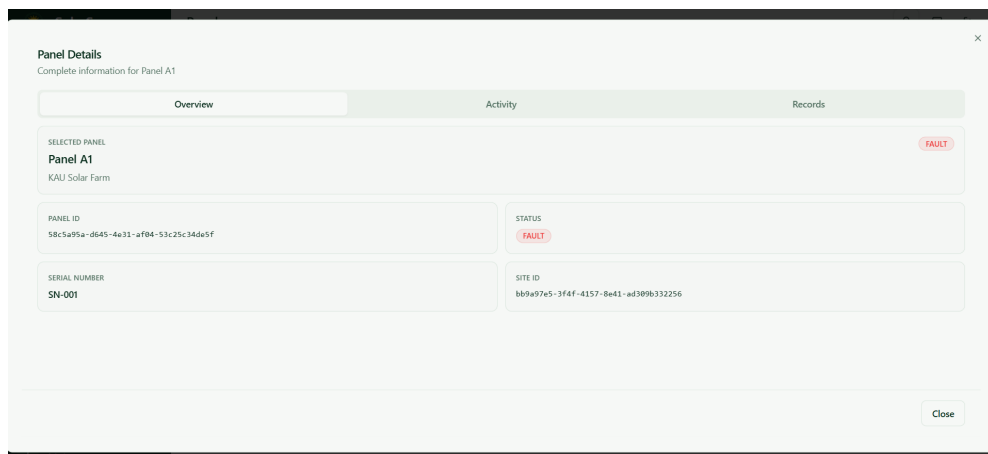
All Panels (2)

Search panels... All Sites All Status

Label	Serial Number	Site	Status	Details
Panel A1	SN-001	KAU Solar Farm	FAULT	Details
test panel	DUMMY-AI-001	test site	FAULT	Details

Wissal Alkhatib
Admin

Figure 5.11: Panels Details



Panel Details
Complete information for Panel A1

Overview Activity Records

SELECTED PANEL
Panel A1
KAU Solar Farm

STATUS
FAULT

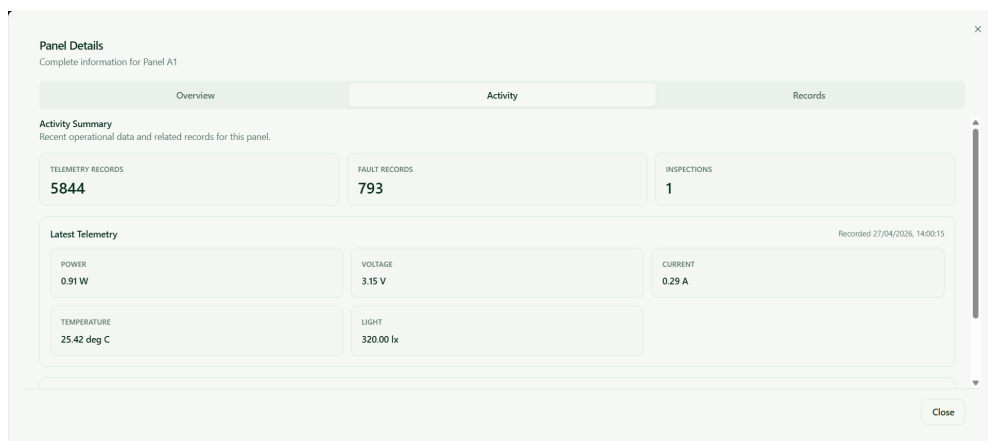
PANEL ID
58c5a95a-d645-4e31-af04-53c25c34de5f

SERIAL NUMBER
SN-001

SITE ID
bb9a97e5-3f4f-4157-8e41-ad309b332256

Close

Figure 5.12: Panel Details Overview



Panel Details
Complete information for Panel A1

Overview Activity Records

Activity Summary
Recent operational data and related records for this panel.

TELEMETRY RECORDS
5844

FAULT RECORDS
793

INSPECTIONS
1

Latest Telemetry
Recorded 27/04/2026, 14:00:15

POWER
0.91 W

VOLTAGE
3.15 V

CURRENT
0.29 A

TEMPERATURE
25.42 deg C

LIGHT
320.00 lx

Close

Figure 5.13: Panel Details Activity

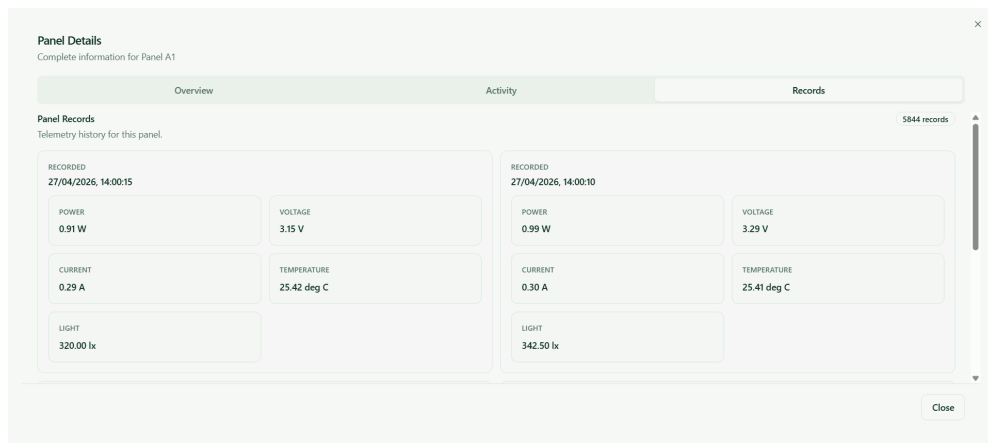


Figure 5.14: Panel Details Records

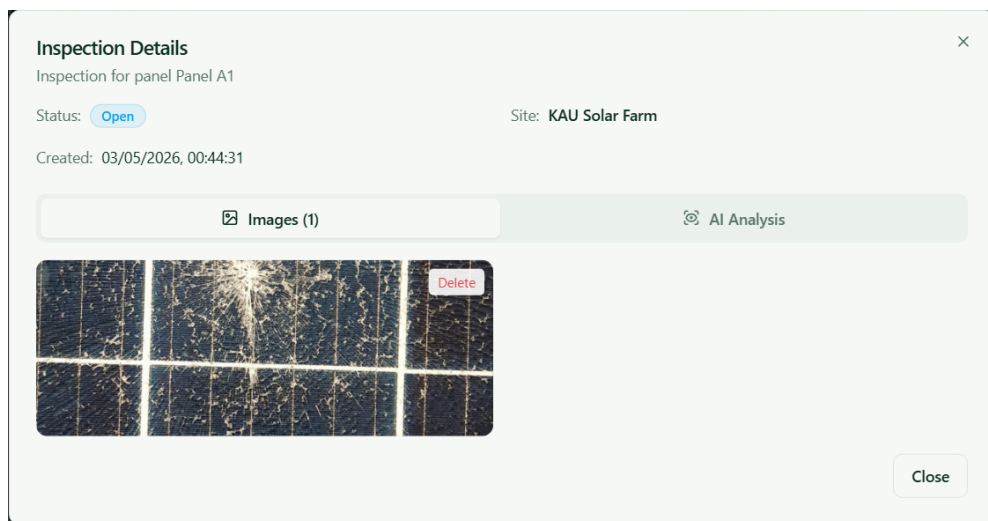


Figure 5.15: Panel Images

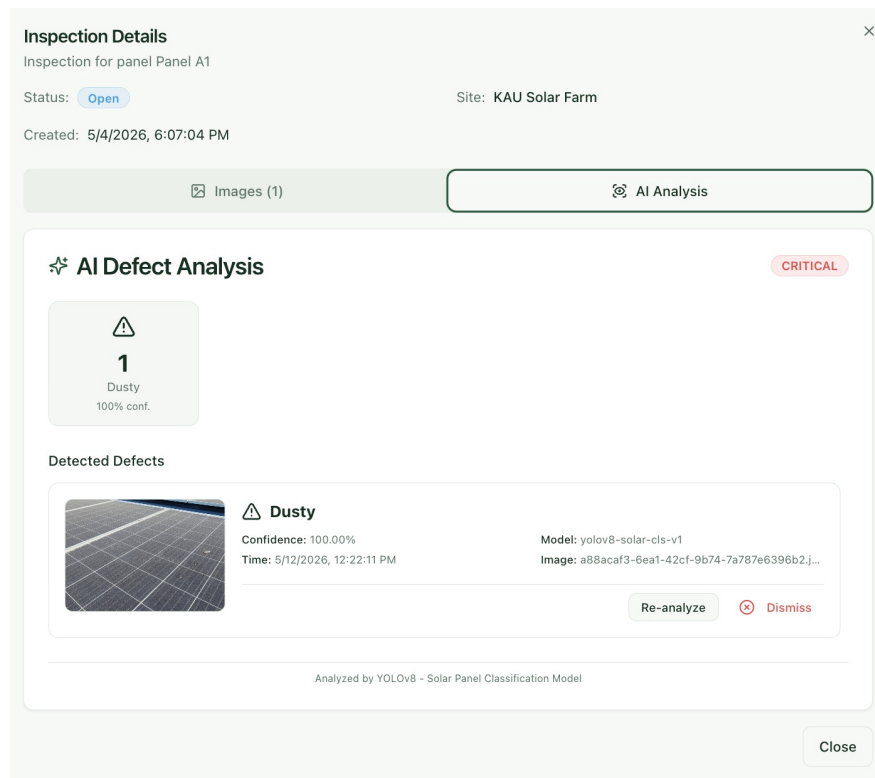


Figure 5.16: CV Results

Overall, the front-end implementation was designed to provide a clear and efficient interaction flow between monitoring, anomaly analysis, and inspection management. The use of reusable components, protected navigation, and integrated data services contributed to an interface that is scalable, maintainable, and appropriate for real-time operational use.

5.7 Back-end Implementation

The back-end layer of SolarSense was implemented to provide secure data management, service orchestration, and integration with the intelligent analysis modules. This layer was developed using FastAPI in Python because of its high performance, clear routing structure, and suitability for building RESTful services. The back-end communicates with the front-end and the database, manages the persistence of telemetry and inspection data, and connects the application with the machine-learning and computer-vision models. To simplify maintenance and extensibility, the back-end was organised into modular components including routers, database models, schemas, services, and security utilities, where each module is responsible for a specific functional domain such as sites, panels, teleme-

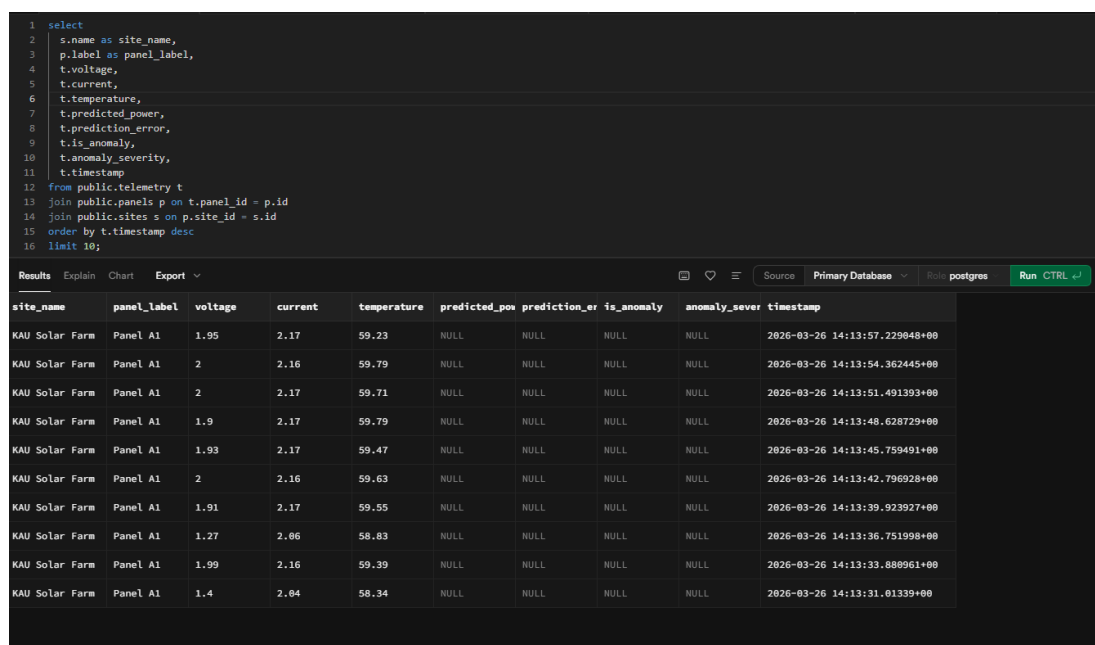
try, faults, missions, mission images, inspection results, profiles, and administration. In addition, Cross-Origin Resource Sharing (CORS) was configured to allow controlled communication between the front-end client and the API.

5.7.1 Database Implementation

The database is one of the core components of SolarSense because it stores operational measurements, user information, inspection records, and analysis results. The implemented solution relies on a PostgreSQL-compatible structure connected through Supabase services, while the back-end accesses the same data layer through SQLAlchemy models. This approach made it possible to combine managed authentication and storage services with a structured relational database design. The database schema was designed to reflect the actual relationships between the monitored entities, where the `sites` table stores the main information about each solar site, the `panels` table stores panel identifiers and status information linked to each site, and the `telemetry` table stores continuous sensor readings and derived analytical attributes such as predicted power, prediction error, anomaly severity, and analysis time. To support anomaly and inspection workflows, additional tables were implemented, including `faults` for machine-learning anomaly events, `missions` for inspection tasks, `mission_images` for uploaded image metadata, `inspection_results` for computer-vision outputs, and `profiles` for user information and access-control roles. The database design was therefore normalised around the monitored solar panel as the central entity, allowing support for historical tracking, fault analysis, inspection auditing, user management, and future scalability.

Figure 5.14 provides direct evidence of the implemented SolarSense database by showing a real SQL query executed on the Supabase-hosted PostgreSQL instance and the returned records. The query joins the `telemetry`, `panels`, and `sites` tables to retrieve recent operational measurements together with their associated site and panel identifiers, demonstrating that the schema is not only designed conceptually but is actively populated and relationally connected in practice. The returned results from KAU Solar Farm and Panel A1 confirm that voltage, current, temperature, and timestamped telemetry readings are being

stored correctly, while the presence of analytical fields such as predicted power, prediction error, and anomaly severity further shows that the database structure supports future machine-learning and anomaly-analysis workflows within the SolarSense platform.



```

1 select
2   s.name as site_name,
3   p.label as panel_label,
4   t.voltage,
5   t.current,
6   t.temperature,
7   t.predicted_power,
8   t.prediction_error,
9   t.is_anomaly,
10  t.anomaly_severity,
11  t.timestamp
12 from public.telemetry t
13 join public.panels p on t.panel_id = p.id
14 join public.sites s on p.site_id = s.id
15 order by t.timestamp desc
16 limit 10;

```

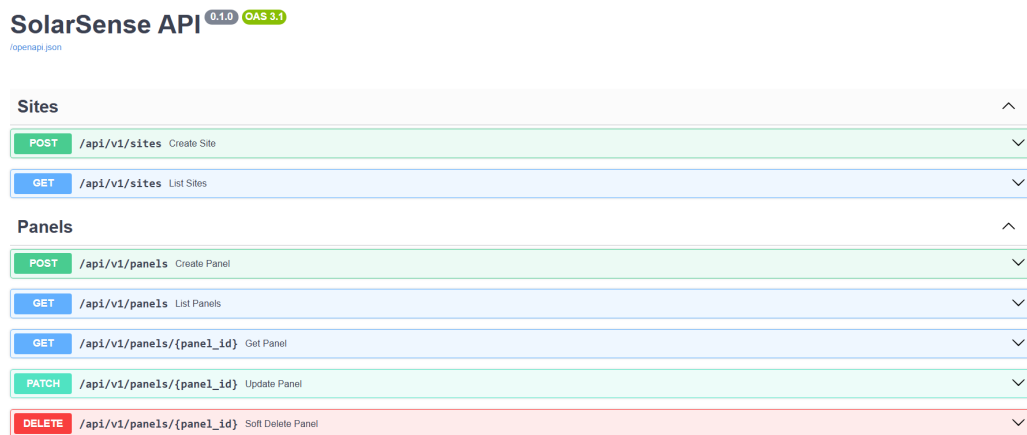
site_name	panel_label	voltage	current	temperature	predicted_pos	prediction_er	is_anomaly	anomaly_sever	timestamp
KAU Solar Farm	Panel A1	1.95	2.17	59.23	NULL	NULL	NULL	NULL	2026-03-26 14:13:57.229048+00
KAU Solar Farm	Panel A1	2	2.16	59.79	NULL	NULL	NULL	NULL	2026-03-26 14:13:54.362445+00
KAU Solar Farm	Panel A1	2	2.17	59.71	NULL	NULL	NULL	NULL	2026-03-26 14:13:51.491393+00
KAU Solar Farm	Panel A1	1.9	2.17	59.79	NULL	NULL	NULL	NULL	2026-03-26 14:13:48.628729+00
KAU Solar Farm	Panel A1	1.93	2.17	59.47	NULL	NULL	NULL	NULL	2026-03-26 14:13:45.759491+00
KAU Solar Farm	Panel A1	2	2.16	59.63	NULL	NULL	NULL	NULL	2026-03-26 14:13:42.796928+00
KAU Solar Farm	Panel A1	1.91	2.17	59.55	NULL	NULL	NULL	NULL	2026-03-26 14:13:39.923927+00
KAU Solar Farm	Panel A1	1.27	2.66	58.83	NULL	NULL	NULL	NULL	2026-03-26 14:13:36.751998+00
KAU Solar Farm	Panel A1	1.99	2.16	59.39	NULL	NULL	NULL	NULL	2026-03-26 14:13:33.880961+00
KAU Solar Farm	Panel A1	1.4	2.64	58.34	NULL	NULL	NULL	NULL	2026-03-26 14:13:31.01339+00

Figure 5.17: implemented SolarSense database in Supabase

5.7.2 API Implementation

The API was implemented using FastAPI to facilitate communication between the user interface, the database, and the intelligent analysis services. The API follows a modular REST-based design in which each router handles a specific domain, and separate endpoints were developed for sites, panels, telemetry, faults, missions, mission images, inspection results, profiles, administrative operations, and computer-vision status checking. The telemetry API plays a central role because it receives new sensor readings, stores them in the database, and can automatically trigger machine-learning analysis on recent telemetry records in order to predict power output and detect anomalies. When abnormal behaviour is detected, the back-end persists the analytical metrics, creates fault records, and can automatically generate an open inspection mission for the affected panel, thereby reducing manual intervention and helping operators respond more quickly to potential failures. The mission-images API was developed to support visual inspection by retrieving uploaded images from storage, applying

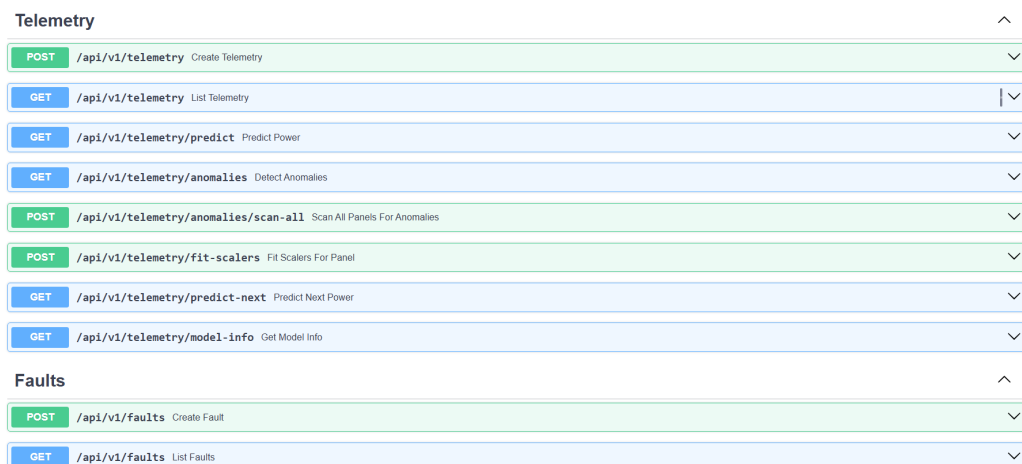
the computer-vision detection service, and storing the resulting defect detections in the `inspection_results` table. The implemented computer-vision service primarily uses a YOLOv8 model for solar-panel defect detection, while a heuristic OpenCV-based procedure may be used as a fallback when the trained model is unavailable. In addition to operational endpoints, the back-end also includes administrative services for user-role management and security enforcement, where access to selected endpoints is restricted so that only authorised users can create inspections, upload images, or manage other users. Consequently, the API implementation serves not only as a communication layer, but also as the coordination layer that links monitoring, prediction, defect detection, and inspection management into one integrated system.



SolarSense API 0.1.0 OAS 3.1
/openapi.json

Sites	
POST	/api/v1/sites Create Site
GET	/api/v1/sites List Sites
Panels	
POST	/api/v1/panels Create Panel
GET	/api/v1/panels List Panels
GET	/api/v1/panels/{panel_id} Get Panel
PATCH	/api/v1/panels/{panel_id} Update Panel
DELETE	/api/v1/panels/{panel_id} Soft Delete Panel

Figure 5.18: API Implementation in PostgreSQL Part 1



Telemetry	
POST	/api/v1/telemetry Create Telemetry
GET	/api/v1/telemetry List Telemetry
GET	/api/v1/telemetry/predict Predict Power
GET	/api/v1/telemetry/anomalies Detect Anomalies
POST	/api/v1/telemetry/anomalies/scan-all Scan All Panels For Anomalies
POST	/api/v1/telemetry/fit-scalers Fit Scalers For Panel
GET	/api/v1/telemetry/predict-next Predict Next Power
GET	/api/v1/telemetry/model-info Get Model Info
Faults	
POST	/api/v1/faults Create Fault
GET	/api/v1/faults List Faults

Figure 5.19: API Implementation in PostgreSQL Part 2

Missions			^
POST	/api/v1/missions	Create Mission	▼
GET	/api/v1/missions	List Missions	▼
GET	/api/v1/missions/{mission_id}	Get Mission	▼
POST	/api/v1/missions/{mission_id}/complete	Complete Mission	▼
Mission Images			^
POST	/api/v1/mission-images	Create Mission Image	▼
GET	/api/v1/mission-images	List Mission Images	▼
GET	/api/v1/mission-images/{image_id}	Get Mission Image	▼
DELETE	/api/v1/mission-images/{image_id}	Delete Mission Image	▼
POST	/api/v1/mission-images/{image_id}/analyze	Analyze Mission Image	▼
POST	/api/v1/mission-images/{image_id}/re-analyze	Reanalyze Mission Image	▼
GET	/api/v1/mission-images/{image_id}/results	Get Image Analysis Results	▼
Inspection Results			^
POST	/api/v1/inspection-results	Create Inspection Result	▼
GET	/api/v1/inspection-results	List Inspection Results	▼
GET	/api/v1/inspection-results/cv-anomalies	List Cv Anomalies	▼
GET	/api/v1/inspection-results/{inspection_id}	Get Inspection Result	▼

Figure 5.20: API Implementation in PostgreSQL Part 3

CV			^
GET	/api/v1/cv/status	Cv Status	▼
Admin			^
GET	/api/v1/admin/users	List Users	▼
PATCH	/api/v1/admin/users/{user_id}/role	Update User Role	▼
profiles			^
GET	/profiles/{user_id}	Get Profile	▼
PUT	/profiles/{user_id}	Update Profile	▼
default			^
GET	/health	Health	▼

Figure 5.21: API Implementation in PostgreSQL Part 4

5.8 Chip Implementation

The SolarSense system includes a hardware component responsible for real-time data acquisition from solar panels. This component is implemented using an ESP32 microcontroller integrated with multiple sensors to monitor environmental and electrical parameters. The ESP32 acts as an IoT device that collects, processes, and transmits telemetry data to the backend system via Wi-Fi.

The sensing unit is designed to measure key parameters such as voltage, current, temperature, and light intensity. These readings are essential for an-

alyzing the performance of solar panels and detecting anomalies. The ESP32 continuously reads analog and digital sensor values, converts them into meaningful units, and sends them as structured JSON data to the backend API.

The system utilizes the following sensors and modules:

- **BH1750 Light Sensor:** Measures light intensity (lux) to evaluate sunlight exposure.
- **Analog Voltage Sensor:** Reads the voltage generated by the solar panel.
- **Current Sensor:** Measures the electrical current flowing through the system.
- **Temperature Sensor:** Monitors the temperature of the panel or surrounding environment.
- **LCD Display (I²C):** Displays real-time readings locally for quick monitoring.

The ESP32 is programmed using the Arduino IDE, where Wi-Fi connectivity is established to enable communication with the backend server. Once connected, the device periodically sends telemetry data using HTTP POST requests in JSON format. This allows seamless integration with the backend API and database for storage and further analysis.

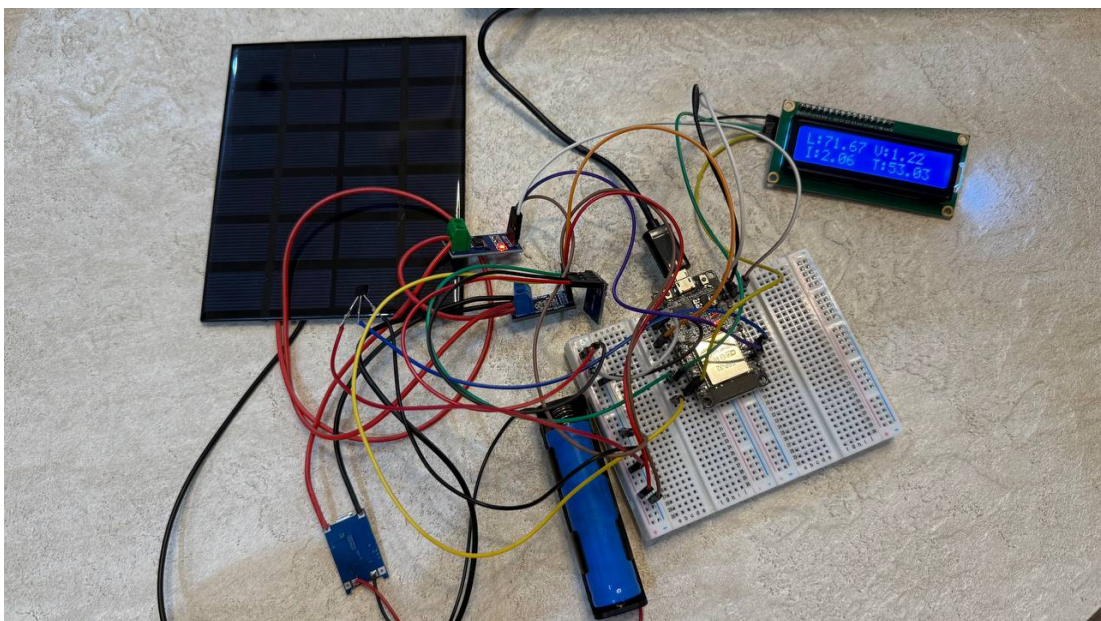


Figure 5.22: SolarSense Chip

```

Light: 107.50 | I:2.16 | V:2.01 | T:60.04
Sending JSON:
{"panel_id":"58c5a95a-d645-4e31-af04-53c25c34de5f","voltage":2.01,"current":2.16,"temperature":60.04,"light":107.50}
POST response code: 200
Response body:
{"id":"4043e5ce-706a-4ff9-b12f-20b427bf38f8","panel_id":"58c5a95a-d645-4e31-af04-53c25c34de5f","voltage":2.01,"current":2.16,"temperature":60.04,"light":107.50}
Light: 106.67 | I:2.23 | V:2.14 | T:59.47
Sending JSON:
{"panel_id":"58c5a95a-d645-4e31-af04-53c25c34de5f","voltage":2.14,"current":2.23,"temperature":59.47,"light":106.67}
POST response code: 200
Response body:
{"id":"fbee4c0-e862-4181-9625-8c8265d65db5","panel_id":"58c5a95a-d645-4e31-af04-53c25c34de5f","voltage":2.14,"current":2.23,"temperature":59.47,"light":106.67}
Light: 105.00 | I:2.22 | V:2.14 | T:59.88
Sending JSON:
{"panel_id":"58c5a95a-d645-4e31-af04-53c25c34de5f","voltage":2.14,"current":2.22,"temperature":59.88,"light":105.00}
POST response code: 200

```

Figure 5.23: ESP32 Serial Monitor Output Showing Real-Time Telemetry Data

5.9 Conclusion

In conclusion, this chapter has presented the implementation and evaluation of the SolarSense system. The development of the main software and hardware components has been described, and the adopted testing and evaluation methods have been outlined to examine their correctness, functionality, and usability.

Overall, the chapter has shown how the proposed system was translated from design into an operational prototype. The presented implementation details and evaluation outcomes provide support for the feasibility of the system and demonstrate that its main components were able to perform the intended monitoring, analysis, and inspection functions.

CHAPTER 6

TESTING

6.1 Introduction

Testing is an essential activity used to verify and validate whether a software system satisfies its specified requirements and behaves as expected [Sommerville, 2016]. In the SolarSense project, testing was required because the system consists of multiple integrated components, including the front-end dashboard, back-end API, database, AI models, and the ESP32-based sensing unit. Therefore, the testing process was designed to evaluate both the technical correctness of the system and the usability of the web platform.

The testing and evaluation phase of SolarSense was conducted to verify the functionality, reliability, and overall performance of the developed system. The adopted testing approach includes component testing, integration testing, system testing, black-box functional testing, AI model testing, hardware validation, and usability testing. These testing activities were selected to assess the system from different perspectives, including functionality, data flow, module interaction, model performance, sensor behaviour, task completion, and user satisfaction.

In this chapter, section 6.2 presents the purpose, scope, and objectives of the testing process. Then, section 6.3 discusses the testing of the main system components, while section 6.4 explains the different types of testing conducted, including component, unit, and integration testing. Section 6.5 presents the overall system testing, followed by section 6.6, which discusses software testing for both the front-end and back-end components. Section 6.7 presents the black-box functional testing activities, while section 6.8 discusses the testing and evaluation of the AI models, including both the computer vision and machine learning

models. Section 6.9 presents the hardware testing performed on the ESP32 sensing unit and connected sensors, including sensor validation, power calculation, data transmission, and real-time telemetry validation. Section 6.10 explains the usability testing process, including the participants, testing environment, evaluation tasks, and usability analysis. Finally, section 6.11 provides the conclusion of this chapter.

6.2 Purpose, Scope, and Objectives

This section defines the purpose, scope, and objectives of the SolarSense testing plan. It explains why testing was conducted, which system components were included in the validation process, and what outcomes were expected from the testing activities. The section also provides a clear boundary for the testing work by identifying the software, hardware, AI, and usability aspects covered in the prototype evaluation.

6.2.1 Purpose

The purpose of the SolarSense System Testing Plan (STP) is to define the testing approaches, procedures, and evaluation methods used to assess the implemented system. The plan aims to verify that the system functions correctly, handles data reliably, supports user interaction effectively, and provides accurate monitoring and inspection outputs. Testing was also used to identify defects, confirm expected behaviour, and provide confidence in the overall quality of the developed prototype.

6.2.2 Scope

The testing scope covers the main components of the SolarSense system, including:

- Front-end dashboard
- Back-end API
- Database interactions
- CV and ML model pipelines

- ESP32-based hardware sensing unit
- The usability of the web-based platform

The ESP32 sensing unit collects real-time telemetry data, including voltage, current, temperature, and light intensity. The software platform receives, stores, analyses, and displays these data through the dashboard. The CV model supports image-based fault detection, while the ML model supports telemetry-based prediction and anomaly-related analysis. The usability evaluation focuses on whether end users can complete the required system tasks effectively and efficiently.

6.2.3 STP Objectives

The objectives of the testing plan are as follows:

- Verify the functional correctness of the implemented system features;
- Test individual software and hardware components before full integration;
- Evaluate the interaction between the front-end, back-end, database, AI models, and hardware unit;
- Apply black-box testing to validate user-facing functions based on inputs and expected outputs;
- Test the AI model pipelines using model-specific performance and validation measures;
- Validate the hardware sensing unit using real sensor readings and calculated power values;
- Evaluate the usability of the web platform based on task completion, number of clicks, task duration, and user satisfaction;
- Identify remaining limitations for future improvement and deployment-oriented testing.

6.3 System Components Testing

The SolarSense system consists of several integrated components, including the user interface, the back-end API, the database, the AI models, and the ESP32-based sensing unit. The purpose of system component testing was to verify that these components operated correctly both individually and as part of the complete system workflow.

The testing focused on confirming that:

- Data were transmitted correctly between the sensing unit, back-end, database, and dashboard;
- Data retrieved from the database were displayed correctly to end users;
- User actions on the dashboard triggered the expected back-end requests;
- CV and ML outputs were generated and displayed correctly;
- Telemetry readings were collected, stored, and visualised in real time.

6.4 Types of Testing

The testing strategy for SolarSense was designed to cover both technical validation and user-centred evaluation. Since SolarSense combines a web-based dashboard, a back-end API, database services, AI models, and an ESP32-based sensing unit, different testing types were required. Each testing type was selected according to the role of the component being evaluated and the expected behaviour of that component within the complete system.

Table 6.1 summarises the testing types applied in SolarSense, their definitions, and the reason each type was selected for this project.

Table 6.1: Types of Testing Applied in SolarSense

Testing Type	Definition	Reason for Selection in SolarSense
Component Testing	Testing complete components separately before they are connected to the full system.	This type was selected to verify larger project components such as the ESP32 sensing unit, CV service, ML pipeline, back-end services, and dashboard modules before full system integration.
Unit Testing	Testing individual functions or small modules in isolation.	This type was selected to verify focused functions such as telemetry formatting, power calculation, image preprocessing, model loading, scaler loading, prediction output validation, panel status calculation, and ML sequence creation.
Integration Testing	Testing the interaction between different components or modules after they have been connected.	This type was selected because SolarSense depends on data flow between the ESP32 sensing unit, back-end API, database, AI models, and front-end dashboard.
System Testing	Testing the complete system as one integrated prototype according to the project scope and requirements.	This type was selected to confirm that the main workflows, including login, dashboard viewing, telemetry display, mission creation, image upload, CV result viewing, and logout, operated together correctly.
Black-Box Testing	Testing system behaviour based on inputs and expected outputs without examining the internal source code.	This type was selected to evaluate user-facing workflows from the operator perspective, including authentication, panel viewing, anomaly viewing, image upload, telemetry viewing, and logout.
Front-End Testing	Testing the user interface, navigation, displayed data, forms, pages, and user interactions.	This type was selected because SolarSense provides its monitoring and inspection functions through a web dashboard that must be clear, navigable, and responsive for users.
Back-End Testing	Testing server-side logic, API endpoints, database access, request handling, authentication checks, and error handling.	This type was selected because the back-end is responsible for connecting the dashboard, database, telemetry records, inspection missions, image uploads, and AI results.
AI Model Testing	Testing and evaluating the performance of the CV and ML models using suitable metrics.	This type was selected because the CV model must classify panel conditions from images, while the ML model must predict power output from telemetry sequences.
Hardware Testing	Testing the physical sensing unit, sensors, power calculation, data transmission, and real-time telemetry behaviour.	This type was selected because SolarSense relies on an ESP32-based sensing unit to collect real solar panel measurements and transmit them to the platform.
Usability Testing	Testing the system from the end-user perspective to evaluate ease of use, satisfaction, task completion, clicks, and task duration.	This type was selected to determine whether users could complete the main SolarSense tasks effectively and whether the interface was clear and satisfactory.

Overall, these testing types provided a structured validation approach for the SolarSense prototype. Technical testing was used to verify the correctness of the implemented components, while usability testing was used to evaluate the system from the user perspective. The following sections present the executed testing activities and their results.

6.4.1 Component Testing

Component testing was used to verify the behaviour of major SolarSense components before they were integrated into the complete system [Sommerville, 2016]. Unlike unit testing, which focuses on small functions, component testing evaluates a larger part of the system as a working component.

In SolarSense, component testing was applied to the ESP32 sensing unit, the CV service, the ML telemetry pipeline, the back-end services, and the front-end dashboard modules. For the hardware component, testing focused on checking whether the voltage, current, temperature, and light sensors produced reasonable readings. For the CV component, testing focused on image loading, model availability, classification behaviour, confidence filtering, and result extraction. For the ML component, testing focused on preprocessing, sequence generation, scaler loading, model loading, and prediction output format.

This testing helped confirm that each major component could perform its expected role before being connected with other system parts.

6.4.2 Unit Testing

Unit testing was used to test individual functions and small modules in isolation before they were used by larger components [Sommerville, 2016]. In SolarSense, unit testing was applied to focused software functions, including telemetry formatting, power calculation, required-feature validation, image preprocessing, scaler loading, sequence creation, prediction validation, panel status calculation, and fault deduplication.

The automated unit tests were implemented using `pytest`. These tests checked both normal and edge-case behaviour. For example, the telemetry tests verified that missing light values were handled safely, the preprocessing tests checked that missing or malformed telemetry fields were rejected correctly, and the panel status tests confirmed that telemetry and CV confidence values were converted into the correct panel status. The fault deduplication tests also verified that repeated ML faults within the cooldown period were not duplicated.

This testing reduced implementation errors by confirming that small parts of the system behaved correctly before they were used inside integration and

system-level workflows.

6.4.3 Integration Testing

Integration testing evaluates the interfaces and interactions between different system components [Sommerville, 2016]. In SolarSense, integration testing was required because the system depends on the interaction between the front-end, back-end, database, AI models, and the ESP32 sensing unit.

The integration tests focused on verifying that user actions in the dashboard triggered the correct back-end requests, that the back-end could read from and write to the database, that uploaded images could be linked to inspection missions, and that model outputs could be displayed correctly in the user interface. Integration-oriented checks were also conducted to confirm that telemetry readings could move from the sensing unit to the database and then to the dashboard.

6.5 System Testing

System testing evaluates the behaviour of the whole system according to the defined project scope [Sommerville, 2016]. In SolarSense, system testing was performed to verify that the main implemented modules worked together as one functional prototype. This included checking authentication, dashboard navigation, telemetry display, anomaly viewing, mission creation, image upload, CV result display, and logout.

The system testing was performed in a practical test environment that reflected the implemented prototype. Although the system was not deployed as a full production system, the tests provided practical evidence that the main workflow could be completed successfully.

Table 6.2 summarises the testing strategy used in SolarSense. The automated unit and integration tests were executed using `pytest`, where 40 automated test cases passed successfully. System testing was performed through black-box workflow testing, where 10 complete user workflows passed and 0 failed.

Table 6.2: Summary of Testing Strategy Results

Testing Strategy	Description	No. of Test Cases	No. of Testers	Result
Unit Testing	Tested individual functions and small modules in isolation, including telemetry preprocessing validation, panel status logic, and fault deduplication rules.	9	4	Passed
Component Testing	Tested larger system components separately, including the CV service, ML inference service, model artefact loading, scaler loading, sequence creation, and prediction output validation.	19	4	Passed
Integration Testing	Tested communication between connected components, including FastAPI telemetry insertion, Supabase REST fallback, automatic ML analysis triggering, fault creation, and telemetry prediction fallback behaviour.	12	4	Passed
System Testing	Tested the complete SolarSense prototype through main user workflows, including sign-up, login, dashboard viewing, panel details, anomaly viewing, mission creation, image upload, CV result viewing, telemetry viewing, and logout.	10	4	Passed

6.6 Software Testing

Front-end and back-end testing were conducted to verify the main software components of the SolarSense platform. The front-end testing focused on the web dashboard, user interactions, navigation, forms, data display, and inspection-related pages. The back-end testing focused on API behaviour, database connectivity, request handling, authentication-related checks, and data exchange between the system components.

This separation was used because the front-end and back-end perform different roles in the system. The front-end represents the part of the system directly used by operators, while the back-end manages data processing, storage, communication, and integration with other services. Testing both parts was necessary to confirm that the SolarSense platform worked correctly from both the user interface and server-side perspectives.

6.6.1 Front-End Testing

Front-end testing was performed to verify that the SolarSense web interface displayed the required information correctly and allowed users to complete the main monitoring and inspection workflows. The tested pages included the sign-up page, login page, dashboard, panels page, panel details page, anomalies page, inspections page, image upload page, and computer vision results page.

The front-end testing focused on checking whether users could navigate between pages, submit forms, view telemetry values, inspect panel information, create inspection missions, upload panel images, view CV results, and log out successfully. The results showed that the implemented interface supported the main user workflows required by the prototype. This confirmed that the front-end was suitable for presenting real-time monitoring data, inspection results, and system status information to users.

6.6.2 Back-End Testing

Back-end testing was conducted to validate the reliability of the FastAPI services and their interaction with the database and external services. First, database connectivity was verified using a dedicated database-check script to confirm that the application could establish a correct connection with the data layer. After that, the core API behaviour was validated through script-based HTTP calls, including analysis-related flows, to ensure correct request handling and response generation.

The API behaviour was also evaluated by sending valid and invalid HTTP requests and comparing the returned responses with the expected outputs. This included testing successful data retrieval, telemetry insertion, invalid request rejection, mission creation, image upload handling, CV result retrieval, and authenticated session behaviour. In addition, the robustness of the back-end was examined through the runtime safeguards implemented within the FastAPI layer, including authentication checks, role checks, request-parameter validation, and explicit error-handling paths.

Table 6.3: Summary of Software Testing Results

Software Component	Testing Focus	No. of Test Cases	No. of Testers	Result
Front-End Testing	Tested the SolarSense dashboard interface, including sign-up, login, dashboard navigation, panel details, anomaly viewing, mission creation, image upload, CV result display, telemetry viewing, and logout.	10	4	Passed
Back-End Testing	Tested FastAPI services, database connectivity, telemetry insertion, Supabase REST fallback, ML analysis triggering, fault creation, saved model artefact loading, request handling, and error-handling behaviour.	30	4	Passed

6.7 Black-Box Functional Testing

Black-box functional testing was conducted to evaluate the deployed SolarSense system from the user and application-interface perspective[Sommerville, 2016]. The tests were based on the main user workflows and deployed interfaces used by the front-end, including Supabase authentication, Supabase storage, and the configured FastAPI back-end. Each workflow was evaluated using an input or action, an expected output, the actual system response, and a final pass or fail result.

The black-box test was executed on 30 April 2026 against the deployed back-end and configured Supabase project. The test run started at 17:08:42 UTC and ended at 17:08:52 UTC. A disposable operator account was created for the test, and the workflows covered sign-up, login, dashboard loading, panel details, anomaly details, mission creation, image upload, computer vision result retrieval, telemetry viewing, and logout. The test results showed that all tested workflows were completed successfully, with 10 passed workflows and 0 failed workflows.

Table 6.4: Black-Box Testing Execution Summary

Item	Details
Test date	30 April 2026
Run start time	2026-04-30T17:08:42.491Z
Run end time	2026-04-30T17:08:52.615Z
Environment	Deployed FastAPI back-end and configured Supabase project
Total workflows tested	10
Passed workflows	10
Failed workflows	0

Table 6.5: Black-Box Functional Testing Results

Workflow	Input / Action	Expected Output	Actual Output	Result
Sign up	Create an operator account using a generated email address	Account request succeeds and the system returns the confirmation or session state	User was created successfully, a session was returned, and a user ID was generated	Passed
Login	Sign in using the generated operator email and password	Valid credentials create an authenticated session and return access to the protected application area	Authenticated session was returned with the same generated user ID	Passed
View dashboard	Load dashboard data sources, including sites, panels, missions, faults, CV anomalies, and telemetry	Dashboard widgets load without API errors and show KPI, list, and chart data	All requested dashboard data sources returned status 200, with 2 panels, 46 missions, and 5884 telemetry records	Passed
View panel details	Open details for Panel A1	Selected panel details and related telemetry or activity are displayed	Panel details returned status 200, and 5844 telemetry records were retrieved for Panel A1	Passed
View anomaly details	Open the anomaly list and inspect returned ML and CV anomaly rows	Anomaly rows load with fault type, panel, site, model or confidence, severity, and date information	ML faults and CV anomalies were retrieved successfully, with 794 ML faults and 20 CV anomalies	Passed
Create mission	Create a new inspection mission for Panel A1	Mission is created with an open status and is retrievable in the mission queue	Mission was created successfully with an OPEN status	Passed
Upload inspection images	Upload a sample physical-damage image to the created mission	Image is stored and a mission image record is created	Image was uploaded successfully and linked to the mission using a stored image record	Passed
View CV results	Run CV analysis for the uploaded image and open the stored results	CV analysis completes and the results endpoint returns the detection result list	CV analysis returned status 200, one detection was produced, and one result row was retrieved	Passed
View telemetry values	Open telemetry records for Panel A1	Voltage, current, temperature, light, timestamp, and calculated power values are available	Telemetry values were returned successfully, including voltage, current, temperature, light, timestamp, and calculated power	Passed
Logout	Sign out from the authenticated session	Session is cleared and protected application data requires a new login	Session was cleared successfully after logout	Passed

The results indicate that the deployed SolarSense prototype satisfied the expected behaviour for the tested user-facing workflows. Since the tests were performed through the deployed interfaces rather than by inspecting the source code, the results provide suitable evidence for black-box functional validation. However, the test was limited to the selected workflows and does not replace long-term stress testing, security testing, or full production acceptance testing.

6.8 AI Model Testing and Evaluation

The AI components of the SolarSense system were tested and evaluated to determine their suitability for the required analytical tasks. The CV model was evaluated based on its ability to classify solar panel surface conditions from images, while the ML model was evaluated based on its ability to process telemetry sequences and predict solar panel power output.

Table 6.6: Summary of AI Model Testing Results

AI Model	Testing Focus	No. of Test Cases	No. of Testers	Results	Status
Computer Vision Model	Tested image-based solar panel condition classification, including Clean, Dusty, Electrical-damage, Physical-Damage, and Snow-Covered classes. Automated tests also checked classification output, confidence filtering, fallback behaviour, and physical-damage filtering.	6	4	• Test accuracy: 0.84 • Weighted F1-score: 0.84	Passed
Machine Learning Model	Tested telemetry preprocessing, required-feature validation, sequence creation, scaler loading, model artifact loading, prediction output validation, and end-to-end power prediction inference.	18	4	• RMSE: 4.36 W • MAE: 2.62 W • R^2: 0.9410	Passed

6.8.1 Computer Vision Model Testing

The computer vision component was tested to evaluate its ability to classify solar panel surface conditions from inspection images. In the final SolarSense prototype, the CV pipeline follows an image-classification approach using YOLOv8 classification. The model analyses a complete solar panel image and assigns it to one of five implemented classes: Clean, Dusty, Electrical-damage, Physical-Damage, and Snow-Covered.

During development, an earlier experimental version of the CV pipeline used YOLOv8 as an object detection model. In that version, each image was assigned a full-image bounding box. However, the available dataset labels represented image-level panel conditions rather than exact defect locations. Therefore, the detection pipeline was not fully suitable for the available annotations because the model did not learn the precise position of defects. Instead, it mainly learned the overall appearance of the full image while still producing detection-style outputs.

This created a mismatch between the dataset format and the model task. Since the bounding boxes covered the full image, the detection model behaved similarly to a classifier, although it was evaluated using detection metrics. This limitation reduced the suitability of the detection approach for the project objective. Broad surface conditions, such as Dusty and Snow-Covered, could also be confused with Physical-Damage because the model was not trained using accurate localisation annotations.

To address this issue, the final pipeline was converted into a classification-first YOLOv8 workflow. The dataset was reorganised into classification folders for training, validation, and testing. Each image was assigned directly to its corresponding class without using artificial bounding boxes. The classification model used the YOLOv8 classification backbone `yolov8n-cls.pt`, which was more appropriate for the available image-level labels and the objective of identifying the overall panel condition.

The training process was executed using the project script `CV/yolov8.py`. The pipeline performed image curation and perceptual deduplication before the dataset was divided into training, validation, and testing sets. Data augmentation was applied only to the training split, while the validation and testing images remained unaugmented. This was done to support fair evaluation and reduce the possibility of data leakage. After augmentation, the training data were balanced to approximately 560 images per class.

The main training outputs were stored under the `CV/YOLO_RESULTS` directory. The important artefacts included the prepared classification dataset, the trained classifier weights, the stable runtime copy `best-cls.pt`, and the classification evaluation reports. During back-end runtime, the system prioritises the classification model weights when they are available.

Table 6.7: Comparison Between the Previous and Updated YOLOv8 Pipelines

Aspect	Previous YOLOv8 Detection Pipeline	Updated YOLOv8 Classification Pipeline
Model task	Object detection	Image classification
Dataset structure	YOLO detection format with bounding-box labels	Classification folders for training, validation, and testing
Annotation style	Full-image bounding box assigned to each image	Image assigned directly to one condition class
Learning objective	Detect a labelled image region	Classify the overall solar panel condition
Main limitation	Defect location was not truly learned because the boxes covered the full image	Precise localisation is not provided, but the task matches the image-level labels
Evaluation approach	Detection metrics such as precision, recall, mAP50, and mAP50-95	Classification metrics such as accuracy, F1-score, and confusion matrix

Table 6.8: Computer Vision Model Classes

Class	Description
Clean	Solar panel surface without visible fault or obstruction
Dusty	Solar panel surface affected by dust or soiling accumulation
Electrical-damage	Visible electrical-related damage or abnormal electrical condition
Physical-Damage	Visible physical damage such as cracks or broken surface areas
Snow-Covered	Solar panel surface covered by snow

The updated classification model was evaluated using the validation and held-out test splits. Since the final model performs image classification, accuracy and weighted F1-score were used as the main evaluation metrics. Detection metrics such as mAP50 and mAP50–95 were not used as the main final evaluation metrics for the updated classifier because the model no longer performs bounding-box localisation.

Table 6.9: Comparison of Computer Vision Testing Results Before and After Pipeline Change

Version	Model / Task	Metric	Result
Previous YOLOv8	YOLOv8 detection	Precision	0.7299
		Recall	0.7067
		mAP50	0.8202
		mAP50–95	0.7820
Updated YOLOv8	YOLOv8 classification	Validation accuracy	0.83
		Validation weighted F1-score	0.82
		Test accuracy	0.84
		Test weighted F1-score	0.84

The updated classifier showed different levels of performance across the five classes. The strongest result was achieved for the Snow-Covered class, with an F1-score of 1.00. Physical-Damage also achieved a strong F1-score of 0.88, followed by Dusty with an F1-score of 0.84. Electrical-damage and Clean achieved comparatively lower F1-scores of 0.78 and 0.79, respectively. These results indicate that some solar panel conditions were easier to distinguish than others.

Overall, the updated YOLOv8 classification pipeline was more consistent with the available dataset than the earlier detection-based approach. The previous pipeline used detection metrics on full-image bounding boxes, while the updated pipeline used classification metrics that matched the image-level labels. However, the final model identifies the overall condition of a solar panel image and does not provide precise defect localisation. Further testing with real inspection images from different cameras, lighting conditions, distances, and viewing angles would be required before full deployment.

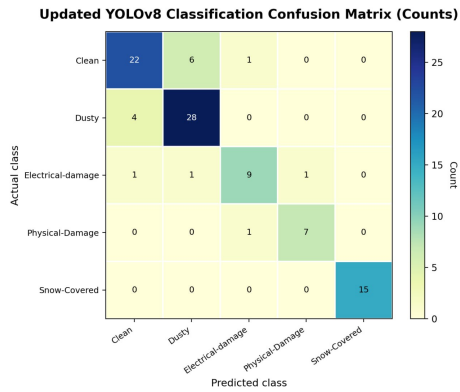


Figure 6.1: Confusion Matrix (Counts)

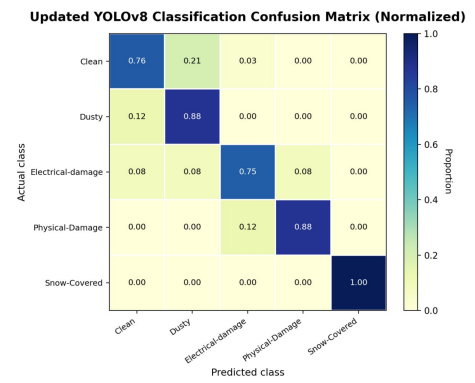


Figure 6.2: Normalized Confusion Matrix

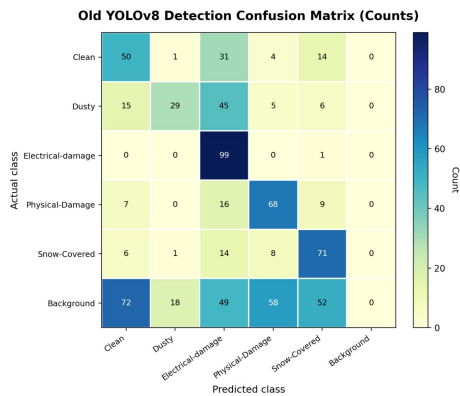


Figure 6.3: Old Confusion Matrix (Counts)

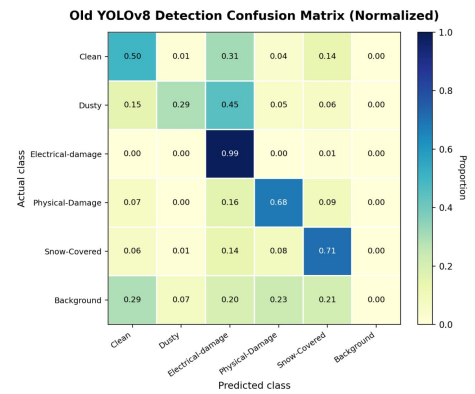


Figure 6.4: Old Normalized Confusion Matrix

The confusion matrices in Figures 6.1–6.4 further highlight the improvement achieved by the updated YOLOv8 classification pipeline. In the updated confusion matrix, most predictions are concentrated along the main diagonal, indicating that the majority of solar panel images were classified correctly. The Snow-Covered class achieved perfect classification performance, while Physical-

Damage and Dusty also produced strong results with only a few incorrect predictions. Some confusion remained between the Clean and Dusty classes, as both conditions can appear visually similar in certain lighting conditions. Electrical-damage was occasionally misclassified as Physical-Damage because some defect patterns share similar visual characteristics. In comparison, the earlier detection-based pipeline showed a larger number of incorrect predictions across multiple classes. This was mainly caused by the use of full-image bounding boxes, which prevented the model from learning accurate defect localisation. Overall, the updated classification-first approach produced more balanced and reliable results, demonstrating that it was better suited to the available image-level dataset annotations and the project objective.

6.8.2 Machine Learning Model Testing

The machine learning component was tested to evaluate its ability to predict solar panel performance from telemetry readings. The implemented model uses a Long Short-Term Memory (LSTM) neural network to process historical time-series sensor data and predict a selected target value, such as power, voltage, current, or temperature. In the SolarSense prototype, the main evaluation focused on power prediction because power represents the operational output of the solar panel and can be compared directly with the measured value calculated from voltage and current.

The model input consists of a time window of voltage, current, and temperature readings. The default sequence length was 20 time steps. The model architecture consisted of two LSTM layers with dropout, followed by a dense hidden layer and a final linear output layer. This architecture was selected because LSTM models are suitable for sequential telemetry data, where current behaviour may depend on previous readings.

The ML pipeline was tested through automated tests stored in the project-level `tests/` folder. These tests covered telemetry preprocessing, required-feature validation, sequence creation, window-size edge cases, scaler loading, model artefact loading, and lightweight inference using deterministic telemetry samples. In addition, the full test suite supported optional end-to-end inference using the

saved model artefact `ml/telemetry_power_model.h5`. This ensured that both the data-processing pipeline and the trained model artefact could be validated before integration with the back-end system.

Table 6.10: Machine Learning Pipeline Test Coverage

Test Area	Description	Purpose
Telemetry preprocessing	Validates that telemetry records contain the required input features, including voltage, current, and temperature	Data readiness
Sequence creation	Checks that time-series windows are created using the required sequence length	Input-shape validation
Window-size edge cases	Tests insufficient or boundary-size telemetry inputs	Robustness validation
Scaler loading	Verifies that preprocessing scaler artefacts can be loaded correctly	Pipeline reliability
Model artefact loading	Verifies that the saved trained model can be loaded successfully	Model availability
Lightweight inference	Runs prediction using deterministic telemetry samples	Output validation
End-to-end inference	Runs optional inference using the saved <code>telemetry_power_model.h5</code> artefact	Integration validation

The telemetry regression model was evaluated on a held-out test set for power prediction. The evaluation used standard regression metrics, including MSE, RMSE, MAE, R^2 , and correlation. These metrics were selected because the model output is numerical, and the objective was to measure the difference between predicted and actual power values.

Table 6.11: Machine Learning Power Prediction Results

Metric	Result
MSE	18.9800
RMSE	4.3600 W
MAE	2.6200 W
R^2	0.9410
Correlation	0.9701

The results show strong agreement between the predicted and actual power output. The RMSE value of 4.3600 W and the MAE value of 2.6200 W indicate that the model predictions usually remained within a few watts of the measured panel output. In addition, the R^2 value of 0.9410 and the correlation value of 0.9701 show that the model captured the overall power trend effectively. There-

fore, the model was considered suitable for supporting telemetry-based prediction and anomaly interpretation within the prototype scope.

Although the results were promising, several limitations were identified. First, the model was evaluated as a regression model and should not be described as a direct fault classifier. Fault or anomaly interpretation depends on the difference between predicted and actual telemetry behaviour. Second, the model performance depends on the quantity and quality of the collected telemetry data. Third, wider testing under different sunlight levels, shading conditions, temperatures, and panel conditions would be required before production deployment.

6.9 Hardware Testing

Hardware testing was conducted to validate the ESP32-based sensing unit and its ability to collect, process, transmit, and display real-time telemetry data from the solar panel environment. The hardware component was tested using actual connected sensors, including a voltage sensor, current sensor, temperature sensor, and BH1750 light sensor. The purpose of this testing was to verify that the sensing unit could produce valid measurements, calculate power correctly, transmit data to the back-end system, and support real-time dashboard monitoring.

Embedded system testing was selected to verify that the programmed chip could correctly control and communicate with the connected hardware components. Environmental testing was also applied to observe how the hardware responded under different light and operating conditions. Finally, hardware validation testing was conducted to confirm that the final hardware design met the required functionality and performed as expected in practical use.

6.9.1 Chip Testing

The ESP32 sensing unit is responsible for collecting real-time telemetry data from the solar panel system, including voltage, current, temperature, and light intensity. The chip was tested independently to verify the accuracy, reliability, and consistency of sensor readings and to confirm that telemetry data were transmitted correctly to the back-end system using HTTP requests.

6.9.1.1 Sensor Data Acquisition Testing

Each sensor connected to the ESP32 was tested individually to verify correct functionality and reasonable data readings.

- **Voltage Sensor Testing:** The voltage sensor was tested by measuring the output voltage of the solar panel under different lighting conditions. The readings were compared with expected solar panel behaviour to ensure that the output was reasonable.
- **Current Sensor Testing:** The current sensor was tested by observing current variation under different load and lighting conditions. The sensor successfully detected changes in current values.
- **Temperature Sensor Testing:** The temperature sensor was tested by exposing it to different environmental conditions. The readings changed logically when the surrounding temperature changed.
- **Light Sensor Testing:** The BH1750 light sensor was tested by varying the light intensity on the solar panel. The sensor readings increased under stronger light and decreased under lower light, confirming correct light measurement behaviour.

6.9.1.2 Power Calculation Validation

The system calculates real-time power using the measured voltage and current values according to the following Equation:

$$P = V \times I$$

A recorded chip reading showed a voltage of 21.85 V, a current of 4.41 A, and a displayed power value of 96.34 W. The manual calculation was performed as follows:

$$P = 21.85 \times 4.41 = 96.36W$$

The manually calculated value of 96.36 W closely matched the recorded chip value of 96.34 W. The small difference was considered acceptable because it was caused by normal rounding tolerance in the displayed readings. Therefore, the hardware test confirmed that the sensing unit calculated real-time power correctly for the recorded test case.

Table 6.12: Hardware Testing Evidence from Live Solar Panel Input

Measurement	Recorded Value	Result
Voltage	21.85 V	Passed
Current	4.41 A	Passed
Power	96.34 W	Passed
Light intensity	46%	Passed
Data transmission	JSON data sent to back-end API	Passed
Real-time dashboard display	Telemetry displayed on dashboard	Passed

6.9.1.3 Data Transmission Testing

The ESP32 was tested to verify its ability to send telemetry data to the back-end API using HTTP POST requests. During testing, the ESP32 successfully connected to Wi-Fi and transmitted JSON-formatted data containing voltage, current, temperature, and light readings. The back-end API returned a successful response, which indicated that the telemetry data were received and inserted into the database correctly.

```
Light: 107.50 | I:2.16 | V:2.01 | T:60.04
Sending JSON:
{"panel_id":"58c5a95a-d645-4e31-af04-53c25c34de5f","voltage":2.01,"current":2.16,"temperature":60.04,"light":107.50}
POST response code: 200
Response body:
{"id":"4043e5ce-706a-4ff9-b12f-20b427bf38f8","panel_id":"58c5a95a-d645-4e31-af04-53c25c34de5f","voltage":2.01,"current":2.16,"temperature":60.04,"light":107.50}
Light: 106.67 | I:2.23 | V:2.14 | T:59.47
Sending JSON:
{"panel_id":"58c5a95a-d645-4e31-af04-53c25c34de5f","voltage":2.14,"current":2.23,"temperature":59.47,"light":106.67}
POST response code: 200
Response body:
{"id":"f8e5ff4c0-e862-4181-9625-8c9265d65db5","panel_id":"58c5a95a-d645-4e31-af04-53c25c34de5f","voltage":2.14,"current":2.23,"temperature":59.47,"light":106.67}
Light: 105.00 | I:2.22 | V:2.14 | T:59.88
Sending JSON:
{"panel_id":"58c5a95a-d645-4e31-af04-53c25c34de5f","voltage":2.14,"current":2.22,"temperature":59.88,"light":105.00}
POST response code: 200
```

Figure 6.5: Example of transmitted JSON data and the corresponding API response

6.9.1.4 Real-Time Data Validation

Real-time data validation was performed to confirm that telemetry readings could move correctly from the sensing unit to the back-end system and then to the front-end dashboard. During testing, the ESP32 sent updated sensor readings repeatedly, and the values were reflected on the SolarSense dashboard. This confirmed the end-to-end integration between the hardware unit, database, back-end service, and front-end interface (Figure 6.5).

The results demonstrated that:

- sensor values were updated dynamically;
- telemetry data were correctly stored in the database;
- the front-end displayed the latest telemetry readings in real time.

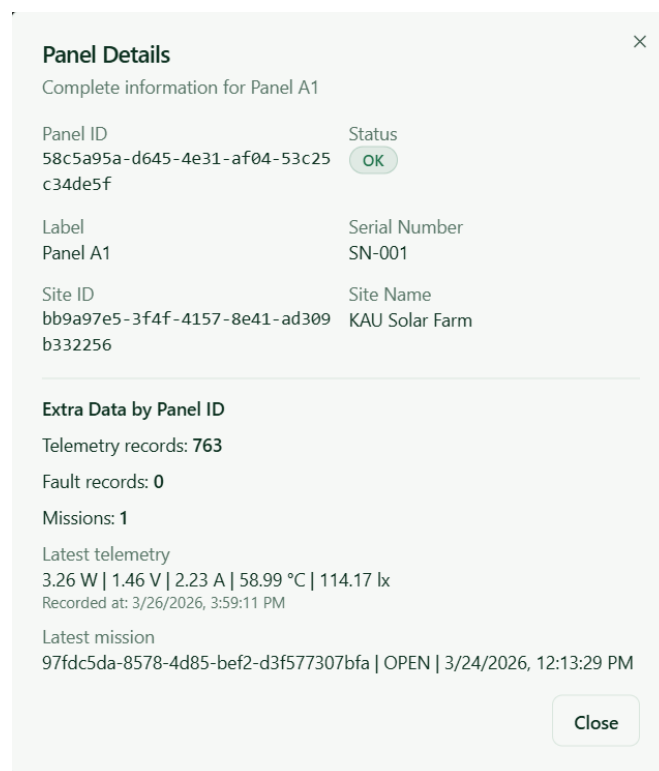


Figure 6.6: Real-time telemetry data displayed on the SolarSense dashboard

6.10 Usability Testing

Usability testing was conducted to evaluate the SolarSense web platform from the end-user perspective. The evaluation focused on whether users could complete

the required system tasks effectively and efficiently, and whether the interface was clear and satisfactory to use. The usability test was conducted remotely through an online Google Form after participants interacted with the SolarSense website.

6.10.1 Participants

The participants in this study consisted of users from different academic and technical backgrounds who were recruited to evaluate the usability of the SolarSense website through an online Google Form. Some of the participants had a technical background, while others had an electrical engineering background, which allowed the evaluation to reflect perspectives from users with different levels of familiarity with software systems and solar-related applications. The form was distributed electronically, and the evaluation was conducted remotely, enabling participants to access the SolarSense website through the provided link, perform the required usability tasks, and then submit their feedback online. This participant diversity was valuable for the usability evaluation because it supported the collection of opinions from users with varied experience and expectations regarding system functionality, interface clarity, navigation, and task efficiency.

6.10.2 Environment of the Test

Since SolarSense consists of two main components, a web-based dashboard and a hardware sensing unit, the testing environment was divided accordingly. The web application was tested by participants on their personal devices, such as laptops or desktop computers, in real-world environments such as homes or workplaces. These environments varied between quiet and moderately noisy settings, reflecting the actual conditions in which the system is expected to be used.

The hardware component, which consists of the ESP32 with connected sensors, was tested separately in a controlled setup using actual sensors connected to the system. The sensors were used to collect real measurements, including voltage, current, temperature, and light intensity, and the data were transmitted to the back-end system. Overall, these testing conditions represented realistic usage scenarios for both the software and hardware components of the SolarSense system.

6.10.3 Evaluation Tasks

The participants were asked to perform the following tasks during the usability evaluation of the SolarSense system:

1. Create a new user account in the system.
2. Log in to the system using the created or provided account credentials.
3. Open the overview dashboard and identify the displayed telemetry values, including power, voltage, current, and light intensity.
4. Locate an anomaly on the anomaly page and view its details.
5. Create a new mission for a selected site or panel.
6. Upload inspection images to the created mission.
7. Open the computer vision detection results and identify the detected fault type and confidence value.
8. Return to the dashboard and verify that the related mission or inspection status has been updated.
9. Log out of the system successfully.

6.10.4 Objective Measure Analysis

Two objective measures were used to evaluate task efficiency: the number of clicks required to complete each task and the time taken to complete the task. These measures were selected because they provide measurable indicators of navigation clarity and task completion effort. Lower click counts and shorter completion times generally indicate that users were able to locate functions and complete tasks more efficiently.

1. Number of Clicks

The first applied metric was the number of clicks required to complete each task. Table 6.13 summarises the five participants' results based on the number of clicks.

Table 6.13: Task Evaluation: Participants' Number of Clicks

No.	Task Description	Min Clicks	Max Clicks
1	Create a new user account (Sign Up)	1	More than two
2	Log in to the system	1	More than two
3	Open dashboard and identify values	1	2
4	Find panel anomaly and view details	2	2
5	Create a new mission (Site/Panel)	2	More than two
6	Upload inspection images	1	More than two
7	Open CV detection results (Fault/Conf)	2	2
8	Verify mission/inspection status update	1	2
9	Log out of the system successfully	1	1

2. Duration

The second applied metric was the amount of time taken to complete each task.

Table 6.14 summarises the five participants' results based on the task duration.

Table 6.14: Task Evaluation: Participants' Duration

No.	Task Description	Min Duration	Max Duration
1	Create a new user account (Sign Up)	Less than 1 minute	1 minute
2	Log in to the system	Less than 1 minute	2 minutes
3	Open dashboard and identify values	Less than 1 minute	2 minutes
4	Find panel anomaly and view details	Less than 1 minute	2 minutes
5	Create a new mission (Site/Panel)	Less than 1 minute	2 minutes
6	Upload inspection images	Less than 1 minute	1 minute
7	Open CV detection results (Fault/Conf)	Less than 1 minute	Less than 1 minute
8	Verify mission/inspection status update	Less than 1 minute	Less than 1 minute
9	Log out of the system successfully	Less than 1 minute	1 minute

6.10.5 Subjective Measure Analysis

Subjective measurement was used to evaluate users' opinions, perceived ease of use, satisfaction, and improvement suggestions after interacting with the SolarSense platform. A short post-test survey was used to collect responses related to ease of use, overall satisfaction, and open comments.

1. **Ease of Use:** Ease of use is a key usability factor that reflects how simple and intuitive the system interface is. Based on the collected responses, 80% of participants indicated that the SolarSense system was easy to use, while 20% rated it as moderate. None of the participants reported difficulty in using the system. These results suggest that the system interface was generally clear, user-friendly, and easy to navigate.



Figure 6.7: Ease of use results

2. **Satisfaction:** User satisfaction measures how well the system meets user expectations and provides a positive experience. According to the results, 80% of participants reported being very satisfied with the system, while 20% selected “maybe”. No participants reported being unsatisfied. This indicates that the system provided a satisfying user experience and met user expectations in terms of functionality and design.

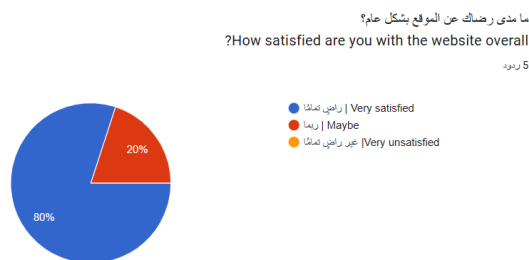


Figure 6.8: Satisfaction results

3. **Feedback:** Participants were asked to provide additional feedback and suggestions for improving the SolarSense system. The responses included the following:

- Add a landing page instead of redirecting directly to the login page.
- Improve the clarity of computer vision results for better understanding of solar panel conditions.
- Add automated email alerts for critical panel defects in future versions.
- Enhance the user interface and improve the AI defect feature.

These suggestions highlight areas for future improvement, particularly in user interface design, system communication, alerting, and result visualisation.

6.11 Conclusion

In conclusion, this chapter has presented the main testing and evaluation activities carried out on the SolarSense system. The results showed that the implemented software modules, AI components, and hardware sensing unit were able to perform their intended functions with acceptable reliability, while the usability evaluation indicated that the web platform was generally clear, usable, and effective for completing the required tasks. Overall, the findings of this chapter provide practical evidence that the developed system is feasible in its current form and establish a basis for future improvement and more comprehensive deployment-oriented validation.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 Introduction

SolarSense was developed as an intelligent solar panel monitoring and inspection platform that combines software engineering, AI, CV, ML, IoT, and hardware components into one integrated prototype. The system supports solar farm operators by collecting telemetry readings, analysing solar panel images, displaying monitoring results, and assisting in fault review and maintenance decision-making. Although the prototype achieved its main objectives, further improvement is still required to increase its real-world reliability, model accuracy, and deployment readiness.

In future development, SolarSense should be enhanced through larger and more diverse datasets collected from actual solar installations. This would improve the generalisation of the ML and CV models across different environmental conditions, panel types, lighting situations, and fault patterns. In addition, the hardware unit, deployment environment, inspection workflow, and dashboard features could be improved to make the system more suitable for outdoor use and large-scale solar farm operation.

This chapter presents the final reflection on the SolarSense project and identifies the main areas that can guide its future development. Section 7.2 discusses the major technical and organisational challenges faced during the project lifecycle and explains how these challenges were managed. Section 7.3 presents the required and learned skills gained throughout the development process. Section 7.4 describes possible future improvements and enhancements for the SolarSense platform. Finally, Section 7.5 provides the summary of the chapter.

7.2 Challenges and Limitations

This section presents the main challenges and difficulties encountered during the development of SolarSense. Since the project combined software engineering, AI, CV, ML, IoT, and hardware components, several technical and organisational challenges were faced throughout the project lifecycle. These challenges were related to scope selection, resource selection, multidisciplinary integration, model development, hardware implementation, deployment, and testing. The following points explain each challenge and describe how it was managed during the development of the prototype.

- **Selection of Suitable Resources:** Another challenge was selecting suitable resources for the different parts of the system. SolarSense required two main types of data: numerical telemetry data for the ML component and image data for the CV component. In addition, appropriate tools, libraries, development frameworks, datasets, and hardware components had to be selected. This challenge was more complex because the project required resources from several related areas, including AI, particularly CV and ML, as well as IoT, electrical engineering, and software engineering. To manage this issue, the tasks were distributed among the team members so that each member could study a specific topic, compare available resources, and contribute to selecting the most suitable datasets, techniques, and tools for the project scope.
- **Multi-Disciplinary Project Integration:** The project was challenging because SolarSense was not based on one isolated component, but on the integration of several connected components. The system included front-end interfaces, a back-end API, a database, CV analysis, ML processing, and an ESP32-based hardware unit. Each component had different requirements, technologies, and implementation constraints, and the required knowledge was distributed across several areas, including AI, IoT, electrical engineering, and software engineering. This challenge was managed by organising the time and effort allocated to each area, studying the requirements of each component separately, and then validating the integration between the components through several testing phases, including unit testing, compo-

nent testing, integration testing, AI model testing, hardware testing, and usability testing.

- AI Model Development and Training Limitations:** AI model development was one of the main challenges because two different AI components had to be developed and integrated. The CV model was required to analyse solar panel images, while the ML model was required to process numerical telemetry data. Several model options and configurations were tested before the final models were selected. Training was initially performed using Google Colab because it provided GPU access, but GPU usage limits and session restrictions created difficulties during experimentation. After training, the models were transferred to the local development environment and integrated with the system using VS Code. To support local testing after the transfer, an NVIDIA RTX 4050 GPU was used, which helped with model execution and testing, although the transfer and compatibility checks increased the overall testing time. This challenge was addressed through additional pipeline testing, including preprocessing, sequence creation, scaler loading, model loading, inference testing, and end-to-end inference testing.
- Scope Reduction Due to External Constraints:** Another difficulty was that not all external collaboration opportunities were successful. Some suggested features required access to external organisations, additional field data, specialised equipment, or real solar installation environments. Since these resources were not always available within the project timeframe, the project scope had to be reduced and controlled. However, this reduction was managed without removing the main functionalities of the system. Instead, the final scope was refined to focus on a functional prototype that still included the core SolarSense functions, such as telemetry monitoring, AI-based analysis, image inspection, dashboard visualisation, and hardware-based sensing.
- Hardware Development Difficulties:** The hardware component introduced additional challenges because it required knowledge beyond software development. The ESP32-based sensing unit required correct sensor con-

nections, voltage and current reading, light and temperature measurement, power calculation, LCD display output, and data transmission to the back-end system. Since this part involved electrical engineering concepts, self-learning was required by the team. In addition, collaboration was conducted with the Center of Research Excellence in Renewable Energy and Power Systems at King Abdulaziz University to supervise and guide the hardware part of the project. This support helped ensure that the hardware structure was more organised and suitable for the prototype. The hardware was also tested separately before being connected with the full system, including voltage, current, power, light intensity, data transmission, and real-time dashboard display.

- Testing Complexity:** Testing was more complex than expected because SolarSense includes several interconnected software, AI, and hardware components. A single testing type was not enough to validate the full system. Unit testing was needed for small functions such as telemetry preprocessing, power calculation, API input validation, and image format validation. Component testing was needed for larger modules such as CV image filtering, panel status logic, fault deduplication, and ML pipeline validation. In addition, integration testing, AI model testing, hardware testing, and usability testing were required to verify that the system worked correctly from different perspectives. Although this increased the testing effort, it improved confidence in the prototype and helped confirm that the main workflows were functioning as expected.
- Drone-Based Inspection Limitation:** At an earlier stage, SolarSense included drone-based image inspection as part of the planned inspection workflow. The aim was to allow solar panel images to be collected automatically from above, especially for large solar farms. However, this part of the scope was reduced because of practical limitations related to drone availability, access to a suitable drone operator, and restrictions on using drones during the project period. As a result, the inspection workflow was simplified into a camera-based image upload approach, where images are captured manually and uploaded to the system for CV-based analysis. This

scope reduction did not negatively affect the CV model because the model was not trained only on drone-captured images. Instead, the model was trained on general solar panel images representing different surface conditions, which made it suitable for analysing manually captured camera images as well as future drone-based images, provided that the images are clear and show the panel condition properly.

7.3 Required Learned Skills

Developing the SolarSense platform required a combination of technical, research, hardware, and project management skills. Since the project combines software engineering, artificial intelligence, IoT, machine learning, computer vision, and hardware integration, completing the system successfully required learning and applying several new skills throughout the project lifecycle. These skills were developed gradually during the analysis, design, implementation, testing, and deployment phases of the project.

The required and learned skills can be divided into the following categories:

1. Technical Skills

Many technical skills were required to design and implement the SolarSense platform, including:

- Ability to write, read, and understand code using different programming languages and technologies.
- Developing front-end interfaces and interactive dashboards for monitoring solar panel conditions.
- Building and managing back-end APIs and database integration.
- Using machine learning and computer vision techniques for solar panel fault detection and anomaly analysis.
- Processing and analysing sensor data collected from solar panels.
- Designing and implementing functional and non-functional system requirements.
- Creating system models, diagrams, and software architecture designs.

- Deploying and integrating the system components into a complete working platform.
- Using development environments, frameworks, and cloud deployment services.

In addition, the project required practical hardware and embedded systems skills. The team learned how to build and connect an ESP32 microcontroller circuit manually using multiple sensors, including voltage, current, temperature, and light sensors. This involved sensor wiring, circuit assembly, calibration, data collection, and establishing communication between the hardware unit and the software platform. The team also learned how IoT devices transmit real-time data to the backend system for monitoring and analysis purposes.

2. Project Management Skills

- Teamwork

The SolarSense project was developed as a collaborative team effort, which required strong teamwork and coordination between team members. This included task distribution, communication, problem solving, respecting deadlines, and supporting each other throughout the development process. Teamwork also helped improve decision-making and integration between the hardware, software, and AI components of the system.

- Effective Communication

Effective communication skills were essential during meetings with the supervisor, project discussions, presentations, and team coordination. The project also required explaining technical concepts clearly and documenting progress regularly through reports and presentations.

- Planning

Planning played an important role in managing the project phases efficiently. The team learned how to organize tasks, set timelines, divide responsibilities, track progress, and handle unexpected technical

challenges during implementation and testing. Proper planning helped ensure that the project components were completed and integrated successfully.

3. Applying Software Engineering Approach to an AI and IoT based Project

SolarSense is not only a software application, but also a research and engineering based project that combines artificial intelligence, IoT, and renewable energy technologies. Therefore, the project required applying software engineering methodologies throughout all development stages, including requirements analysis, system design, implementation, testing, deployment, and maintenance planning.

The team learned how to transform an initial idea into a complete intelligent monitoring platform by considering user requirements, system usability, reliability, and scalability. This also included integrating AI models, sensor based hardware systems, and real-time monitoring features into one unified platform.

4. Research Skills

The project required extensive research in areas such as solar energy systems, fault detection techniques, machine learning models, computer vision, IoT technologies, and smart monitoring systems. Through the different project phases, the team improved their ability to search for academic resources, analyse related work, compare existing solutions, and evaluate different technologies suitable for the project requirements. In addition, the team learned how to investigate practical challenges related to solar panel monitoring and propose suitable technical solutions based on research findings and system requirements.

5. Effective Writing and Documentation

Throughout the project milestones, the team improved their technical writing and documentation skills. Preparing the project reports required organizing ideas clearly, writing technical explanations, documenting system requirements and testing procedures, and presenting results in a structured

academic format. The team also learned how to write professional software engineering documentation, including system analysis, design descriptions, testing sections, risk management, implementation details, and user-related documentation in a clear and organized manner.

7.4 Future Work

Although SolarSense successfully achieved its main objectives and delivered a functional intelligent solar panel monitoring prototype, several improvements can still be implemented in future work to enhance the system capabilities and make it more suitable for real-world deployment.

- **More Detailed Electrical Fault Classification using CV:** The current CV model performs general solar panel fault detection and analysis. In future work, the system can be improved to classify faults into specific electrical fault categories instead of providing only general fault identification. For example, the model could distinguish between different electrical-related issues such as hotspot faults, disconnected cells, burn marks, shading problems, and other panel faults. This would provide more accurate diagnostic information and improve maintenance decision-making.
- **Adding a Detection Task to the CV Model:** The current CV model focuses mainly on classifying the overall condition of a solar panel image. In future work, a detection task can be added to locate the exact fault area within the image. This would allow the system to identify not only the type of fault, but also its position on the panel. For example, the model could detect the location of cracks, burn marks, hotspots, dust accumulation, or physical damage. This improvement would make the visual inspection results more useful for maintenance teams and would support more precise fault diagnosis.
- **Drone-Based Image Collection:** In the current prototype, solar panel images are uploaded manually by an operator. In future work, a drone-based image collection approach can be added to automate the inspection process, especially for large-scale solar farms. A drone could capture images

of multiple panels from different angles and send them to the system for CV-based analysis. This would reduce manual inspection effort, improve inspection coverage, and make the system more scalable for large installations. However, this improvement would require additional work related to drone control, flight planning, image quality, safety, and regulatory requirements.

- **Protective Hardware Enclosure:** The current hardware prototype mainly focuses on functionality and testing. In future work, the ESP32 sensing unit and connected sensors can be placed inside a properly designed protective box or enclosure. This would improve hardware protection, safety, durability, cable management, and suitability for outdoor environments by protecting the components from dust, heat, and environmental conditions.

7.5 Conclusion

This chapter presented the main challenges, required skills, and future work related to the development of SolarSense. The project faced several challenges, including selecting suitable resources, integrating multiple disciplines, developing and testing AI models, reducing the project scope due to external constraints, implementing the hardware unit, and validating the system through different testing phases. Despite these difficulties, the team managed to develop a functional prototype that combines telemetry monitoring, image inspection, AI-based analysis, hardware sensing, and dashboard visualization. The project also helped the team improve several important skills, including technical implementation, research, teamwork, communication, planning, documentation, and applying software engineering principles to an AI and IoT-based system. Finally, the chapter identified future improvements such as enhancing the CV model to classify more detailed electrical faults and placing the hardware components inside a protective enclosure to improve safety and durability. Overall, SolarSense demonstrates how intelligent monitoring technologies can support solar panel maintenance and contribute to more reliable renewable energy systems.

References

- [Abdulla et al., 2024] Abdulla, H., Sleptchenko, A., and Nayfeh, A. (2024). Photovoltaic systems operation and maintenance: A review and future directions.
- [Aghaei et al., 2025] Aghaei, M., Kolahi, M., Nedaei, A., Venkatesh, N. S., Esmailifar, S. M., Moradi Sizkouhi, A. M., Aghamohammadi, A., Oliveira, A. K. V., Eskandari, A., Parvin, P., Milimonfared, J., Sugumaran, V., and R  ther, R. (2025). Autonomous intelligent monitoring of photovoltaic systems: An in-depth multidisciplinary review. *Progress in Photovoltaics: Research and Applications*, 33:381–409.
- [Agrawal and Ahuja, 2021] Agrawal, R. and Ahuja, N. (2021). Deep learning models for anomaly detection in photovoltaic (pv) systems. *IEEE Transactions on Sustainable Energy*, 12(4):2150–2161.
- [Ait Abdelmoula et al., 2023] Ait Abdelmoula, I., Idrissi Kaitouni, S., Lamrini, N., Jbene, M., Ghennioui, A., Mehday, A., and El Aroussi, M. (2023). Towards a sustainable edge computing framework for condition monitoring in decentralized photovoltaic systems.
- [Alfehaid and Young, 2025] Alfehaid, R. M. and Young, K. E. (2025). Saudi arabia’s renewable energy initiatives and their geopolitical implications - center on global energy policy at columbia university sipa: Cgep
- [anikannal, 2020] anikannal (2020). Solar power generation data. Dataset.
- [Ansari, 2020] Ansari, S. (2020). *Building Computer Vision Applications Using Artificial Neural Networks: With Step-by-Step Examples in OpenCV and TensorFlow with Python*. Apress, Berkeley, CA.
- [Arduino, nd] Arduino (n.d.). Arduino ide documentation. <https://docs.arduino.cc/software/ide/>. Accessed 2026-03-29.
- [Asnil et al., 2023] Asnil, A., Krismadinata, K., Husnaini, I., Hazman, H., and Astrid, E. (2023). Real-time monitoring system using iot for photovoltaic parameters. *TEM Journal*, 12(3):1316–1322.
- [Boucif et al., 2025] Boucif, O. H., Lahouaou, A. M., Boubiche, D. E., and Toral-Cruz, H. (2025). Artificial intelligence of things for solar energy monitoring and control. *Applied Sciences*, 15(11):6019.
- [Chaychian et al., 2017] Chaychian, M., Asif, R., Mumtaz, S., Loo, J., and Jamalipour, A. (2017). Embedded trusted monitoring and management modules for smart solar panels. In *Proceedings of the IEEE International Conference on Smart Grid Communications (SmartGridComm)*, pages 251–256.

- [Deutsch et al., 2019] Deutsch, S., Christlein, V., Berger, S., Buerhop-Lutz, C., Maier, A., Gallwitz, F., and Riess, C. (2019). Automatic classification of defective photovoltaic module cells in electroluminescence images. Manuscript / preprint. Dataset and implementation discussed; venue not specified in PDF.
- [Docker, Inc., nd] Docker, Inc. (n.d.). Docker documentation. <https://docs.docker.com/>. Accessed 2026-03-29.
- [Duan et al., 2024] Duan, R., Chen, Q., Hu, Y., Qin, Y., and Li, W. (2024). A method for detecting photovoltaic panel faults using a drone equipped with a multispectral camera.
- [Espressif Systems, nd] Espressif Systems (n.d.). Esp32 documentation. <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/>. Accessed 2026-03-29.
- [FastAPI, nd] FastAPI (n.d.). Fastapi documentation. <https://fastapi.tiangolo.com/>. Accessed 2026-03-29.
- [Figma, nd] Figma (n.d.). Figma help center. <https://help.figma.com/>. Accessed 2026-03-29.
- [Forsyth and Ponce, 2011] Forsyth, D. A. and Ponce, J. (2011). *Computer Vision: A Modern Approach*. Pearson, Upper Saddle River, NJ, 2 edition.
- [Gan et al., 2017] Gan, H., Chaychian, S., Mallett, C., Pissanidis, G., et al. (2017). Embedded trusted monitoring and management modules for smart solar panels. In *Proceedings of a Conference (details unavailable)*. Conference paper; preprint available on ResearchGate.
- [GeeksforGeeks, 2025] GeeksforGeeks (2025). Computer vision tutorial. Last updated 06 Aug 2025.
- [GitHub, nd] GitHub (n.d.). Github documentation. <https://docs.github.com/>. Accessed 2026-03-29.
- [Google, nd] Google (n.d.). Tensorflow documentation. <https://www.tensorflow.org/>. Accessed 2026-03-29.
- [Hardonag, 2024] Hardonag, G. (2024). Machine learning architecture: What it is, components & types. LakeFS Blog.
- [IEA-PVPS, 2021] IEA-PVPS (2021). Iea photovoltaic power systems programme annual report 2021. Technical report, International Energy Agency - Photovoltaic Power Systems Programme.
- [Islam et al., 2023] Islam, M., Rashel, M. R., Ahmed, M. T., Islam, A. K. M. K., and Tlemçani, M. (2023). Artificial intelligence in photovoltaic fault identification and diagnosis: A systematic review. *Energies*, 16(21):7417.
- [Keras, nd] Keras (n.d.). Keras documentation. <https://keras.io/>. Accessed 2026-03-29.

- [Kirubakaran et al., 2022] Kirubakaran, V., Preethi, D. M. D., Arunachalam, U., Rao, Y. K. S. S., Gatasheh, M. K., Hoda, N., and Anbese, E. M. (2022). Infrared thermal images of solar pv panels for fault identification using image processing technique. *Journal (publisher repository: Henry Ford Health Scholarly Commons)*. Research article; repository version.
- [Martinez-Gil et al., 2025] Martinez-Gil, F., Sansom, C., Fernández-García, A., Alcayde-García, A., and Manzano-Agugliaro, F. (2025). Maintenance techniques to increase solar energy production: A review. *Energy Nexus*, 17:100384.
- [MDN Web Docs, nd] MDN Web Docs (n.d.). Javascript guide. <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide>. Accessed 2026-03-29.
- [Meta Platforms, Inc., nd] Meta Platforms, Inc. (n.d.). React documentation. <https://react.dev/>. Accessed 2026-03-29.
- [Microsoft, nda] Microsoft (n.d.a). Microsoft azure documentation. <https://learn.microsoft.com/azure/>. Accessed 2026-03-29.
- [Microsoft, ndb] Microsoft (n.d.b). Typescript documentation. <https://www.typescriptlang.org/docs/>. Accessed 2026-03-29.
- [Microsoft, ndc] Microsoft (n.d.c). Visual studio code documentation. <https://code.visualstudio.com/docs>. Accessed 2026-03-29.
- [Nazeer, 2018] Nazeer, F. (2018). Gamification & volunteerism: How gamification technology is used in volunteerism. Master’s thesis.
- [Netlify, nd] Netlify (n.d.). Netlify documentation. <https://docs.netlify.com/>. Accessed 2026-03-29.
- [OpenCV, nd] OpenCV (n.d.). Opencv documentation. <https://opencv.org/>. Accessed 2026-03-29.
- [Pal, 2025] Pal, S. K. (2025). Iterative waterfall model – software engineering. GeeksforGeeks.
- [Polymeropoulos et al., 2024] Polymeropoulos, I., Bezyrgiannidis, S., Vrochidou, E., and Papakostas, G. A. (2024). Enhancing solar plant efficiency: A review of vision-based monitoring and fault detection techniques. *Technologies*, 12(10):175.
- [PostgreSQL Global Development Group, nd] PostgreSQL Global Development Group (n.d.). Postgresql documentation. <https://www.postgresql.org/docs/>. Accessed 2026-03-29.
- [Python Software Foundation, nd] Python Software Foundation (n.d.). Python documentation. <https://docs.python.org/3/>. Accessed 2026-03-29.
- [pythonafroz, 2023a] pythonafroz (2023a). Solar panel clean and faulty images. Dataset.

- [pythonafroz, 2023b] pythonafroz (2023b). Solar panel images. Dataset.
- [Qureshi et al., 2024] Qureshi, M. S., Nawaz, M. U., et al. (2024). Machine learning for predictive maintenance in solar farms: Introduction. Preprint. Available on ResearchGate.
- [rxhulshxrmx, 2025] rxhulshxrmx (2025). Solar panels performance. Dataset.
- [Salazar and Macabebe, 2016] Salazar, A. M. and Macabebe, E. Q. B. (2016). Hotspots detection in photovoltaic modules using infrared thermography.
- [scikit-learn developers, nd] scikit-learn developers (n.d.). scikit-learn documentation. <https://scikit-learn.org/stable/documentation.html>. Accessed 2026-03-29.
- [Shaik et al., 2025] Shaik, A. H., Narayana, D. S., Zehra, I., and Lavanya, V. (2025). Data-driven forecasting of solar pv output using machine learning: A comprehensive approach for long-term prediction. *Advances in Artificial Intelligence and Machine Learning: Research*, 5(1):189. Open access article; exact page range varies by publisher layout.
- [Sommerville, 2016] Sommerville, I. (2016). *Software Engineering*. Pearson, Boston, MA, 10 edition.
- [SQLAlchemy, nd] SQLAlchemy (n.d.). Ssqlalchemy documentation. <https://docs.sqlalchemy.org/>. Accessed 2026-03-29.
- [Supabase, nd] Supabase (n.d.). Supabase documentation. <https://supabase.com/docs>. Accessed 2026-03-29.
- [Sutikno et al., 2021] Sutikno, T., Purnama, H. S., Pamungkas, A., Fadlil, A., Alsofyani, I. M., and Jopri, M. H. (2021). Internet of things-based photovoltaics parameter monitoring system using nodemcu esp8266. *International Journal of Electrical and Computer Engineering (IJECE)*, 11(6):5578–5587.
- [Tailwind Labs, nd] Tailwind Labs (n.d.). Tailwind css documentation. <https://tailwindcss.com/docs>. Accessed 2026-03-29.
- [Ultralytics, nd] Ultralytics (n.d.). Ultralytics yolo documentation. <https://docs.ultralytics.com/>. Accessed 2026-03-29.
- [Vite, nd] Vite (n.d.). Vite documentation. <https://vite.dev/guide/>. Accessed 2026-03-29.
- [whereareyousolarpanel, 2023] whereareyousolarpanel (2023). solar-pv-panel-detection dataset. <https://universe.roboflow.com/whereareyousolarpanel/solar-pv-panel-detection>. CV Dataset.
- [Wokwi, 2026] Wokwi (2026). Solarsense prototype. <https://wokwi.com/projects/458504413474926593>. Chip Design.

- [Xie et al., 2024] Xie, H., Yuan, B., Hu, C., Gao, Y., Wang, F., Wang, C., Wang, Y., and Chu, P. (2024). St-yolo: A defect detection method for photovoltaic modules based on infrared thermal imaging and machine vision technology. *PLOS ONE*, 19(12):e0310742.
- [Zheng et al., 2022] Zheng, Q., Ma, J., Liu, M., Liu, Y., Li, Y., and Shi, G. (2022). Lightweight hot-spot fault detection model of photovoltaic panels in uav remote-sensing image. *Sensors*, 22(12):4617.

APPENDIX 1

Stakeholder Survey Questions

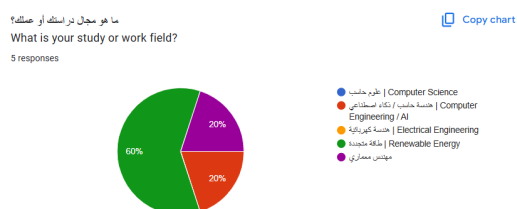


Figure 1.1: Study or work field

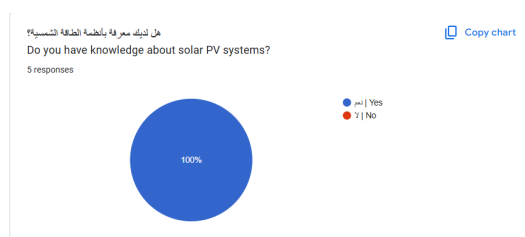


Figure 1.2: Knowledge about solar PV systems

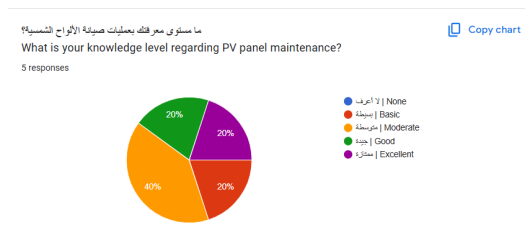


Figure 1.3: Knowledge level regarding PV panel maintenance

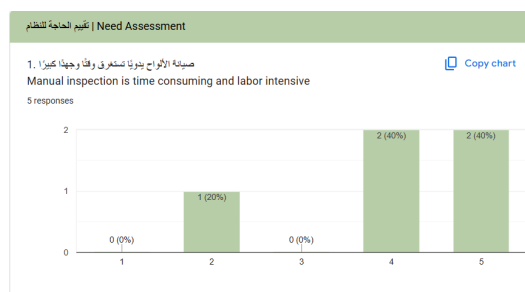


Figure 1.4: Manual inspection is time consuming and labour intensive

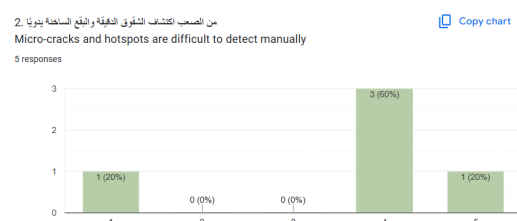


Figure 1.5: Micro-cracks and hotspots are difficult to detect manually

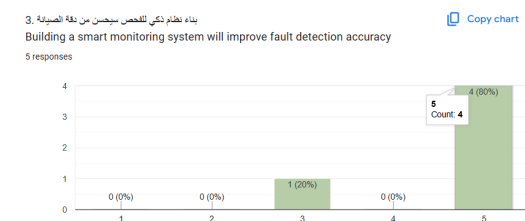


Figure 1.6: Smart monitoring will improve fault detection accuracy

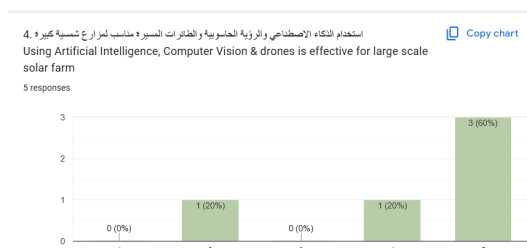


Figure 1.7: AI, Computer Vision & drones are effective for large-scale solar farms

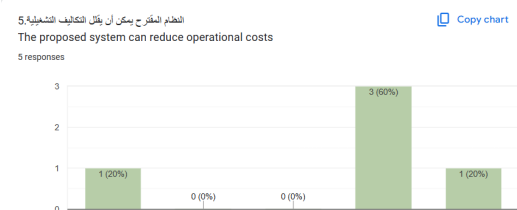


Figure 1.8: The proposed system can reduce operational costs

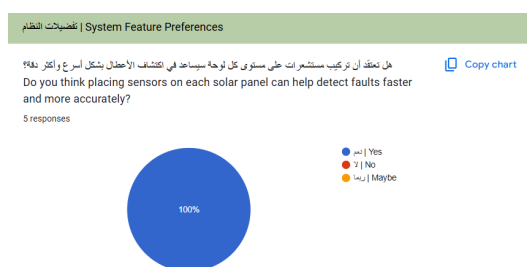


Figure 1.9: Sensors on each panel help detect faults faster and more accurately



Figure 1.10: Using a drone for inspection is an effective solution

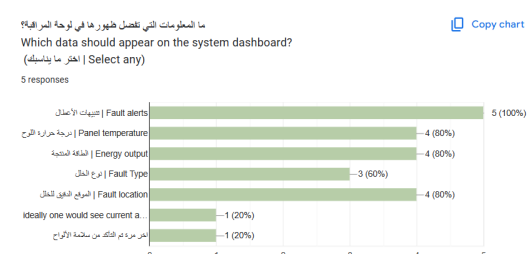


Figure 1.11: Data that should appear on the system dashboard

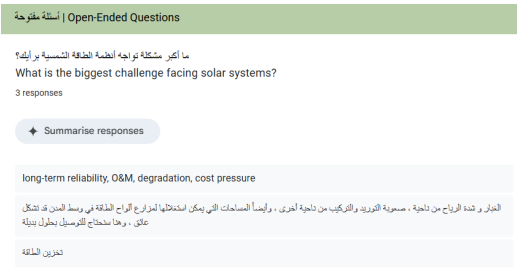


Figure 1.12: The biggest challenge facing solar systems

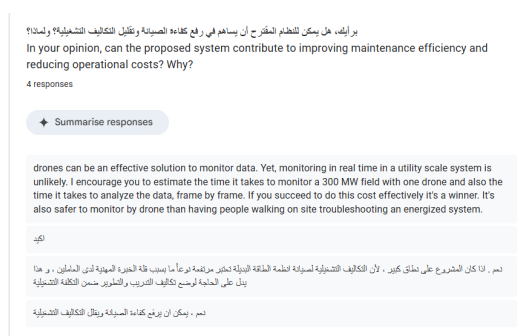


Figure 1.13: Contribution to improving maintenance efficiency and reducing costs

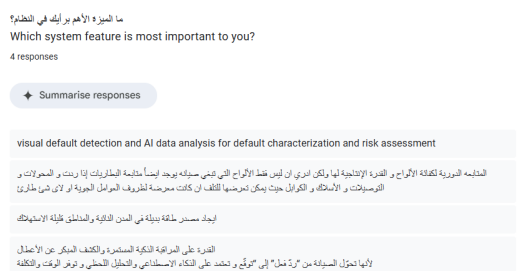


Figure 1.14: Most important system feature

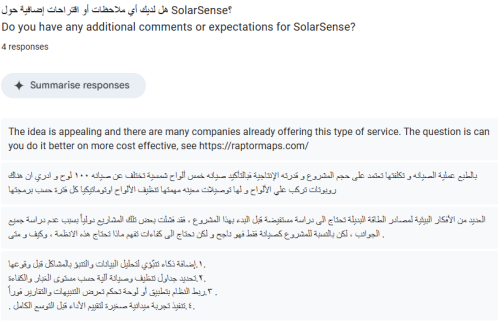


Figure 1.15: Additional comments or expectations for SolarSense

APPENDIX 2

Stakeholder Interview Questions

A) Solar / Electrical Engineer Interview Questions (PV / Maintenance)

1. What are the most common issues that you usually observe in solar panels (e.g., dust, cracks, hotspots, wiring problems)?

Hotspots, Shading, and dust sometimes

2. How are these issues typically detected today? (Manual inspection / performance comparison / thermal camera / drones)?

Mostly the first way to detect the fault mostly through SCADA system
The other way is sometimes you can observe the output of the panel

3. Do you think using sensors on each panel is practical, or is monitoring on the sting level realistic? Why?

It depends on the size of the project and what we're looking for For your project I recommended by panel

4. Which electrical measurements are the most important to monitor panel health? (Voltage, current, temperature, irradiance)?

Mostly Voltage, current because the power will improve from these two
Also temperature, irradiance, and humidity

5. In your experience, when is drone inspection more useful than ground

inspection?

drone is easier in case the

6. What challenges do you expect when installing monitoring hardware in real outdoor PV sites?

First full the accuracy of the sensors, the accuracy of the data, and maybe an unpredicted behavior of the panel

B) Embedded System / Hardware Engineer Interview Questions

7. Which type of sensor is most suitable for measuring panel voltage and current in a stable and reliable way?

Mostly the current sensor and the voltage sensor but sometimes the problem is If you looking for more accurate sensor it could be the cost problem

8. Is it better to send sensor data continuously, or store it locally and send it in intervals? Why?

Both ways real time monitoring plus the stored data because for real time data you can monitor the health of the PV like by time and for the stored data you can predict what is going to happen in the future

9. If the chip is powered from the panel itself, how can we ensure stable power during early morning / late evening?

Storage System

10. What environmental protections should be considered for the hardware installed on the rooftop (heat, dust, humidity, rain)?

First of all, you need to be careful with the high temperatures because if you just leaves it on the roof it could face very high temperatures that effect the prototype

Also, should be in a safe place away from motor away from dust away

from people interacting because sometimes the Montanans people could harm the project

11. Do you recommend using Wi-Fi, LoRa, or wired connection for panel communication? Why?

Wi-Fi is good but you need to be careful with the continues data, because is an interruption happened in the Wi-Fi the data will be damaged or lost

APPENDIX 3

Usability Testing Survey

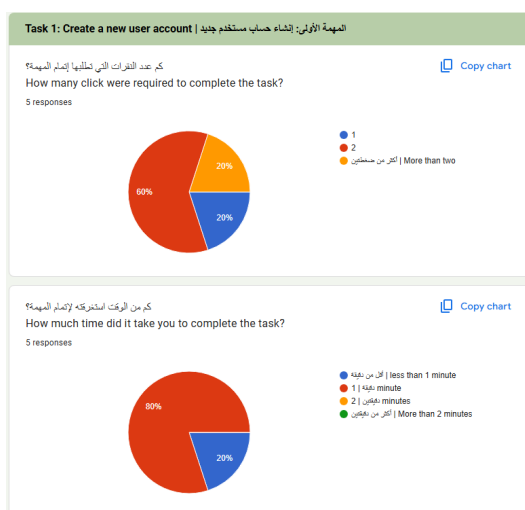


Figure 3.1: Task 1 Part 1 Results

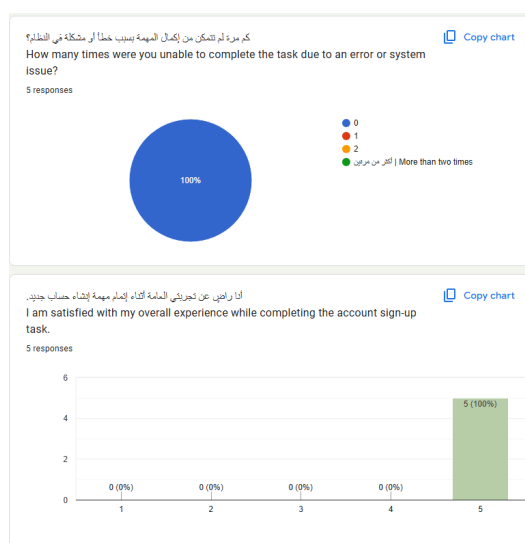


Figure 3.2: Task 1 Part 2 Results

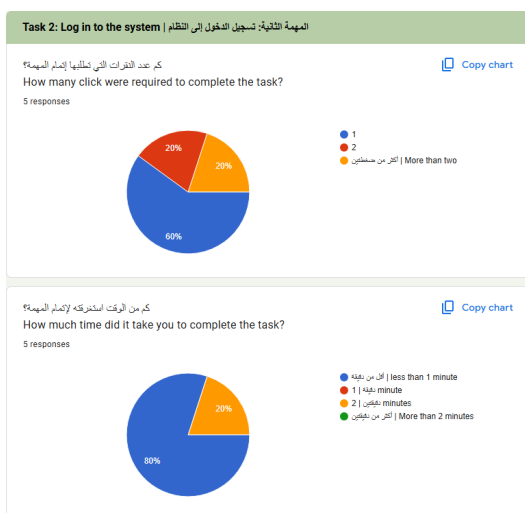


Figure 3.3: Task 2 Part 1 Results

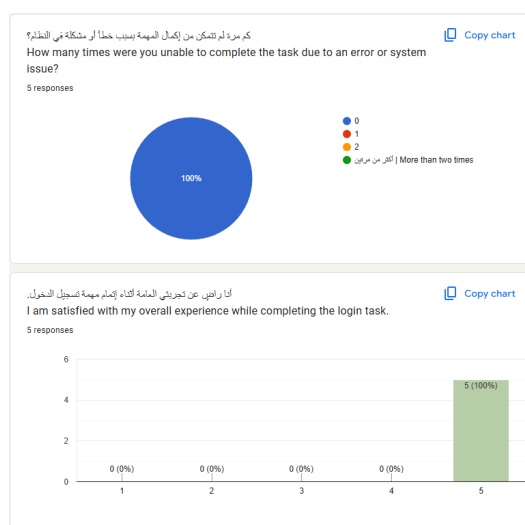


Figure 3.4: Task 2 Part 2 Results

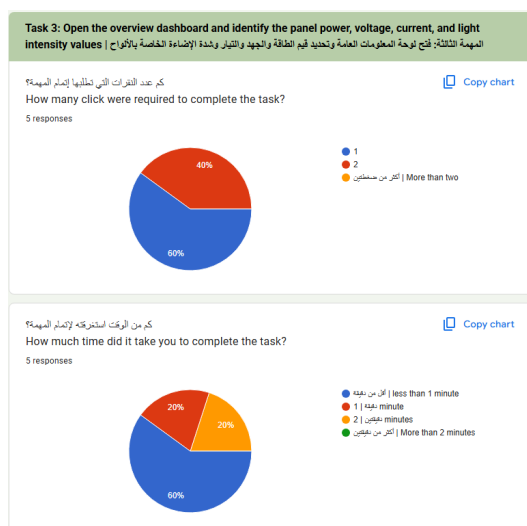


Figure 3.5: Task 3 Part 1 Results

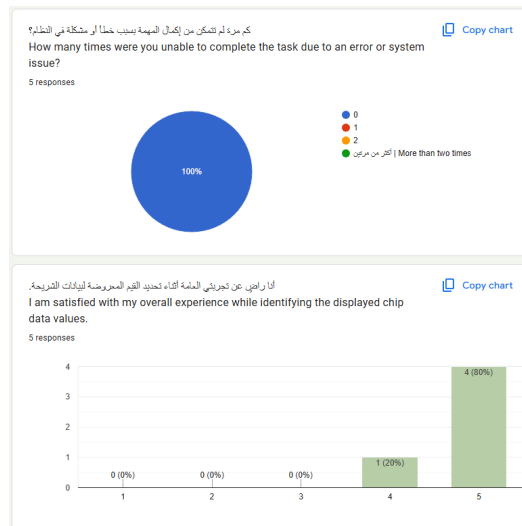


Figure 3.6: Task 3 Part 2 Results

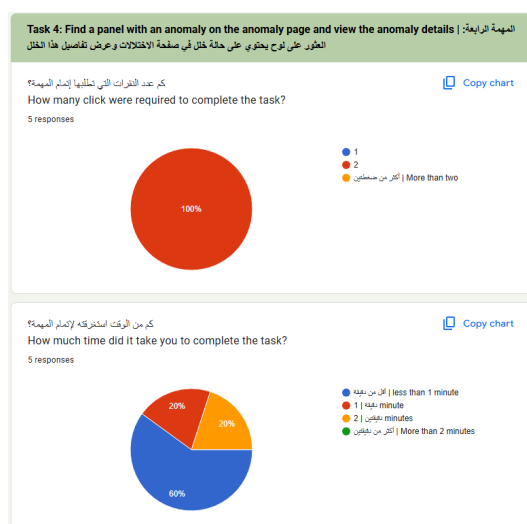


Figure 3.7: Task 4 Part 1 Results

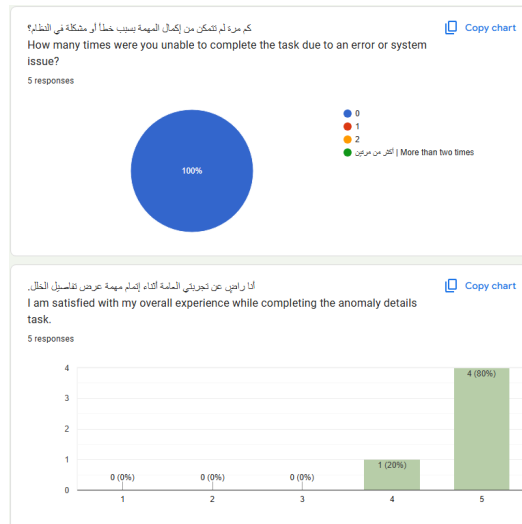


Figure 3.8: Task 4 Part 2 Results

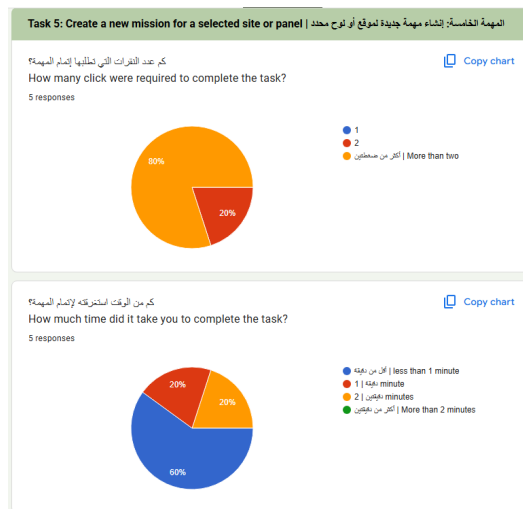


Figure 3.9: Task 5 Part 1 Results

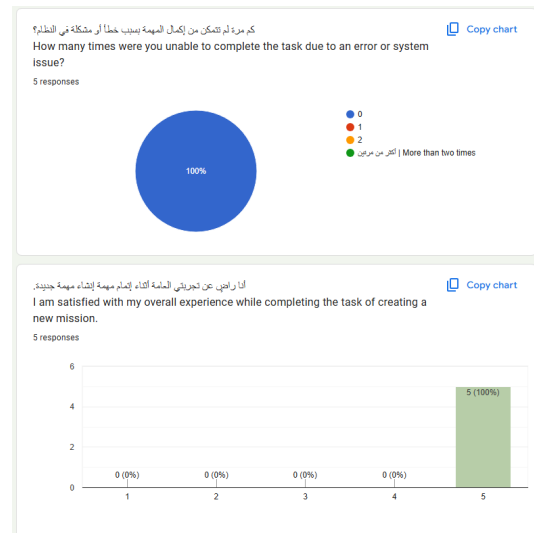


Figure 3.10: Task 5 Part 2 Results

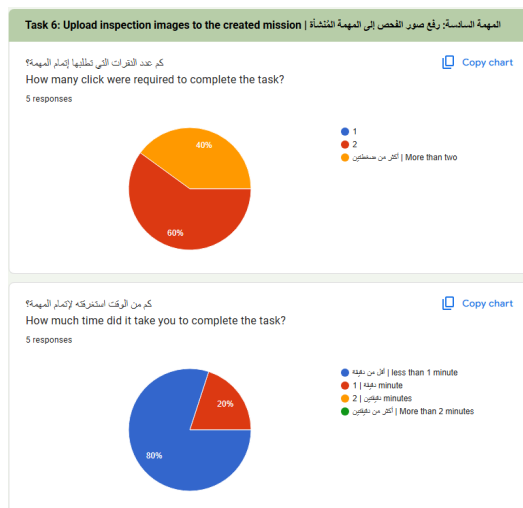


Figure 3.11: Task 6 Part 1 Results

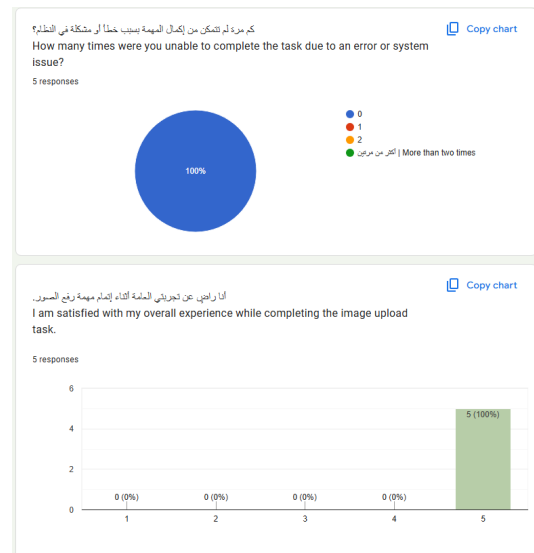


Figure 3.12: Task 6 Part 2 Results

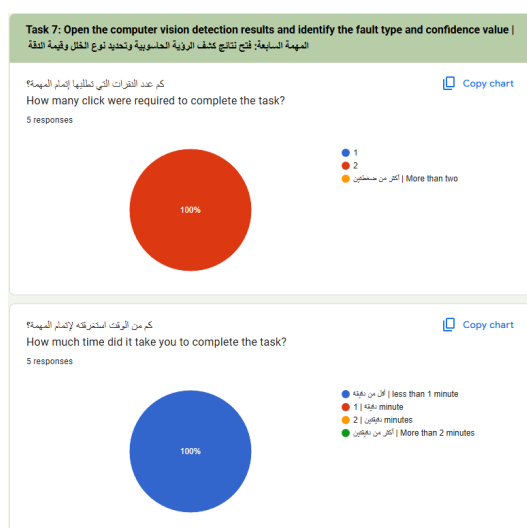


Figure 3.13: Task 7 Part 1 Results

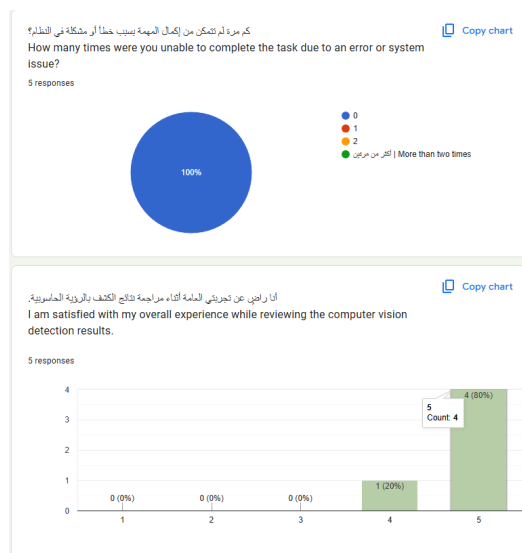


Figure 3.14: Task 7 Part 2 Results

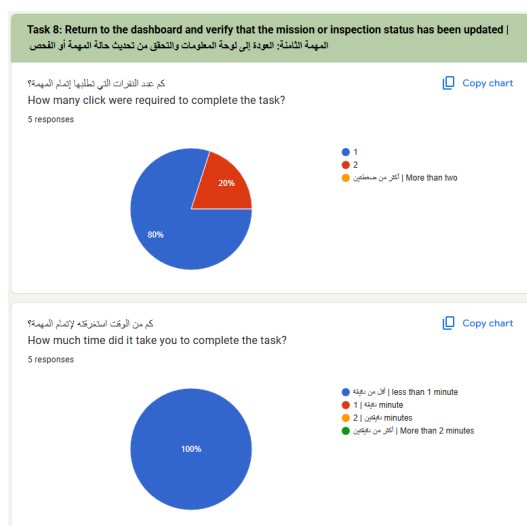


Figure 3.15: Task 8 Part 1 Results

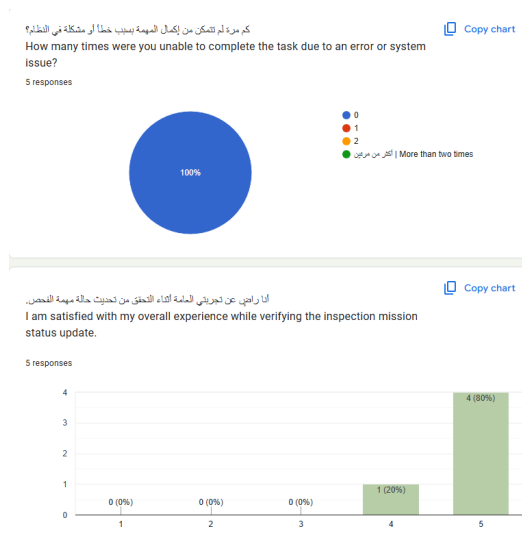


Figure 3.16: Task 8 Part 2 Results

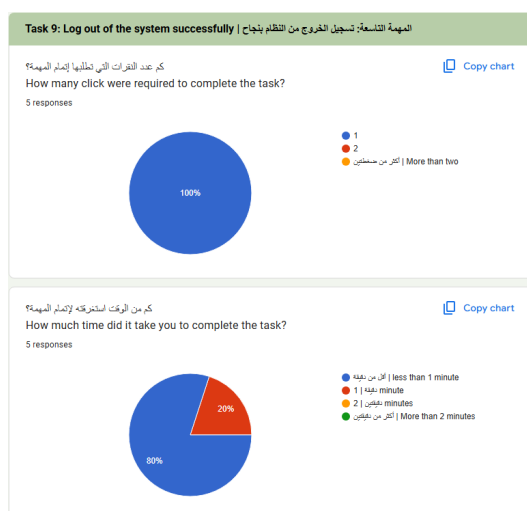


Figure 3.17: Task 9 Part 1 Results

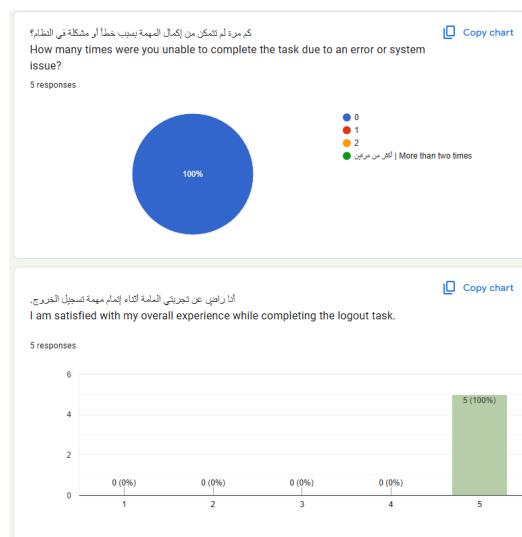


Figure 3.18: Task 9 Part 2 Results

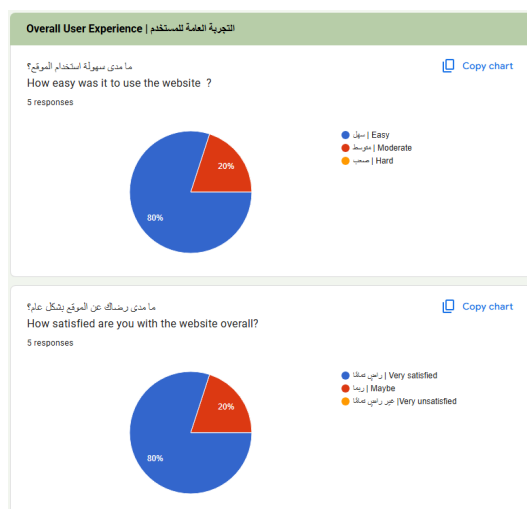


Figure 3.19: Overall User Experience Results Part 1

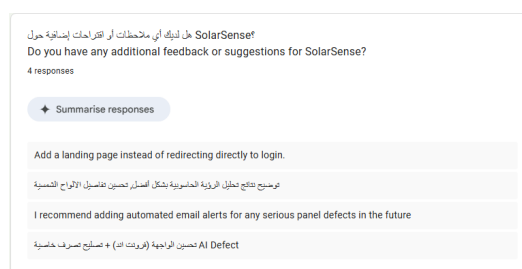


Figure 3.20: Overall User Experience Results Part 1